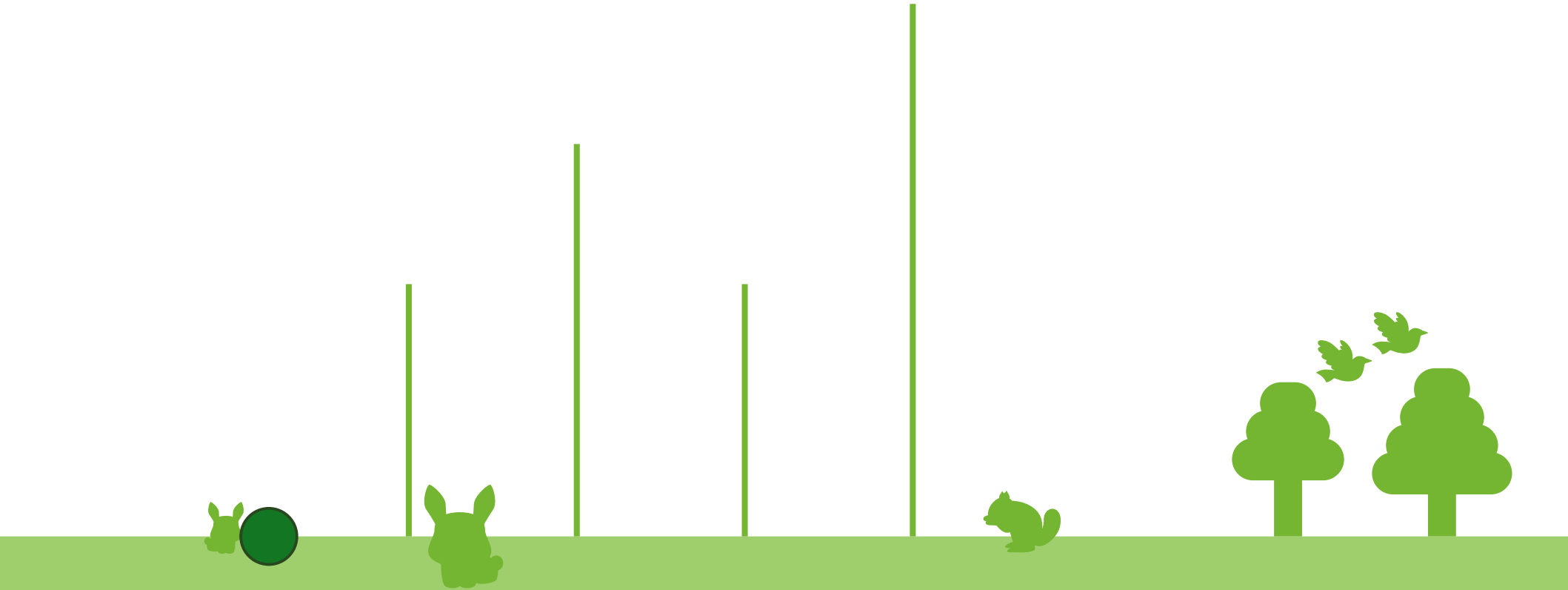


ロケット打ち上げ解説

松尾 凜太郎

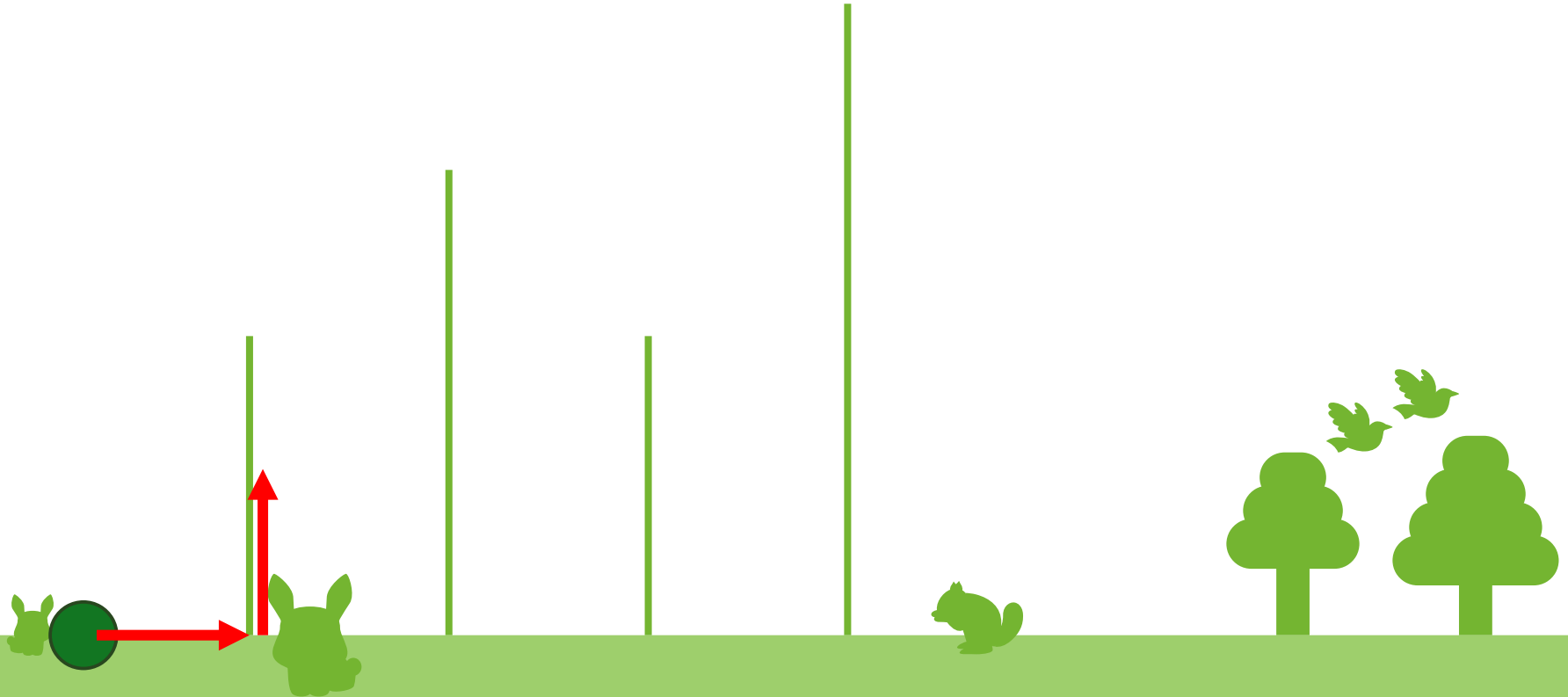


問題概要



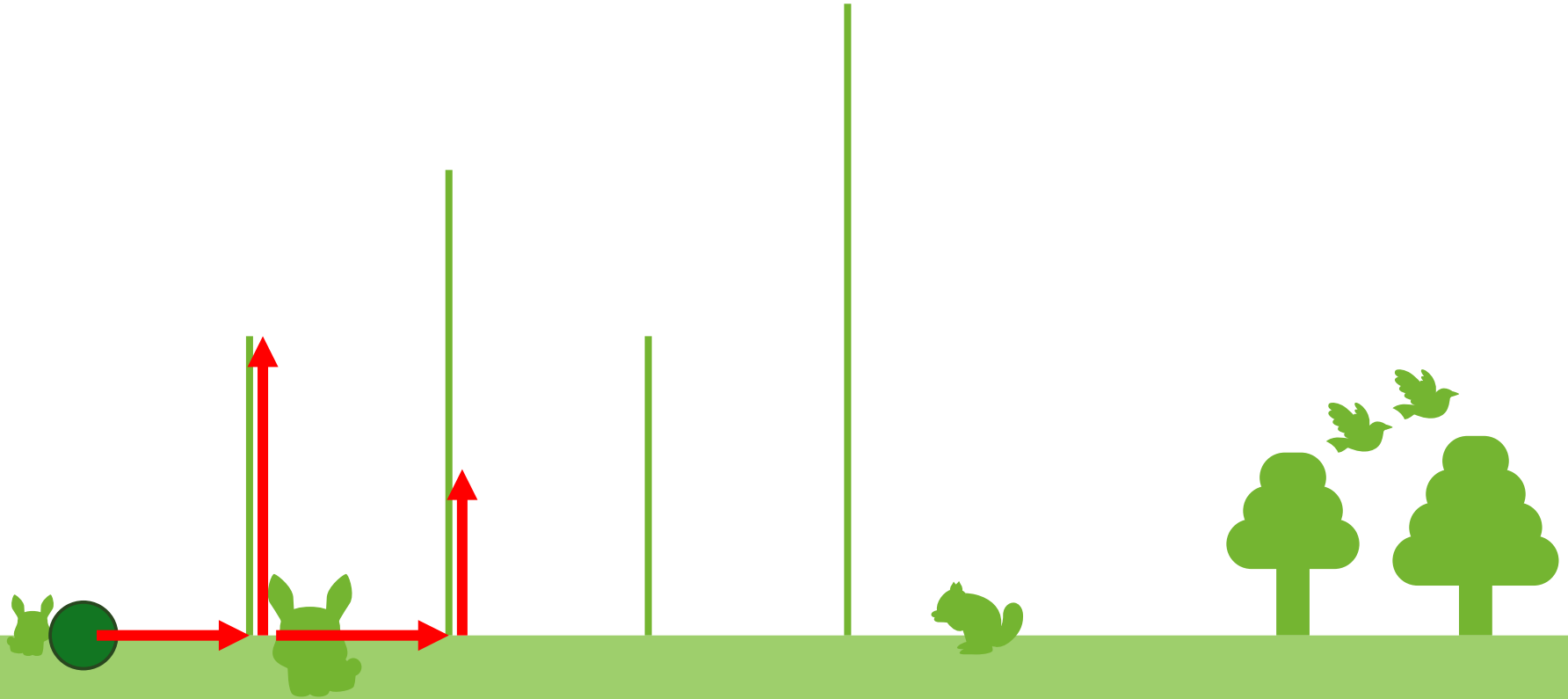
問題概要

2 ビヨウのとき



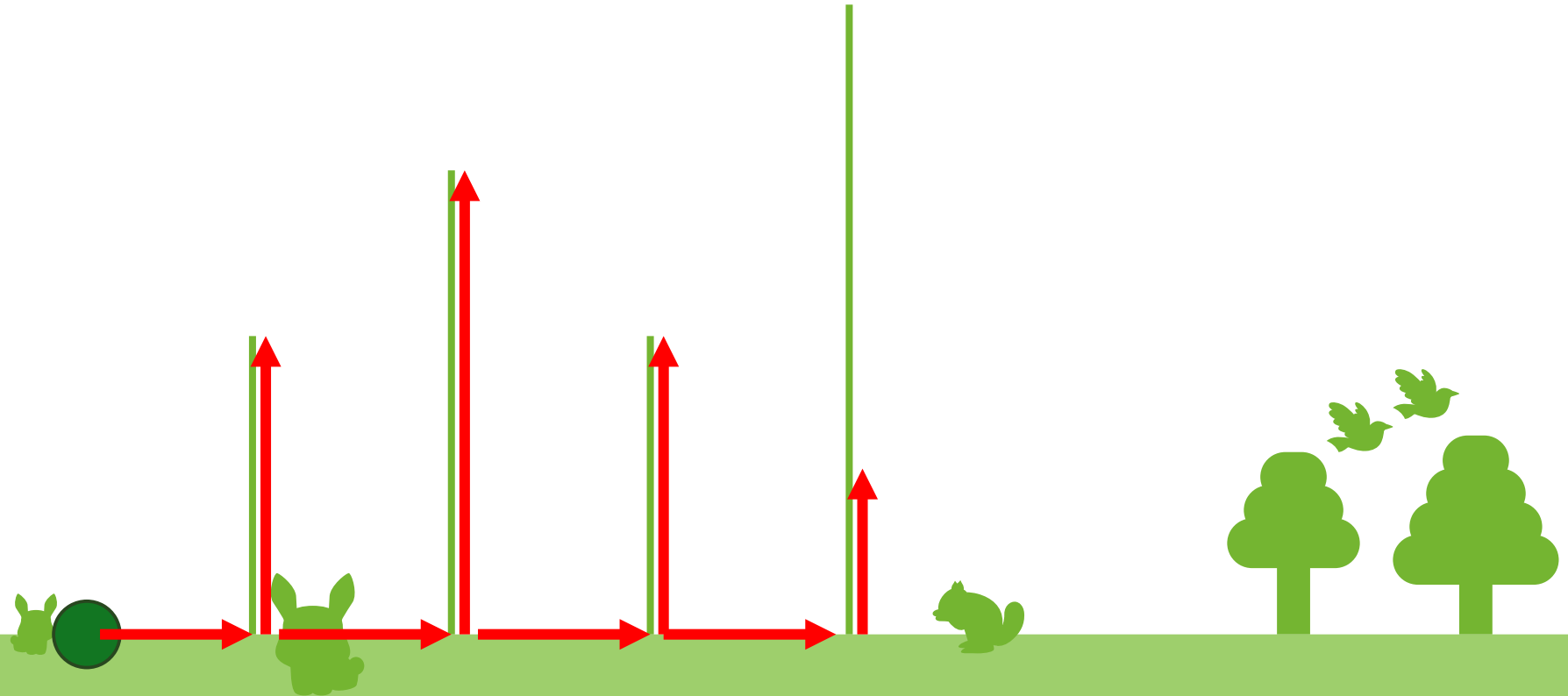
問題概要

3 ビヨウのとき



問題概要

5 ビヨウのとき



小課題 1 (10 点)

上るビルを1つ選ぶ問題

j 番目のクエリで i 番目のビルを選ぶと

$\max(0, \min(H_i, T_j - X_i))$

まで上れる

各クエリについて、N 個のビルを全探索すると $O(NQ)$

N, Q ≤ 1000 では間に合う



小課題 2 (15 点)

$X_i \leq 30$

ビルが全部近い
調べるビルを減らせないか



小課題 2 (15 点)

$X_i \leq 30$

ビルが全部近い
調べるビル数を減らせないか

→ X 座標が同じなら高いビルの方がお得 (少なくとも損はしない)

同じ X_i ごとに H_i 最大のビルだけ調べると30個しか調べなくてよい: $O(N + Q \max X_i)$



小課題 3 (16 点)

$$H_i \leq 30$$

ビルが全部低い
調べるビル数を減らせないか

→ 同じ高さなら近いビルの方がお得 (少なくとも損はしない)

同じ H_i ごとに X_i 最小のビルだけ調べると30個しか調べなくてよい: $O(N + Q \max H_i)$



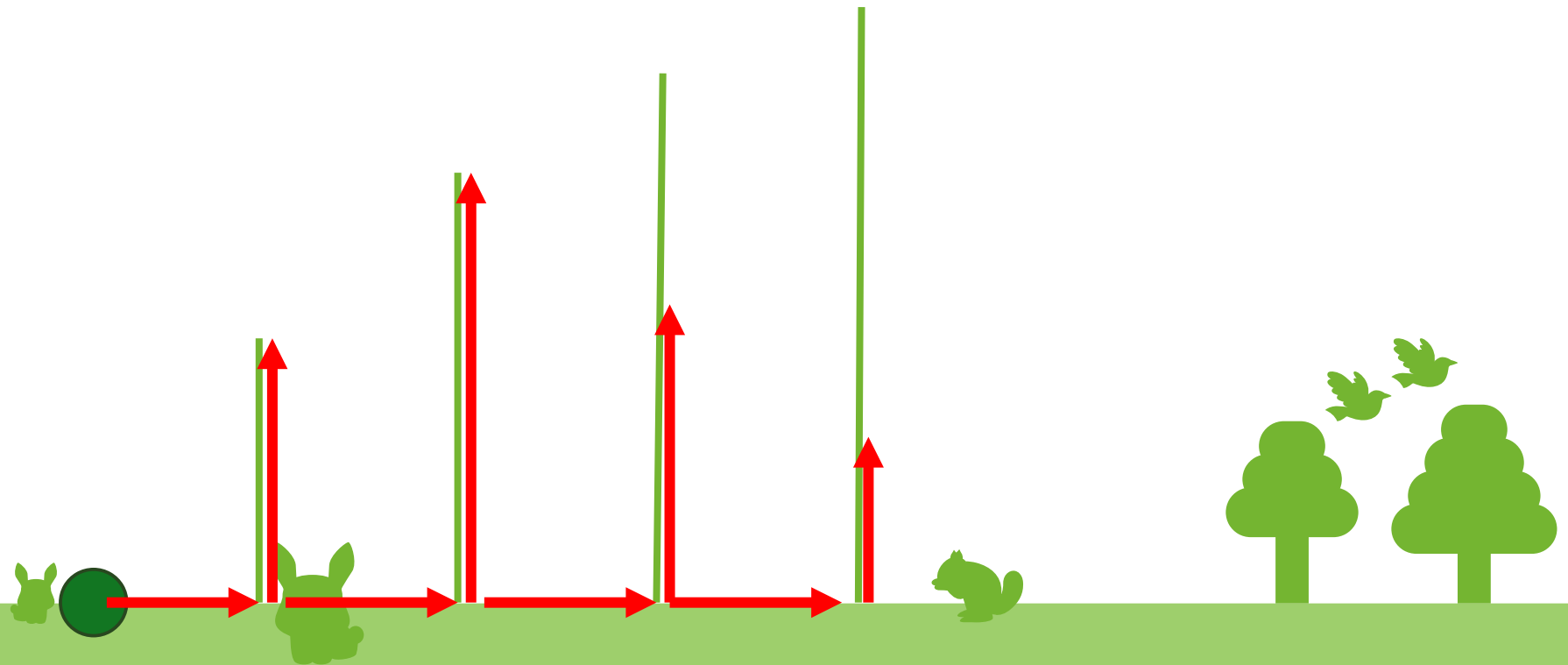
小課題 4 (29 点)

X_i, H_i が単調増加

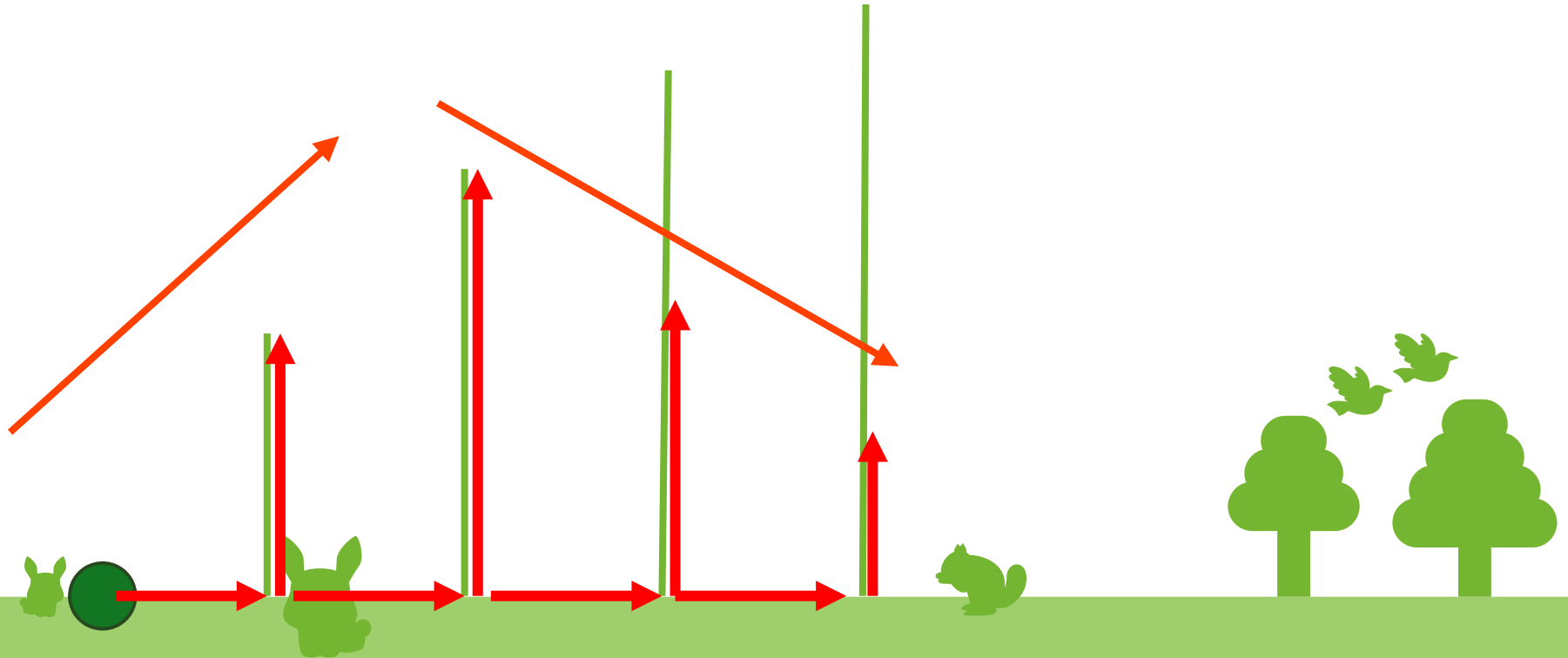
調べるビル数を減らせないか



小課題 4 (29 点)



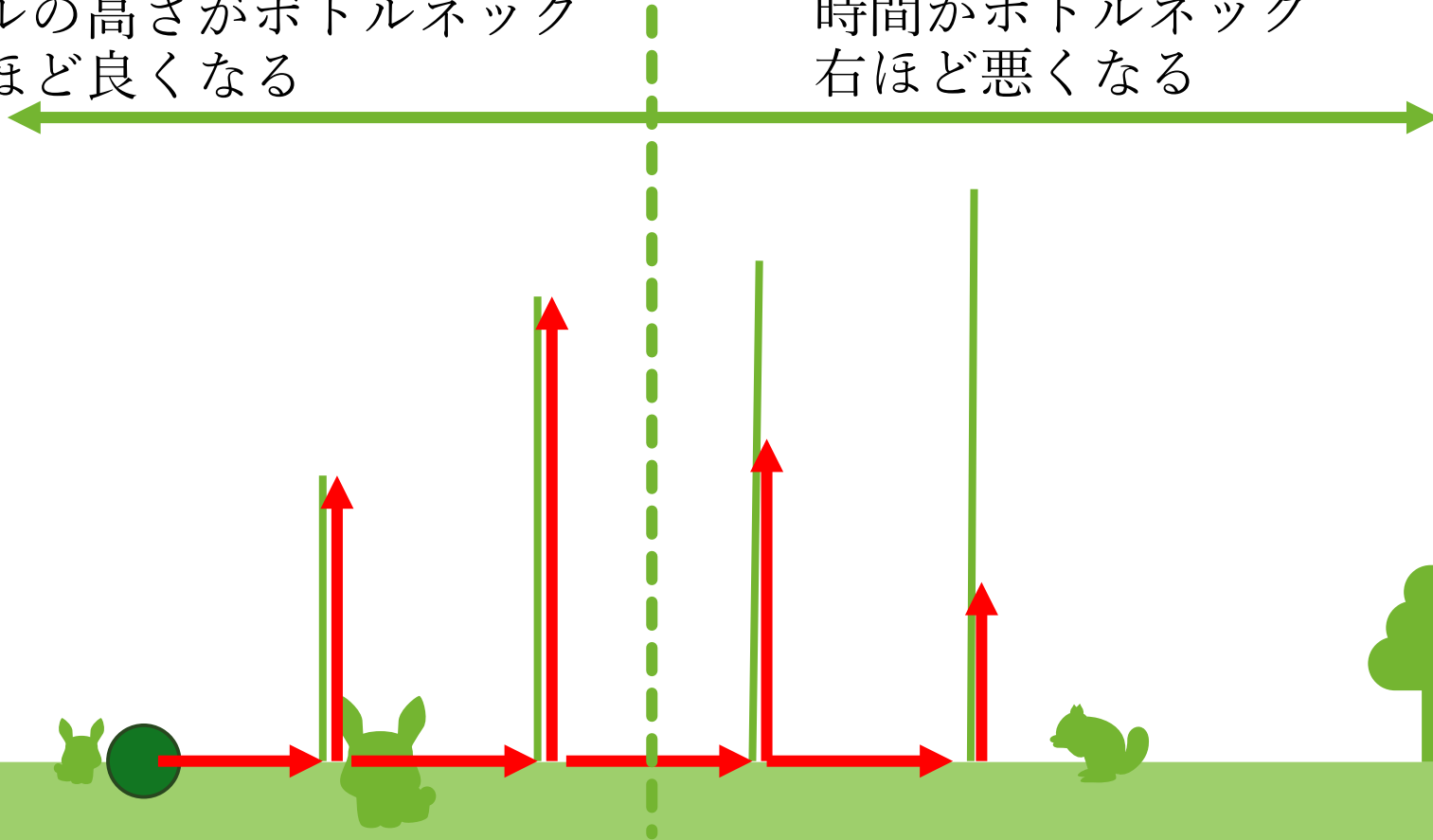
小課題 4 (29 点)



小課題 4 (29 点)

ビルの高さがボトルネック
右ほど良くなる

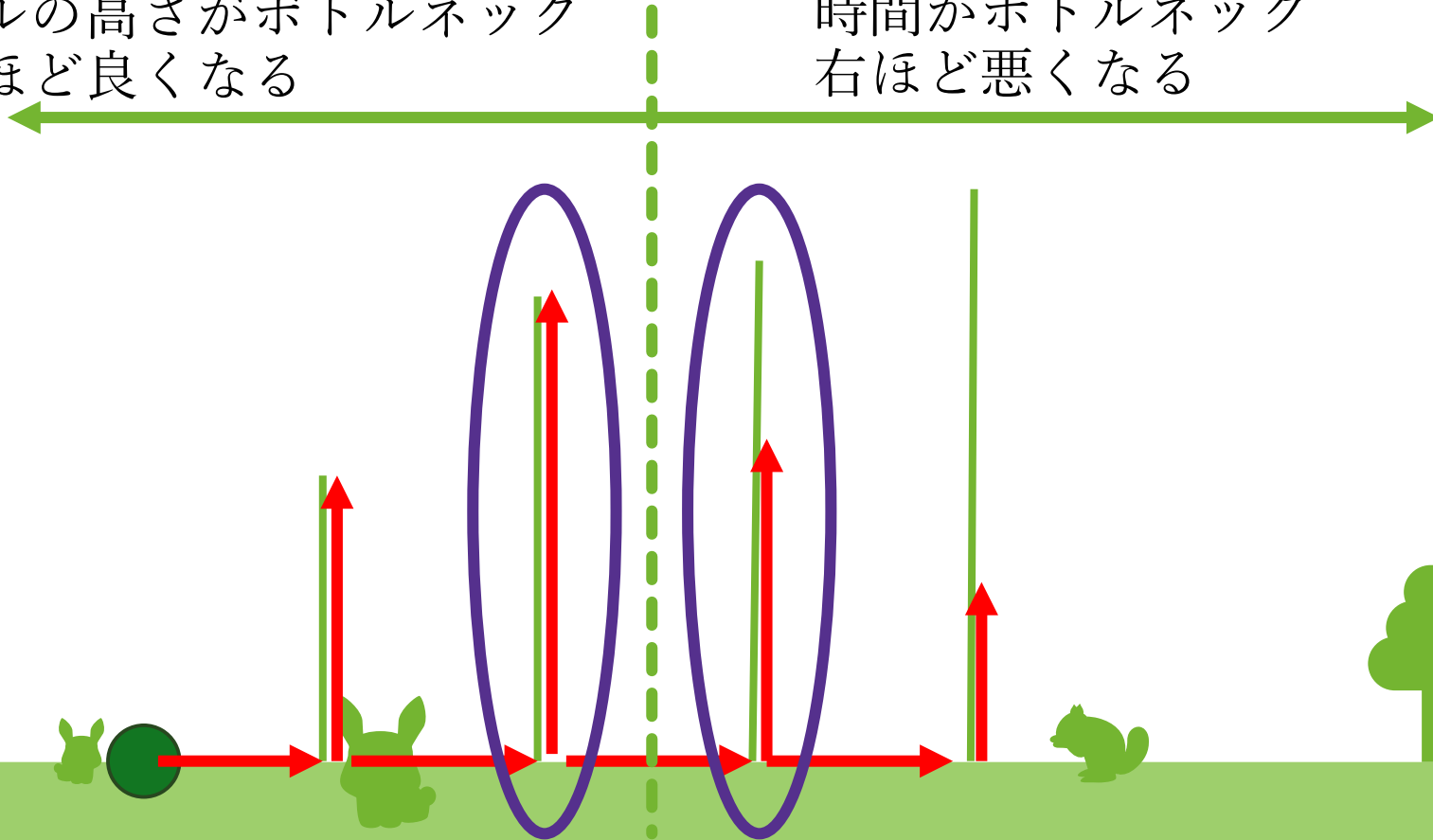
時間がボトルネック
右ほど悪くなる



小課題 4 (29 点)

ビルの高さがボトルネック
右ほど良くなる

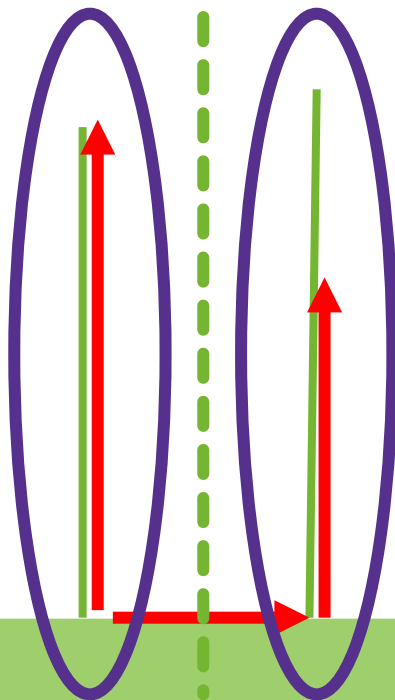
時間がボトルネック
右ほど悪くなる



小課題 4 (29 点)

ビルの高さがボトルネック
右ほど良くなる

時間がボトルネック
右ほど悪くなる



- 頂上まで行ける一番右のビル
 - 頂上まで行けない一番左のビル
- のどちらかが最適



小課題 4 (29 点)

- 頂上まで行ける一番右のビル
 - 頂上まで行けない一番左のビル
- のどちらかが最適

→ クエリ毎に $X_i + H_i \leq T_j$ かどうかで二分探索をする

$O(N + Q \log(N))$



小課題 5 (満点)

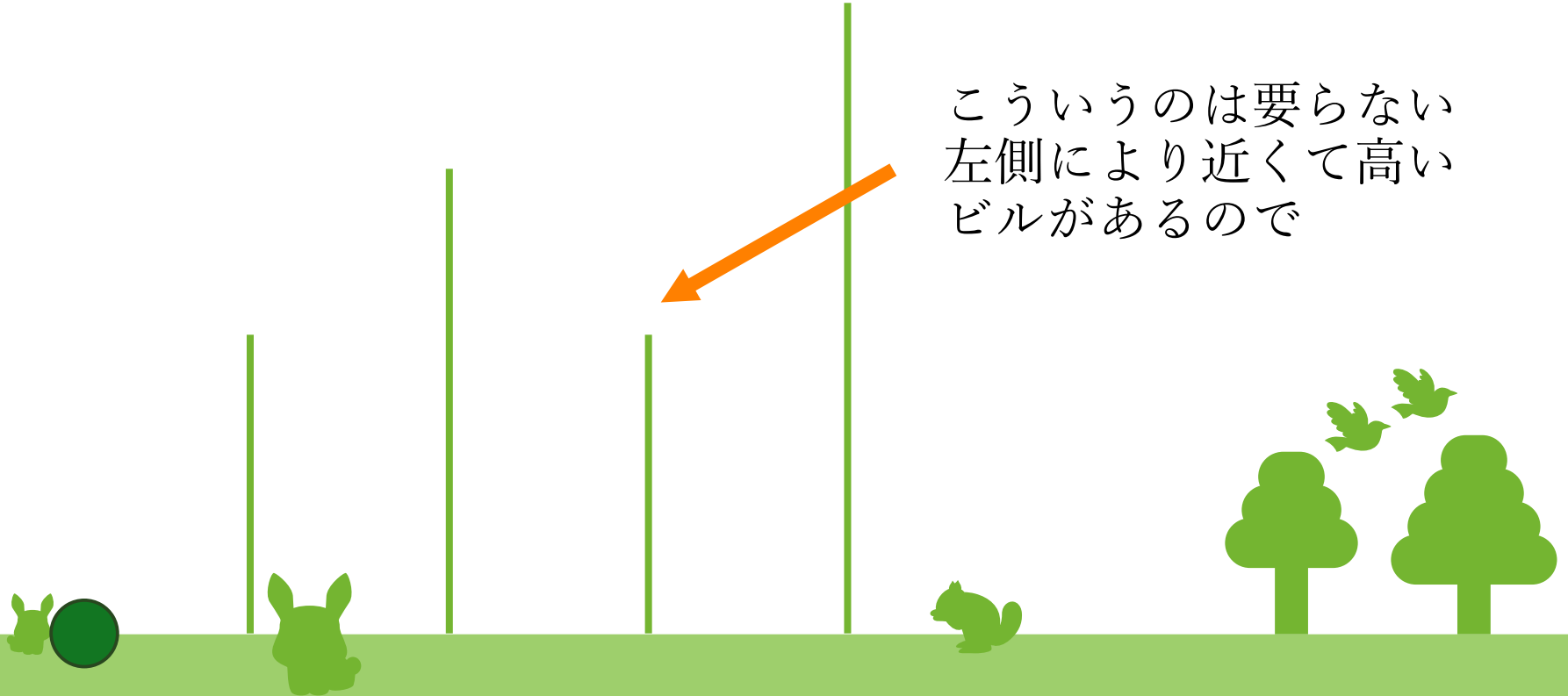
X_i, H_i 単調増加の制限がなくなった



小課題 5 (満点)

X_i, H_i 単調増加の制限がなくなった… が、

こういうのは要らない
左側により近くて高い
ビルがあるので



小課題 5 (満点)

X_i, H_i 単調増加の制限がなくなった… が、

結局単調増加を乱すビルを全部取り除いてよい
→ 小課題4と同じ

X_i が単調増加じゃなくなっているので最初にソートを忘れない
ように

$$O((N + Q)\log(N))$$



別解その1

(小課題4)

途中までは増えて、そこから下がることさえ分かれば

- クエリ毎三分探索
- クエリ毎、隣接差分で二分探索

など



別解その2

クエリごとに答えを二分探索

高さ x まで上れる条件は...

- 高さ x 以上のビルが、距離 $T_j - x$ 以内にある

距離 d 以内のビルの高さの最大値は？というクエリに答えられればよい

左から見てビルの高さの最大値が更新されるところだけ取り出しておいて、クエリ毎に二分探索

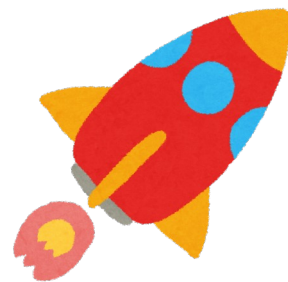
$O(N \log(N) + Q \log(N) \log(\max H_i))$



得点分布



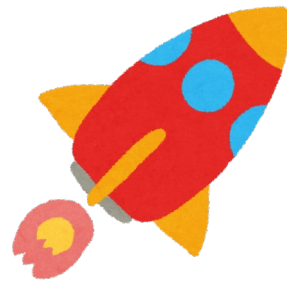
得点分布



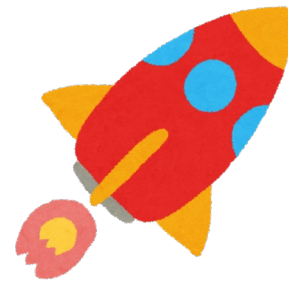
25



得点分布



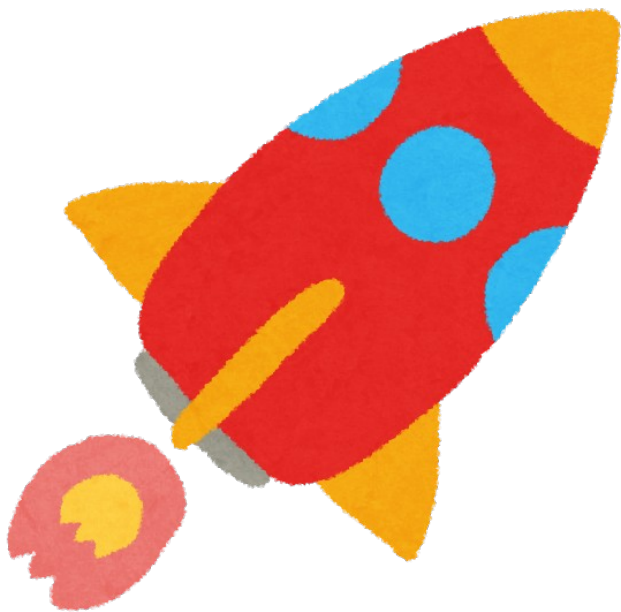
70



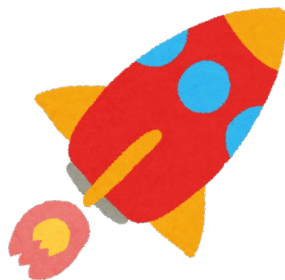
25



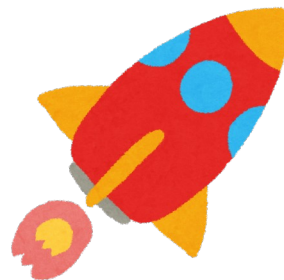
得点分布



100

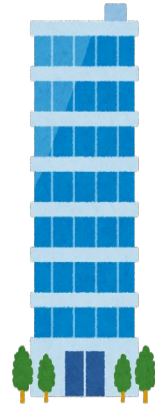
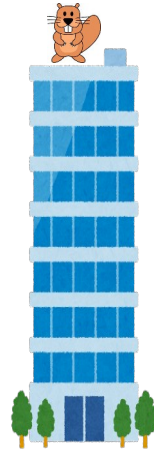
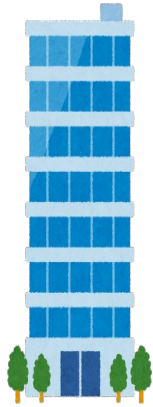
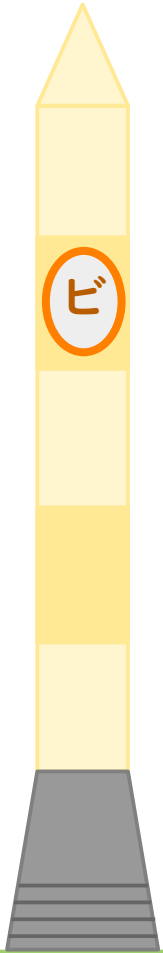


70



25





テレポーター (Teleporter)

解説：星井智仁

問題概要

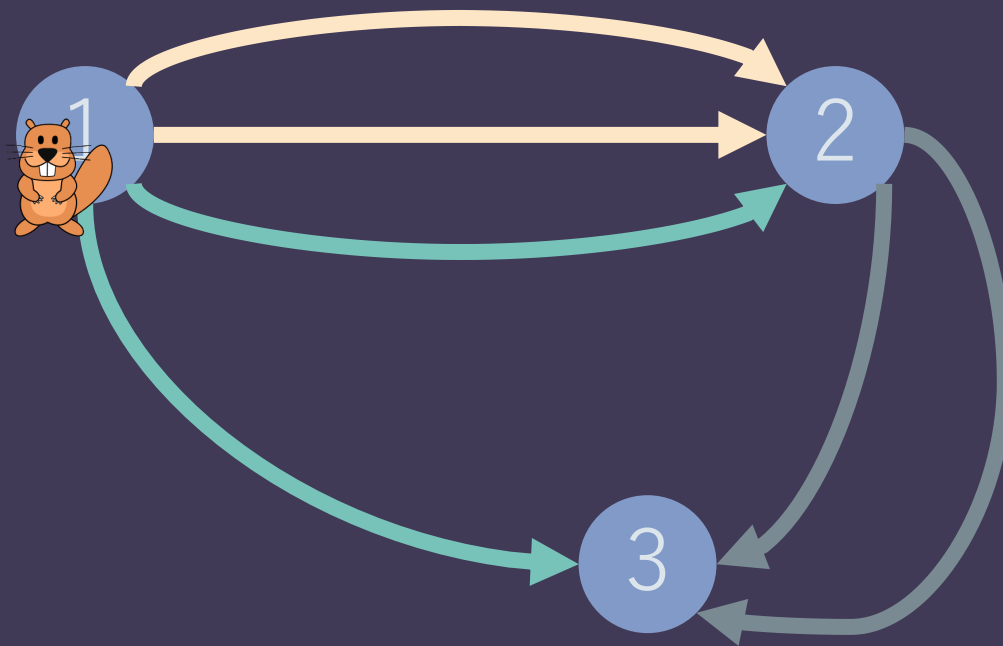
ビ太郎は最初、部屋 1 にいる

以下のラウンドを繰り返す. ビ太郎は行われるラウンドの回数をなるべく大きく, ビバ子は回数をなるべく小さくしたい

1. ビ太郎が部屋にあるテレポーターのうち 1 つを選ぶ
 2. ビバ子がそのテレポーターの行き先を部屋 $P_{i,j}$, 部屋 $Q_{i,j}$ のいずれかに設定する
 3. ビ太郎がその部屋に移動する
 4. もしビ太郎が部屋 N に移動したならゲームは終了する
- 無限ラウンド続くなならそれも判定せよ

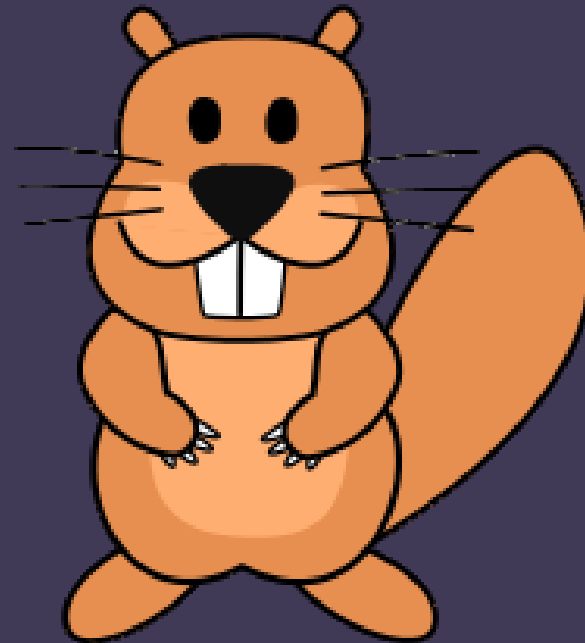
問題概要

サンプル 1



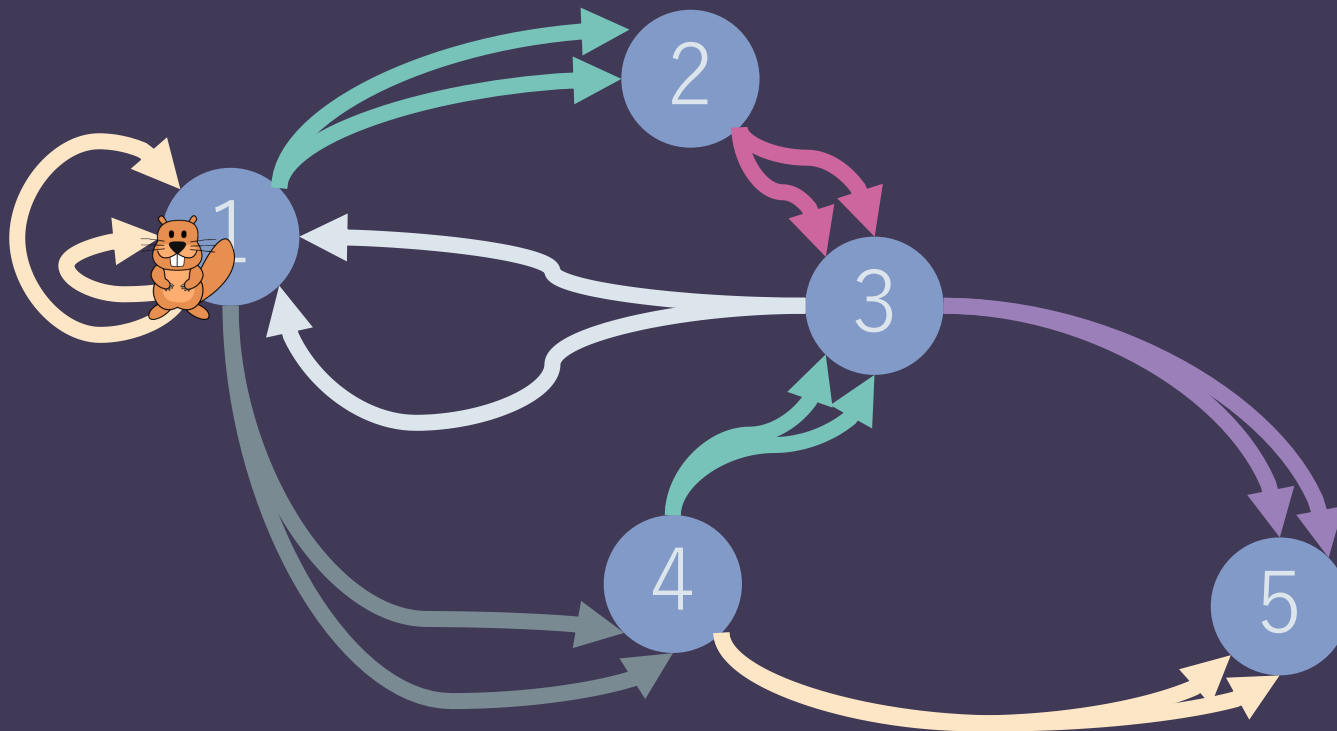
小課題

1. $P_{i,j} = Q_{i,j}$
2. $P_{i,j} > i, Q_{i,j} > i$
3. $\sum A_i \leq 2000$
4. $A_i = 1$
5. 追加の制約はない



小課題1 ($P_{i,j}=Q_{i,j}$)

サンプル 3



小課題1 ($P_{i,j}=Q_{i,j}$)

$P_{i,j}=Q_{i,j}$ が意味すること

→ ビバ子 の意思が関与できない

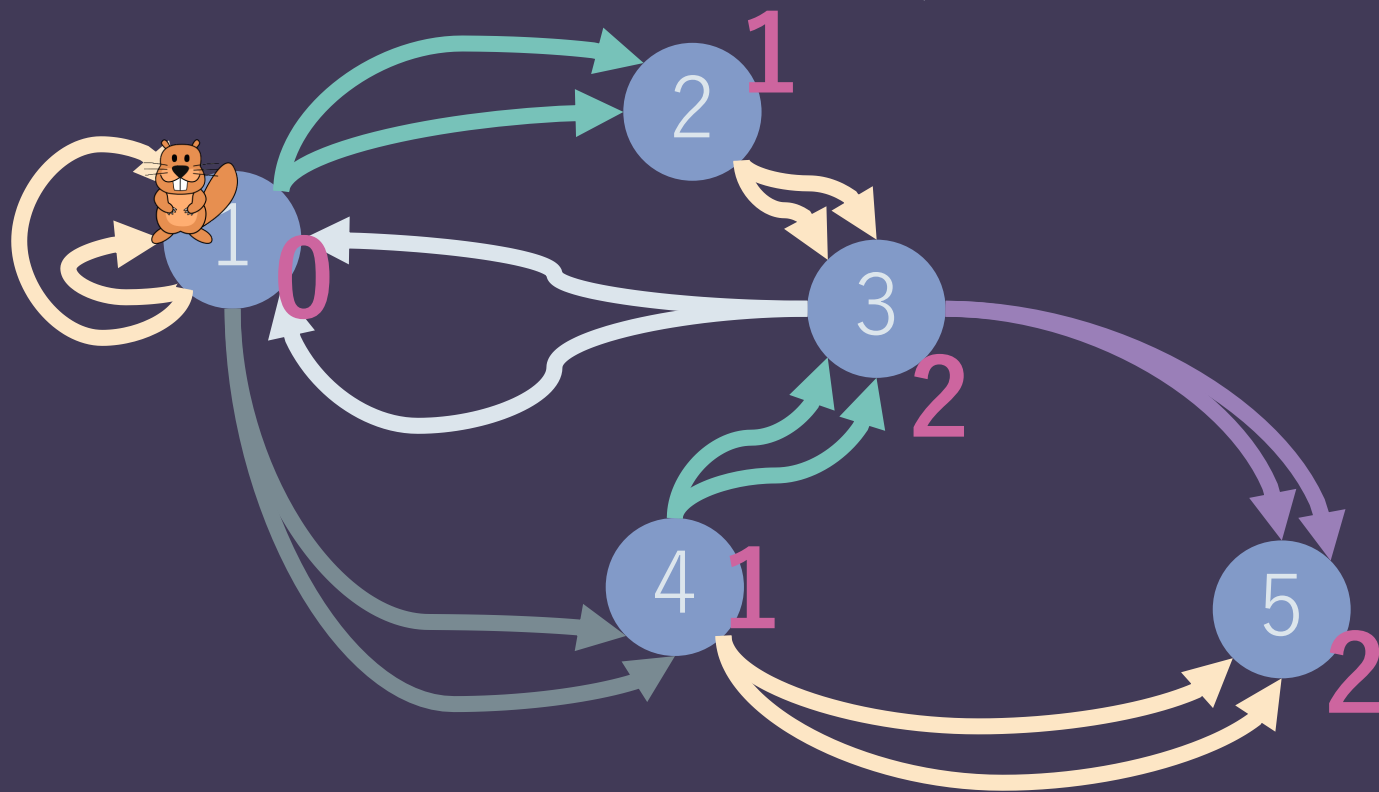
→ ビ太郎 の思うままに進める



小課題1 ($P_{i,j}=Q_{i,j}$)

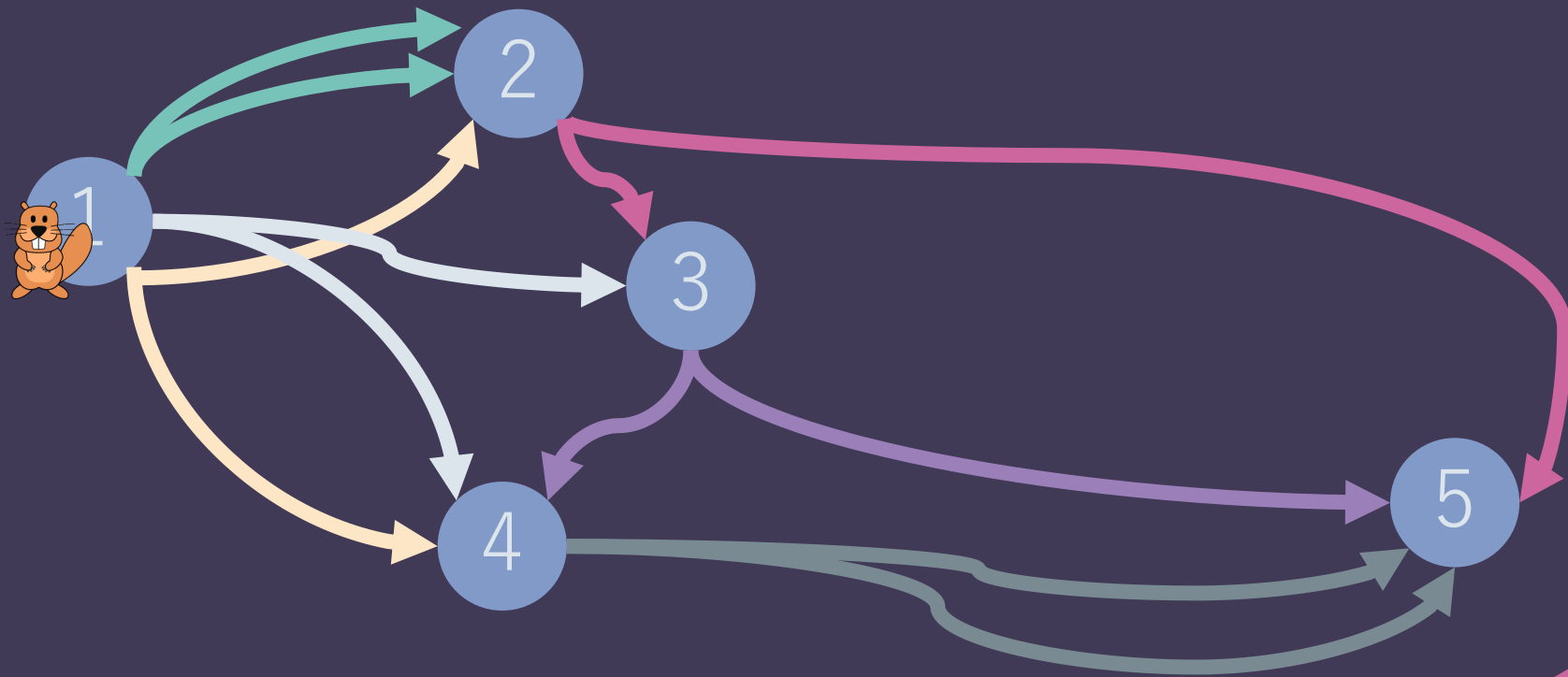
サンプル 3

最短経路問題を解く BFSをして, 計算量は $O(N + \sum A_i)$



小課題 2 ($P_{i,j} > i, Q_{i,j} > i$)

$$P_{i,j} > i, Q_{i,j} > i$$



小課題 2 ($P_{i,j} > i, Q_{i,j} > i$)

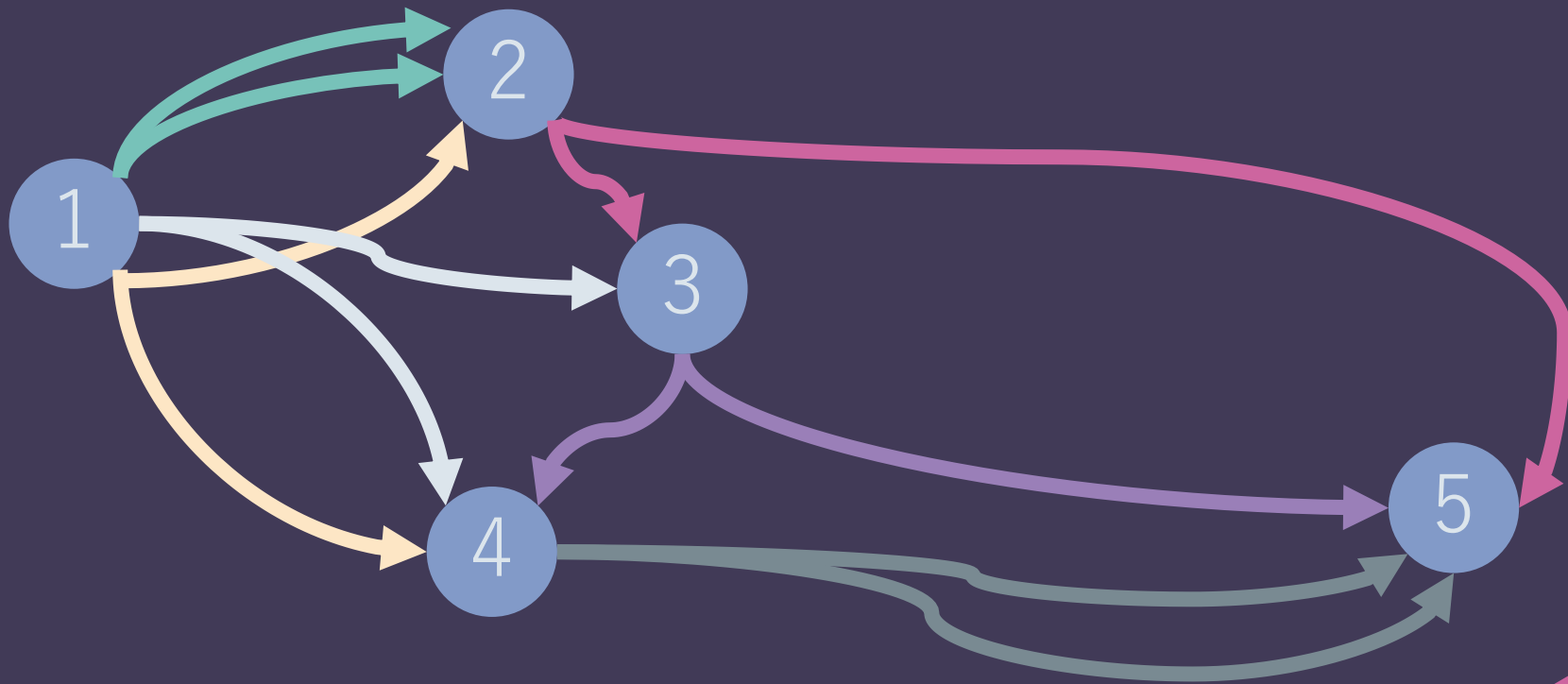
$P_{i,j} > i, Q_{i,j} > i$

→ DAG (有向非巡回グラフ) と呼ばれるもの

→ ? ? ?

小課題 2 ($P_{i,j} > i, Q_{i,j} > i$)

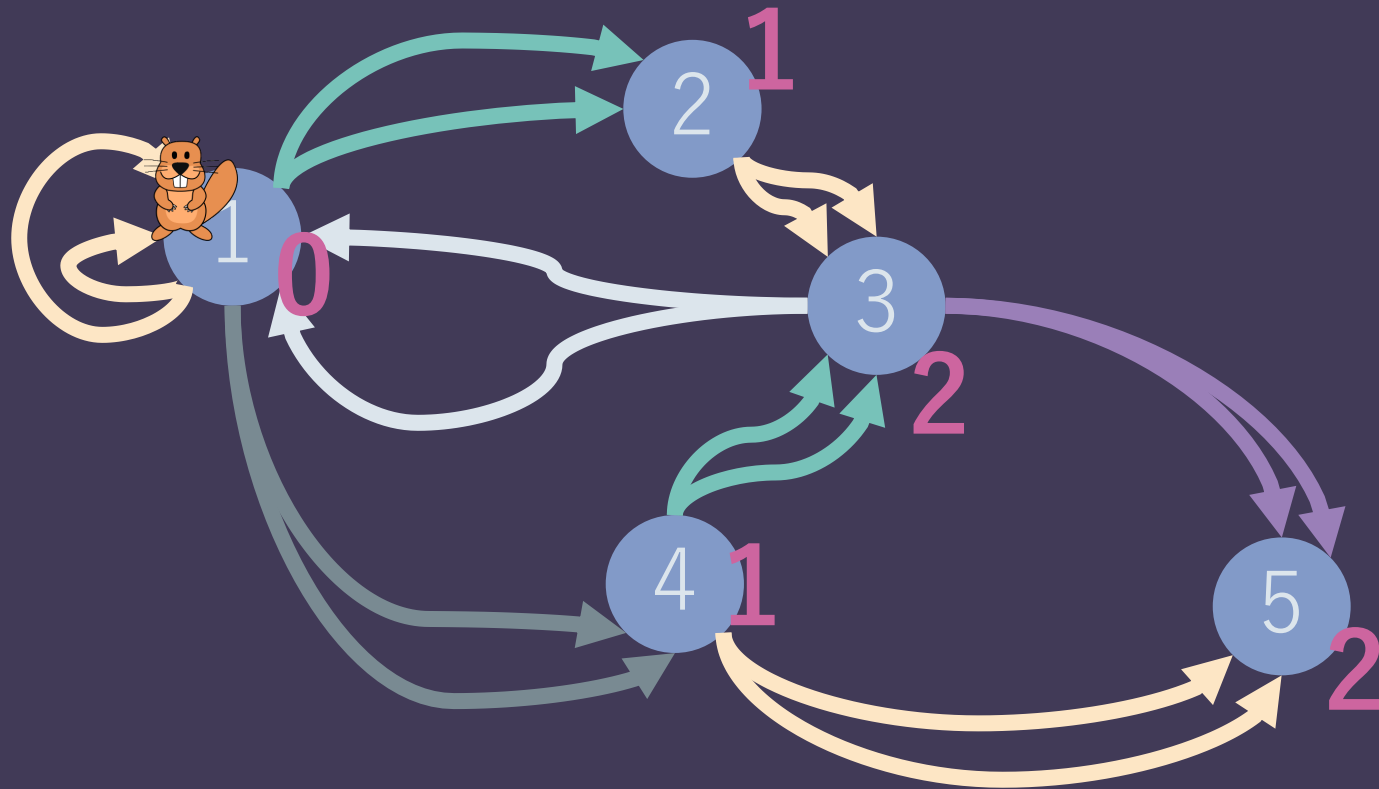
じっと眺める



小課題 2 ($P_{i,j} > i, Q_{i,j} > i$)

わからないときは前に戻って考える

小課題1では, BFSをして, 計算量は $O(N + \sum A_i)$

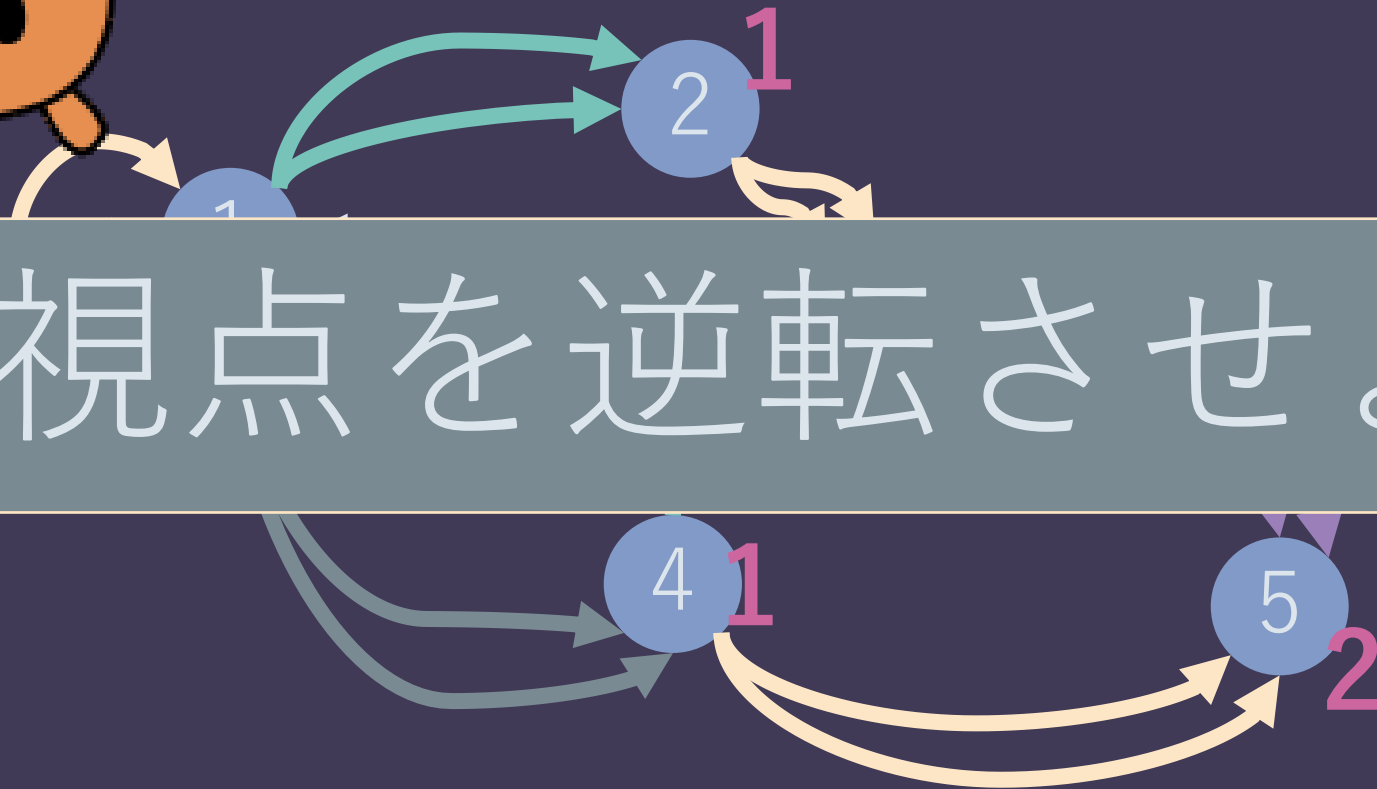


小課題 2 ($P_{i,j} > i, Q_{i,j} > i$)

いときは前に戻って考える

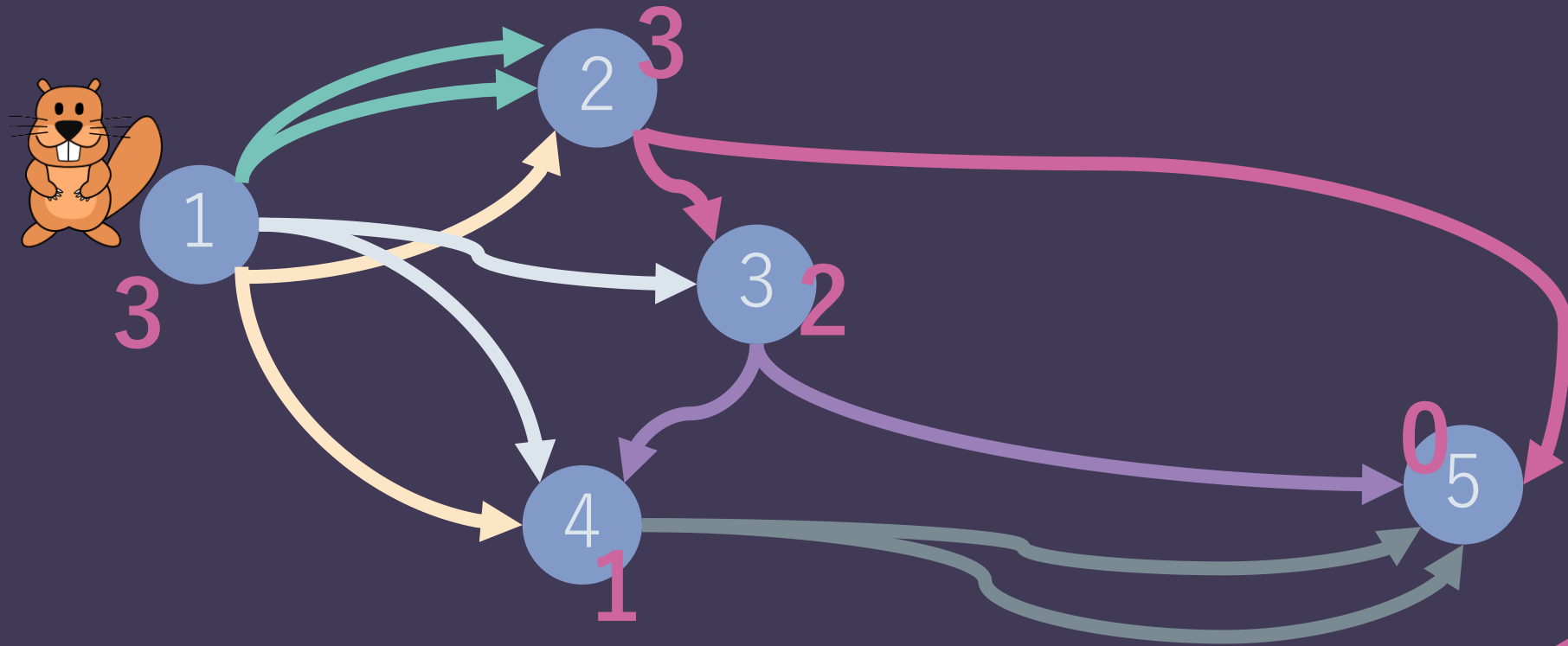
では, BFSをして, 計算量は $O(N + \sum A_i)$

視点を逆転させよう



小課題 2 ($P_{i,j} > i, Q_{i,j} > i$)

逆側からなら，頂点番号の大きいほうから順に距離を決められる



小課題 2 ($P_{i,j} > i, Q_{i,j} > i$)

逆側からなら，番号の大きい頂点から順に距離を決められる

具体的には，
以下のように頂点の番号 i が大きいほうから更新を行う

$$\text{距離}[i] \leftarrow \min(\text{距離}[i], \max(\text{距離}[P_{i,j}], \text{距離}[Q_{i,j}]) + 1)$$

計算量は $O(N + \sum A)$

小課題 3 ($\sum A_i \leq 2000$)

小課題2では「頂点番号の大きい方から」距離を確定させていた

この小課題ではNが小さいかつ $\sum A_i$ が小さいので、「距離が確定している頂点」を求めるのに毎回計算量 $O(N)$ かけても問題ない



小課題 3 ($\sum A_i \leq 2000$)



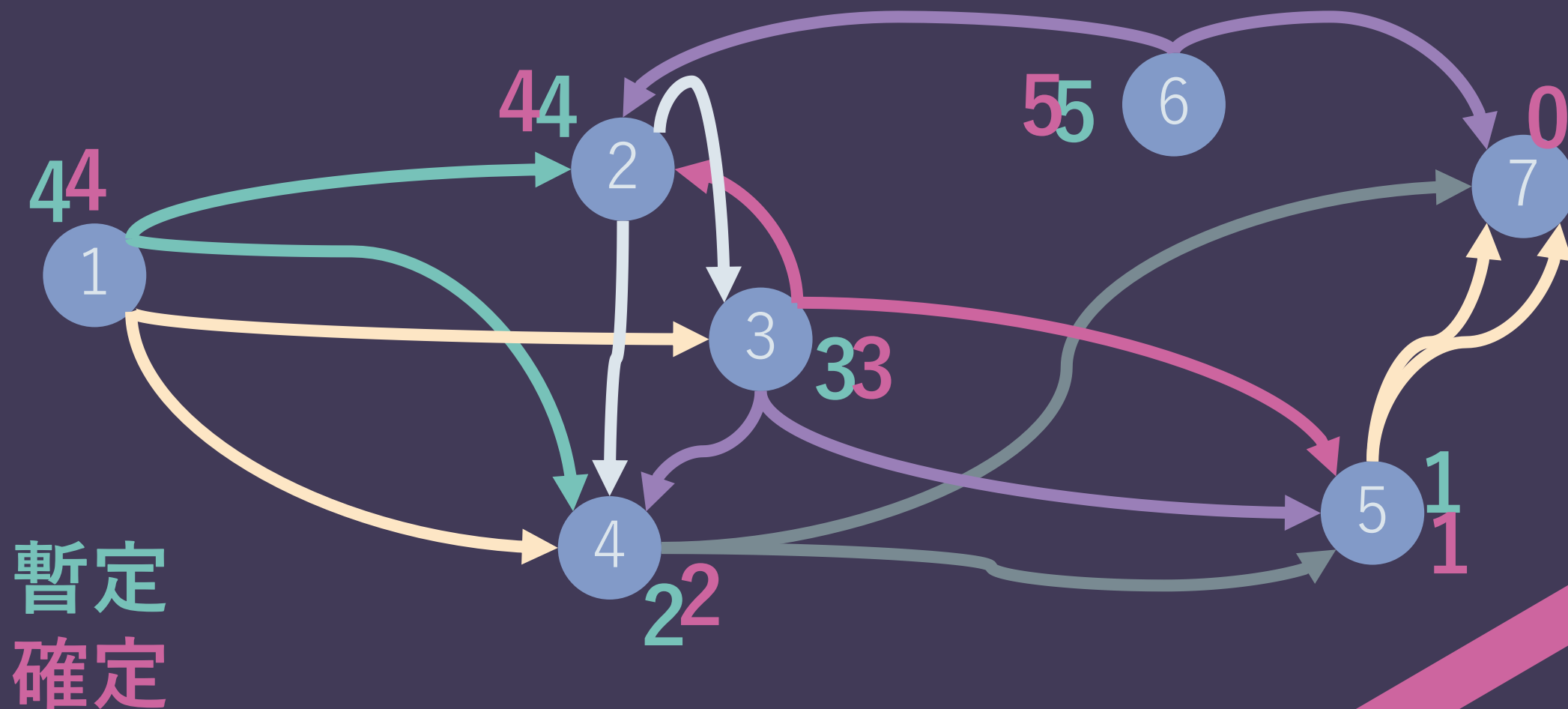
「距離が確定していない頂点」について「暫定の距離」を求める
最初 ∞ に初期化しておいて、以下のような更新を毎回全てのテレポーターについて行う

頂点 $P_{i,j}$, $Q_{i,j}$ のいずれも距離が確定している場合に限り,
暫定の距離 $[i] \leftarrow \min(\text{暫定の距離}[i], \max(\text{距離}[P_{i,j}], \text{距離}[Q_{i,j}]) + 1)$

距離が確定していない頂点のうち、最も暫定の距離が小さい頂点を
「距離が確定している頂点」とする

もし頂点 1 の距離が確定しない場合、答えは無限 (-1 と出力)
計算量は $O(N(N + \sum A_i))$

小課題 3 ($\sum A_i \leq 2000$)



小課題 4 ($A_i=1$)

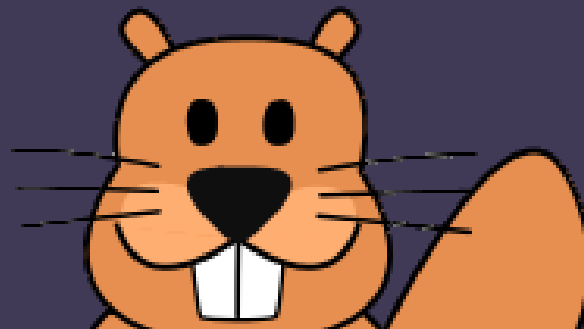
$A_i=1$

- ビ太郎の戦略は、存在するただ一つのテレポーターを選ぶだけ
- ビバ子が全てを左右する



小課題 4 ($A_i=1$)

小課題2や3で紹介した方法と似たように、後ろから解く方法もありますが、ここでは（満点には結びつかないけれど）前から解く方法を紹介します



小課題 4 ($A_i=1$)

ビバ子の思うままにビ太郎を動かせる

→ループに突入させたい, 突入させられるループがなければなるべく長いルートを通りたい



小課題 4 ($A_i=1$)

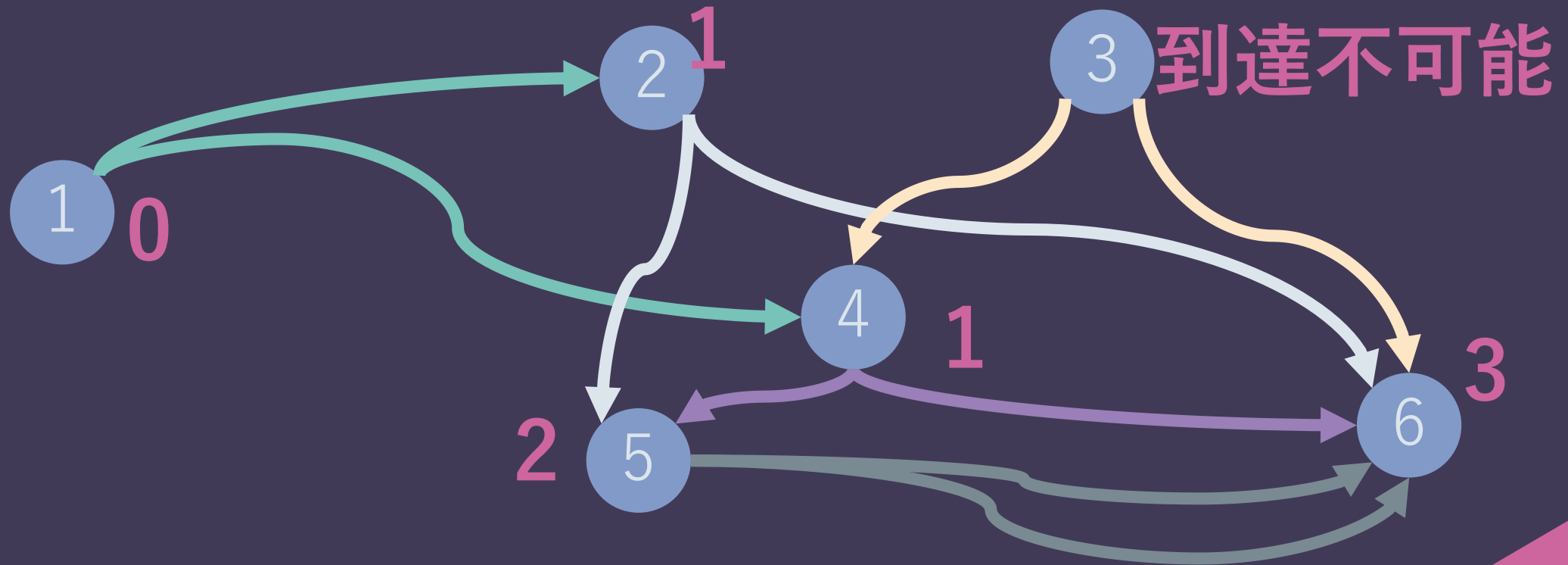
ループの検出方法

「強連結成分分解」というものがあります。
詳しくは調べてください。

ループがなかった場合、あっても頂点 1 からたどり着けない場合は DAG（有向非巡回グラフ）として扱うことができます。
頂点番号を振りなおして小課題 2 と同じようにも解けますし、もう少し簡単な方法もあります。

小課題 4 ($A_i=1$)

前側から求めることができます。
(頂点番号は振りなおしたものとってください)



小課題 5 (満点)

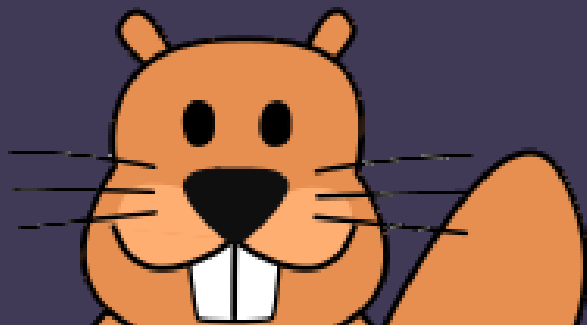
小課題 3 を思い出そう

この小課題では N が小さいかつ $\sum A_i$ が小さいので、「距離が確定している頂点」を求めるのに毎回計算量 $O(N)$ かけても問題ない

→毎回計算量 $O(N)$ かける必要はあるか？

以下のような更新を毎回全てのテレポーターについて行う

→全てのテレポーターについて行う必要はあるか？



小課題 5 (満点)

小課題 3 を思い出そう

この小課題では N が小さいかつ $\sum A_i$ が小さいので、「距離が確定している頂点」を求めるのに毎回計算量 $O(N)$ かけても問題ない

→毎回計算量 $O(N)$ かける必要はあるか？

以下のような更新を毎回全てのテレポーターについて行う

→全てのテレポーターについて行う必要はあるか？

→更新した時だけ見ればよい



小課題 5 (満点)

更新した時だけ見ればよい

「頂点の距離」は小さい順に確定していく

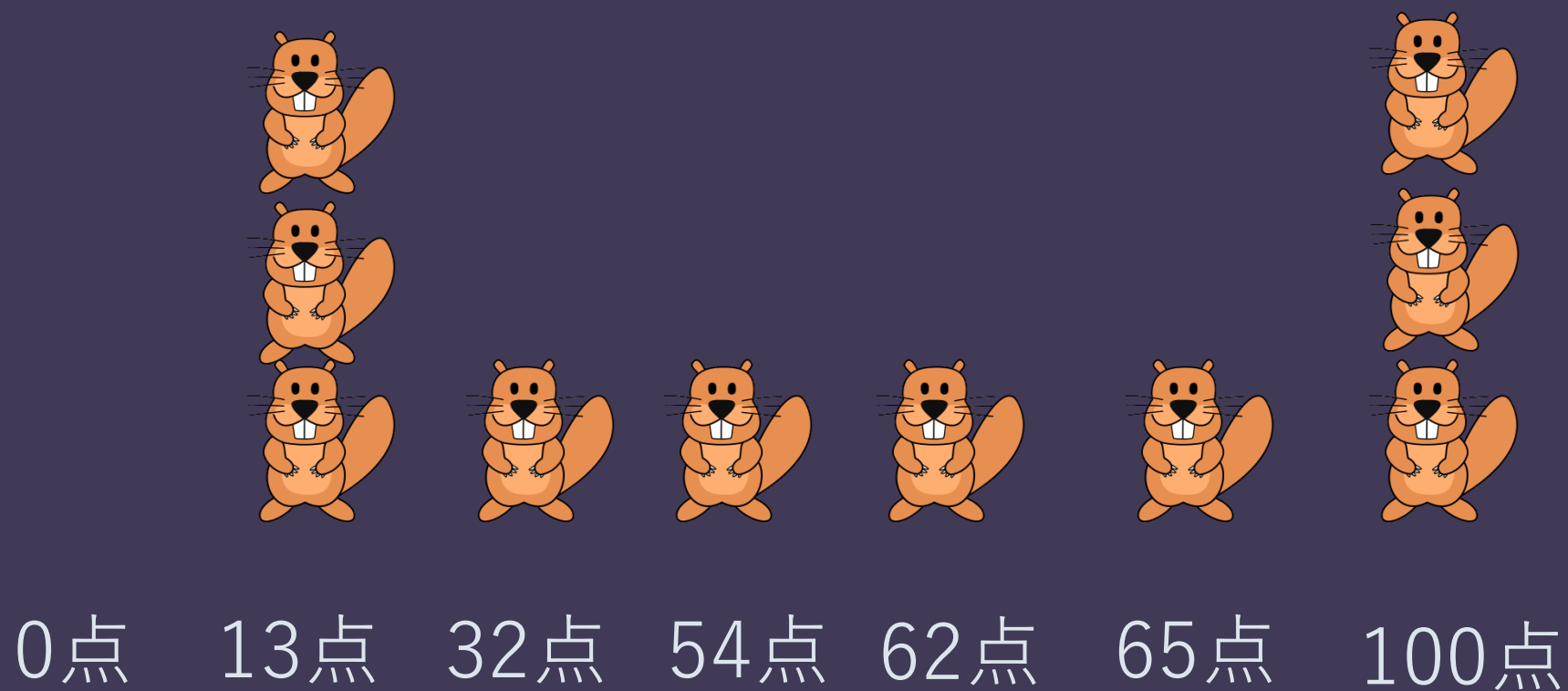
頂点 $P_{i,j}$, $Q_{i,j}$ のいずれも距離が確定している場合に限り,
暫定の距離 $[i] \leftarrow \min(\text{暫定の距離}[i], \max(\text{距離}[P_{i,j}], \text{距離}[Q_{i,j}]) + 1)$

→頂点 $P_{i,j}$, $Q_{i,j}$ の両方の距離が確定する瞬間のみに更新しても問題ない

→テレポーターごとに「頂点 $P_{i,j}$ を見たか」「頂点 $Q_{i,j}$ を見たか」を記憶しておき、どちらのフラグもTrueになった瞬間に更新するのが良い

→BFS (幅優先探索) で計算量 $O(N + \sum A_i)$ で解けました

得点分布



JOI 2022/2023 春季トレーニング Day 2

議会 (Council) 解説

解説担当 : tatyam

問題概要

- N 人の人がいて, M 個の案に投票を行う.
- $A_{i,j} = 1$ のとき, 人 i は案 j に賛成票を投じる.
- $i = 1, 2, \dots, N$ について, 以下の問題を解け.
 - $j (\neq i)$ を選び, 人 i と人 j を投票から除外する.
 - 過半数 ($\lfloor N/2 \rfloor$ 票以上) の賛成票を獲得する案の個数の最大値は?

$$N \leq 3 \times 10^5, \text{ **M} \leq \textbf{20}**$$

小課題 (JOI)

番号	配点	制約	計算量の例
1.	8 点	$N \leq 300$	$O(N^3 M)$
2.	8 点	$N \leq 3000$	$O(N^2 M)$
3.	6 点	$M \leq 2$	
4.	19 点	$M \leq 10$	$O(N 2^M)$
5.	15 点	$M \leq 14$	$O(NM + 4^M)$
6.	22 点	$M \leq 17$	$O(NM + 3^M M)$
7.	22 点	$M \leq 20$	$O(NM + 2^M M^2)$

小課題 (JOIG)

番号	配点	制約	計算量の例
1.	13 点	$N \leq 300$	$O(N^3 M)$
2.	17 点	$N \leq 3000$	$O(N^2 M)$
3.	10 点	$M \leq 2$	
4.	18 点	$M \leq 10$	$O(N 2^M)$
5.	12 点	$M \leq 14$	$O(NM + 4^M)$
6.	22 点	$M \leq 17$	$O(NM + 3^M M)$
7.	8 点	$M \leq 20$	$O(NM + 2^M M^2)$

小課題 1 ($N \leq 300$)

- $O(N^3 M)$ くらいの計算量で解けば良い.

小課題 1 ($N \leq 300$)

- $O(N^3 M)$ くらいの計算量で解けば良い.
- 議長 i を全探索: $O(N)$
- 副議長 j を全探索: $O(N)$
- それぞれの案に何票集まったかを集計: $O(NM)$

$\implies$ 全部合わせて $O(N^3 M)$

小課題 2 ($N \leq 3000$)

- 議長 i を全探索 : $O(N)$
- 副議長 j を全探索 : $O(N)$
- **それぞれの案に何票集まったかを集計 : $O(NM)$**

議長と副議長の 2 人しか変わらないのに、毎回集計し直すのは無駄！

小課題 2 ($N \leq 3000$)

- それぞれの案に何票集まっているかを前計算 : $O(NM)$

その後,

- 議長 i を全探索 : $O(N)$
- 副議長 j を全探索 : $O(N)$
- 議長と副議長の分を取り除く : $O(M)$

$\implies$ 全部合わせて $O(N^2M)$

小課題 2 ($N \leq 3000$)

- それぞれの案に何票集まっているかを前計算 : $O(NM)$

その後,

- 議長 i を全探索 : $O(N)$
- 副議長 j を全探索 : $O(N)$
- 議長と副議長の分を取り除く : $O(M)$

$\implies$ 全部合わせて $O(N^2M)$

実装テクニック：整数をビット列として扱おう

- ビット演算 `|` `&` `^` `<<` `>>` を使って、整数をビット列として扱うことができる.
- 議長と副議長を決めたときに、成立する案の個数を $O(M)$ で求めていたが、これを $O(1)$ にできる.

実装テクニック：整数をビット列として扱おう

A をビット列で入力する

```
vector<int> A(N);  
  
for(int i = 0; i < N; i++) for(int j = 0; j < M; j++) {  
    int a;  
    cin >> a;  
    A[i] |= a << j;  
}
```

考察

各案は

- 議長・副議長に関わらず可決
- 議長・副議長のどちらかが反対なら可決
- 議長・副議長の両方が反対なら可決
- 議長・副議長に関わらず否決

の 4 通りに分けられる. そのうち中央の 2 つのみが重要である.

実装テクニック：整数をビット列として扱おう

- 可決票数 + 2 票以上の賛成票を得ている案の集合
- 可決票数 + 1 票の賛成票を得ている案の集合
- 可決票数 + 0 票の賛成票を得ている案の集合

をそれぞれ整数で持つ

```
for(int j = 0; j < M; j++) {  
    if(vote[j] >= N / 2 + 2) two |= 1 << j;  
    if(vote[j] == N / 2 + 1) one |= 1 << j;  
    if(vote[j] == N / 2) zero |= 1 << j;  
}
```

実装テクニック：整数をビット列として扱おう

- 議長 i と副議長 j を決めたら，可決される案の集合が $O(1)$ で求められる

```
int accept = two | one & ~(A[i] & A[j]) | zero & ~(A[i] | A[j]);
```

あとは，これの立っているビットの個数を数えれば良い.

popcount

- C++20

どうして...

function template

std::popcount ⚡ C++20

```
namespace std {  
    template <class T>  
    constexpr int popcount(T x) noexcept;  
}
```

概要

立っているビットを数える。popcountは「population count」の略。 15

popcount

- `std::bitset::count` を使う

```
int popcnt(int x) { return bitset<32>(x).count(); }
```

- `__builtin_popcount` を使う

```
int popcnt(int x) { return __builtin_popcount(x); }
```

これらは通常賢いビット演算をしている `__popcountdi2` にコンパイルされるが、今回はコンパイルオプションに `-march=native` が指定されていたので、`POPCNT` 命令にコンパイルされる 🙌🙌

popcount

- 自分で実装する

```
int popcnt(int x) {  
    int ans = 0;  
    while(x) {  
        ans += x & 1;  
        x >>= 1;  
    }  
    return ans;  
}
```

計算量は $O(M)$

popcount

- 全部前計算しておく

```
int popcnt(int x) {  
    static array<int, 1 << 20> table = []{  
        // 前計算する  
    }();  
    return table[x];  
}
```

計算量は $O(1)$

クイズ

どっちが速い？

[A]

```
int popcnt(int x) {  
    int ans = 0;  
    while(x) {  
        ans += x & 1;  
        x >>= 1;  
    }  
    return ans;  
}
```

計算量は $O(M)$

[B]

```
int popcnt(int x) {  
    static array<int, 1 << 20> table = []{  
        // 前計算する  
    }();  
    return table[x];  
}
```

計算量は $O(1)$

答え

[A]

```
int popcnt(int x) {  
    int ans = 0;  
    while(x) {  
        ans += x & 1;  
        x >>= 1;  
    }  
    return ans;  
}
```

計算量は $O(M)$

- キャッシュの効かないメモリアクセスは、ビット演算や足し算に比べて非常に遅い！！

popcnt

- 賢いビット演算

```
int popcnt(int x) {  
    x = (x & 0x55555555) + (x >> 1 & 0x55555555);  
    x = (x & 0x33333333) + (x >> 2 & 0x33333333);  
    x = (x & 0x0f0f0f0f) + (x >> 4 & 0x0f0f0f0f);  
    x = (x & 0x00ff00ff) + (x >> 8 & 0x00ff00ff);  
    return (x & 0x0000ffff) + (x >> 16);  
}
```

計算量は $O(\log M)$

小課題 3 ($M \leq 2$)

- なんと $M \leq 2$
- $M = 2$ のとき, $(0, 0), (0, 1), (1, 0), (1, 1)$ の 4 種類のタイプの人間がいる.

こんな例を考える.

	案 1	案 2
人 1	0	1
人 2	0	1
人 3	1	0
人 4	1	0
人 5	1	1
合計	3	3

$$N = 5, M = 2$$

可決票数: 2 票

人 1 が議長の場合と人 2 が議長の場合で、状況が全く同じ.

	案 1	案 2
人 2	0	1
人 3	1	0
人 4	1	0
人 5	1	1
合計	3	2

	案 1	案 2
人 1	0	1
人 3	1	0
人 4	1	0
人 5	1	1
合計	3	2

つまり、答えも同じ.

結局, 「議長と副議長が 4 種類のタイプのうちどのタイプの人間か？」が分かれば可決される案も分かる.

副議長が:	(0, 0)	(0, 1)	(1, 0)	(1, 1)
議長が: (0, 0)	2	2	2	2
(0, 1)	2	1	2	1
(1, 0)	2	2	1	1
(1, 1)	2	1	1	0

この $4 \times 4 = 16$ 種類を調べれば良い.

小課題 4, 5 ($M \leq 10, M \leq 14$)

- M が小さめ
- $2^{14} \approx 1.6 \times 10^4 \ll 3 \times 10^5$ なので, 同じタイプの人間を圧縮することで N を小さくできる.
- 小課題 3 で $4 \times 4 = 16$ 通りを調べたのと同様に, $2^M \times 2^M = 4^M$ 通りを全部調べることができる.

計算量は $O(4^M)$

小課題 6 ($M \leq 17$)

- どうにかして計算量を $O(3^M \text{poly}(M))$ にしたい.

小課題 6 ($M \leq 17$)

i, j を決めたとき, 可決される案の集合は以下のように計算された.

```
accept := two | one & ~(A[i] & A[j]) | zero & ~(A[i] | A[j])
```

ここで i を固定すると,

- j によって可決されるかわ変わる案 (可決票数 + 0 票)
- j に関わらず可決される案 (可決票数 + 1 票以上)

の 2 つを持てば良い.

小課題 6 ($M \leq 17$)

i を固定すると,

- j によって可決されるか変わる案 (可決票数 + 0 票)
- j に関わらず可決される案 (可決票数 + 1 票以上)

の 2 つを持てば良い.

```
ONE := two | (one & ~A[i])  
ZERO := (one & A[i]) | (zero & ~A[i])  
accept := ONE | (ZERO & ~A[j])
```

小課題 6 ($M \leq 17$)

```
accept := ONE | (ZERO & ~A[j])
```

- `popcnt(accept)` を最大化したい.
- つまり, `popcnt(ZERO & ~A[j])` を最大化したい.
- 案の全体集合を `ZERO` に絞ったときの, 0 の個数が (人 i を除いて) 最も多い人を探したい.

小課題 6 ($M \leq 17$)

すべての $x \subset \text{ZERO}$ であるような (x, ZERO) の組 (3^M 個) について, 以下が求めれば良い.

- x に含まれる案に反対で, $\text{ZERO} \setminus x$ に含まれる案に賛成であるような人の数 (ZERO に含まれない案については 賛成 / 反対 どちらでもよい)

これは, 高速ゼータ変換 (M 次元累積和) をして, ZERO の部分だけ戻すことを全ての ZERO について行えば, $O(3^M M)$ 時間になる.

小課題 7 ($M \leq 20$)

- 高速ゼータ変換をどうにかして計算量を $O(2^M \text{poly}(M))$ にしたい.

小課題 7 ($M \leq 20$)

```
accept := ONE | (ZERO & ~A[j])
```

→ わかりやすさのため $A[j]$ を反転させておく.

各 ZERO に対し,

$$\max_{j \neq i} |\text{ZERO} \cap A_j|$$

を求める問題である.

小課題 7 ($M \leq 20$)

$$\max_{j \neq i} |\text{ZERO} \cap A_j|$$

$j \neq i$ の制約が面倒なので、一旦忘れると

$$\max_j |\text{ZERO} \cap A_j|$$

となる.

$$\max_j |\text{ZERO} \cap A_j|$$

A_j を ZERO まで減らしたときの集合の大きさ (popcount) の最大値であるので、各 A_j から要素を減らしていったときに現れる集合を前計算しておく。

$$B = \{b \mid \exists j : b \subset A_j\}$$

これは下位集合への累積 OR で $O(2^M M)$ でできる。問題は

$$\max\{|b| \mid b \in B, b \subset \text{ZERO}\}$$

となった。

$$\max\{|b| \mid b \in B, b \subset \text{ZERO}\}$$

今度は、各 b から要素を増やしていった、ZERO まで増やしたときの元々の集合の大きさ (popcount) の最大値という問題である。

したがって、上位集合への chmax をすることで、各 ZERO に対する答えが $O(2^M M)$ で求められる。

まとめ

$j \neq i$ の制約を忘れると,

1. bool 配列 `dp1[a]` を作る
2. 各 A_j に対し, `dp1[A[j]]` に 1 を代入
3. 下位集合に累積 OR
4. int 配列 `dp2[b]` を作る
5. 各 b に対し, `dp1[b] == 1` であれば, `dp2[b]` に `popcnt(b)` を代入
6. 上位集合に累積 chmax

自分自身の分を取り除くには？

自分自身の分を取り除くには？

→ 個数を数えれば良い！

1. **int** 配列 `dp1[a]` を作る
2. 各 A_j に対し, `dp1[A[j]]` に 1 を加算
3. 下位集合に累積和
4. **int** 配列 `dp2[k][b]` を作る (**popcount** の値ごとに個数を数える)
5. 各 b に対し, `dp2[popcnt(b)][b]` に `dp1[b]` を加算
6. **popcount** の値ごとに, 上位集合に累積和

自分自身の分を取り除くには？

1. int 配列 `dp1[a]` を作る
2. 各 A_j に対し, `dp1[A[j]]` に 1 を加算
3. 下位集合に累積和
4. int 配列 `dp2[k][b]` を作る (**popcount** の値ごとに個数を数える)
5. 各 b に対し, `dp2[popcnt(b)][b]` に `dp1[b]` を加算
6. **popcount** の値ごとに, 上位集合に累積和

自分自身の影響は簡単に数えられるので, その分を引けば良い.

計算量は $O(2^M M^2)$

$O(2^M M)$ にもなりますが, 省略します.

統計情報
(JOI)

点数	1	2	3	4	5	6	7	人数	累積人数
100	○	○	○	○	○	○	○	10	10
56	○	○	○	○	○	-	-	5	15
41	○	○	○	○	-	-	-	6	21
33	○	-	○	○	-	-	-	1	22
22	○	○	○	-	-	-	-	1	23
16	○	○	-	-	-	-	-	3	26
0	-	-	-	-	-	-	-	1	27

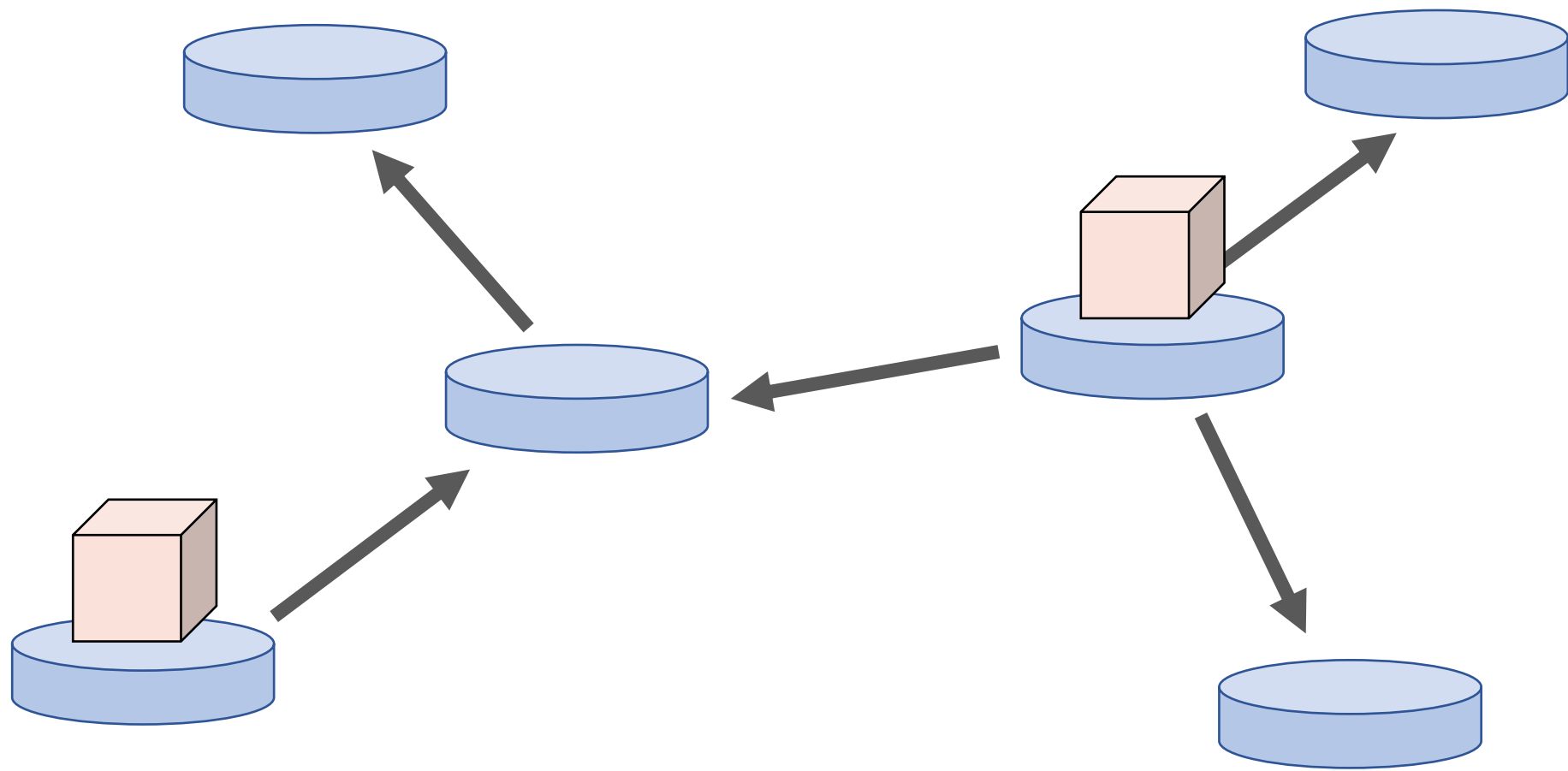
統計情報
(JOIG)

点数	1	2	3	4	5	6	7	人数	累積人数
58	○	○	○	○	-	-	-	3	3
40	○	○	○	-	-	-	-	2	5
30	○	○	-	-	-	-	-	5	10

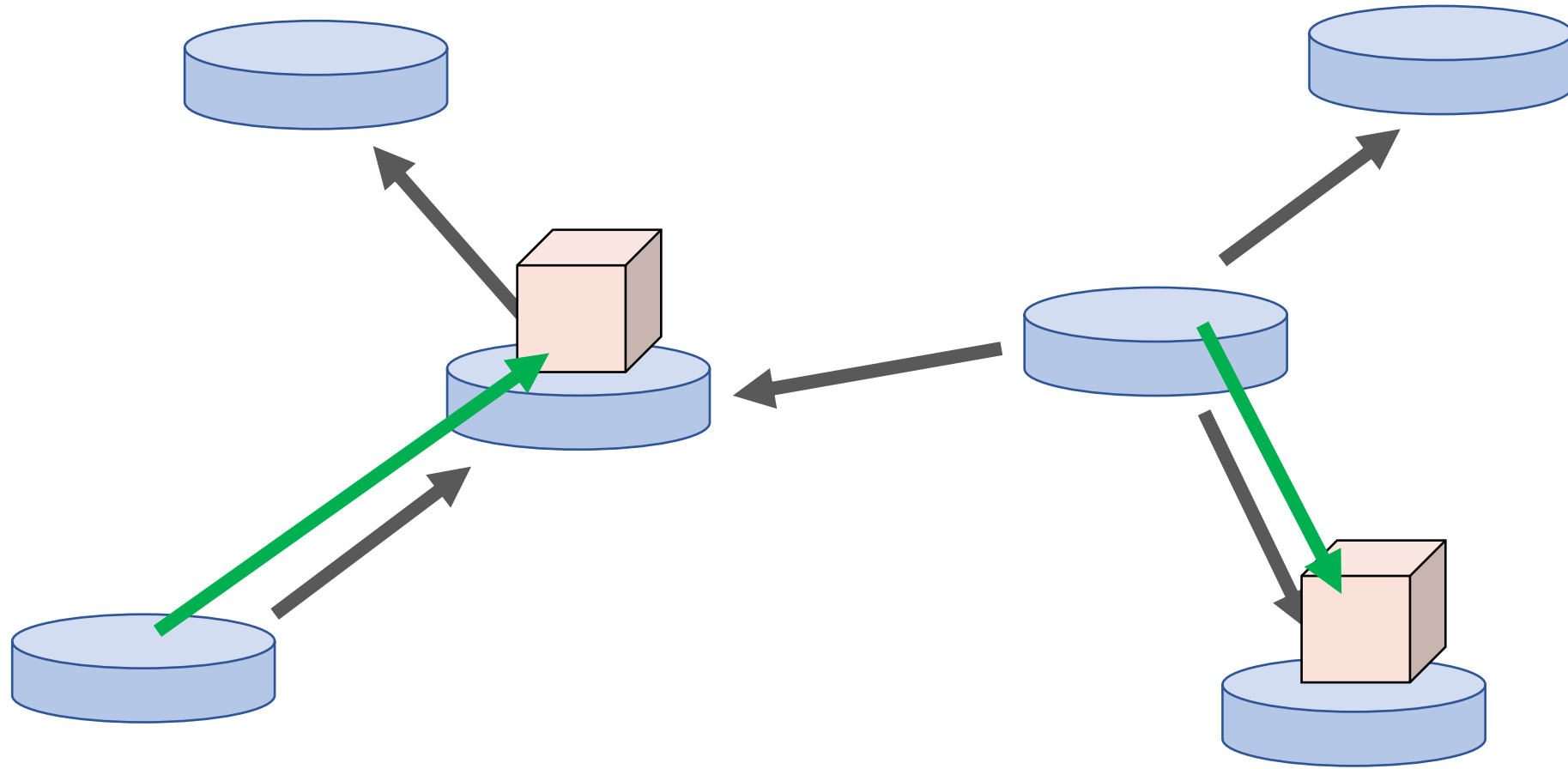
JOI 2022/2023 春 ベルトコンベア

解説: 木ノ下恭範 (noshi91)

問題概要



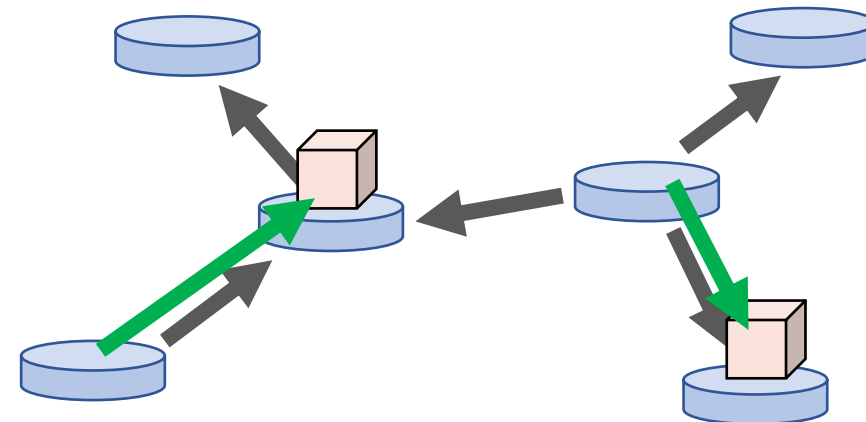
問題概要



問題概要

N 頂点の木がある

各辺は製品を一方向に運ぶが、方向が分からない

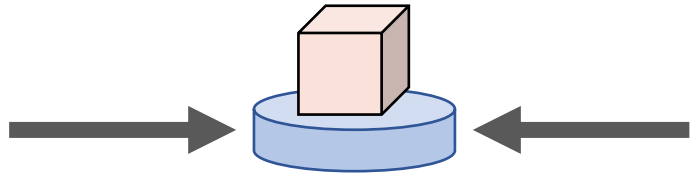


1. いくつかの辺を選んで反転する
2. いくつかの頂点に製品を置き、製品が移動する
3. 各頂点に製品が **1** 個以上あるか確認する
4. 反転した辺を戻す

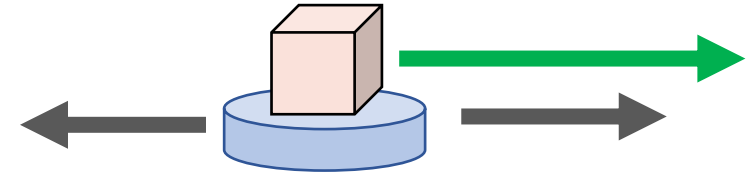
30 クエリ以内に辺の向きを判定せよ

問題概要

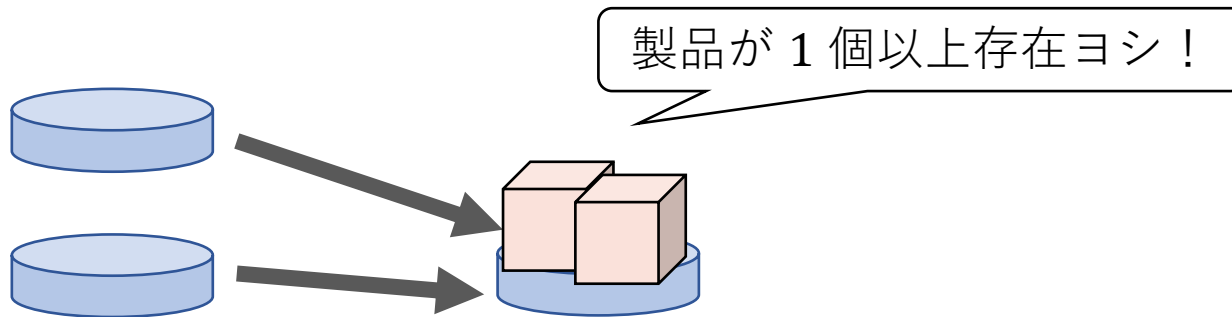
出ていく辺が無ければ動かない



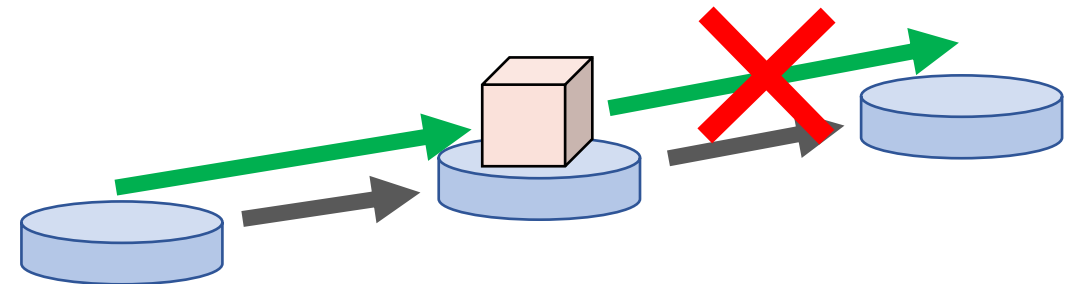
どの辺で移動するかは分からない



製品が合流しても分からない



移動は連鎖せず、1 回だけ

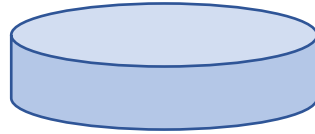
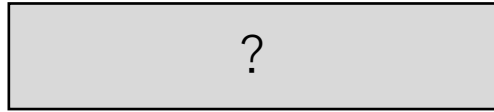
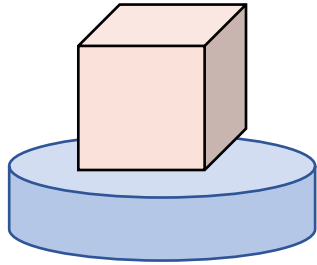


小課題

- ・ (1 点) $N = 2$
- ・ (14 点) $N = 30$
- ・ (10 点) $N = 100\,000$, 入力はパスグラフ
- ・ (75 点) $N = 100\,000$

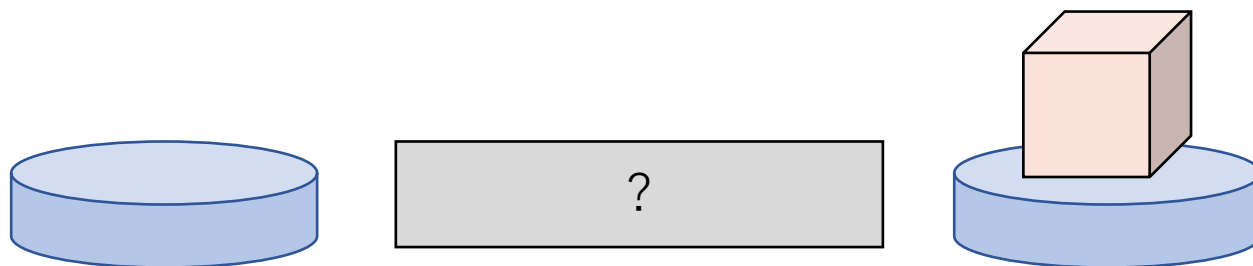
$$N = 2$$

とりあえず置いてみる



$$N = 2$$

とりあえず置いてみる



$$N = 2$$

とりあえず置いてみる



動いたら、その向きに辺が向いていることが確定

$$N = 2$$

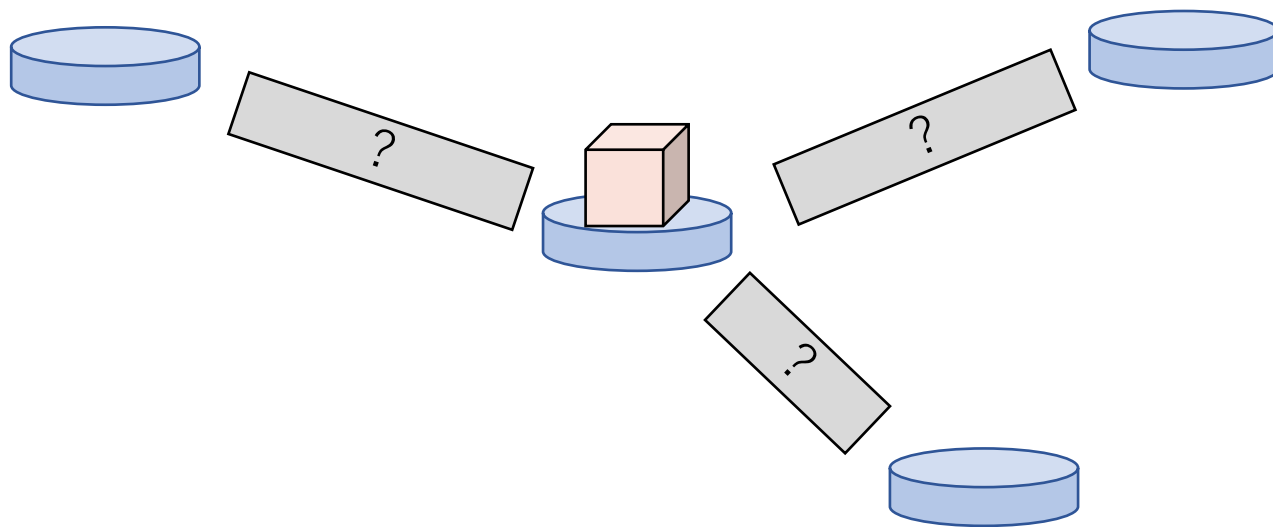
とりあえず置いてみる



留まったら、辺が製品側を向いていることが確定

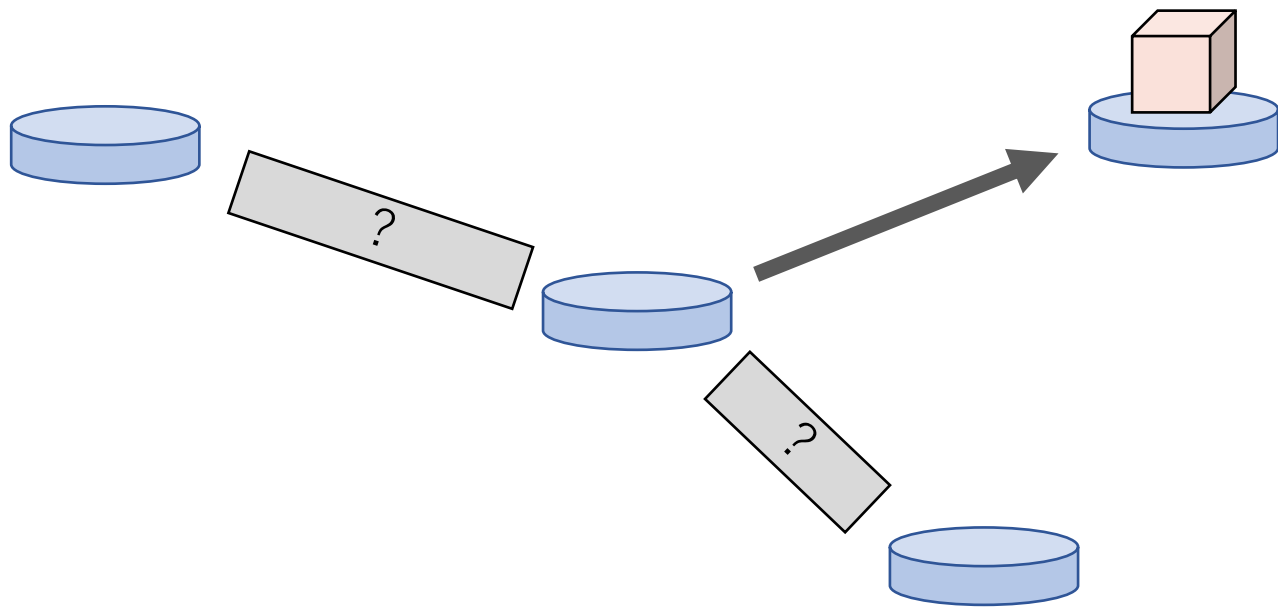
$$N = 30$$

製品を何個も置くと動きがよく分からないので、**1** 個だけ置いてみる



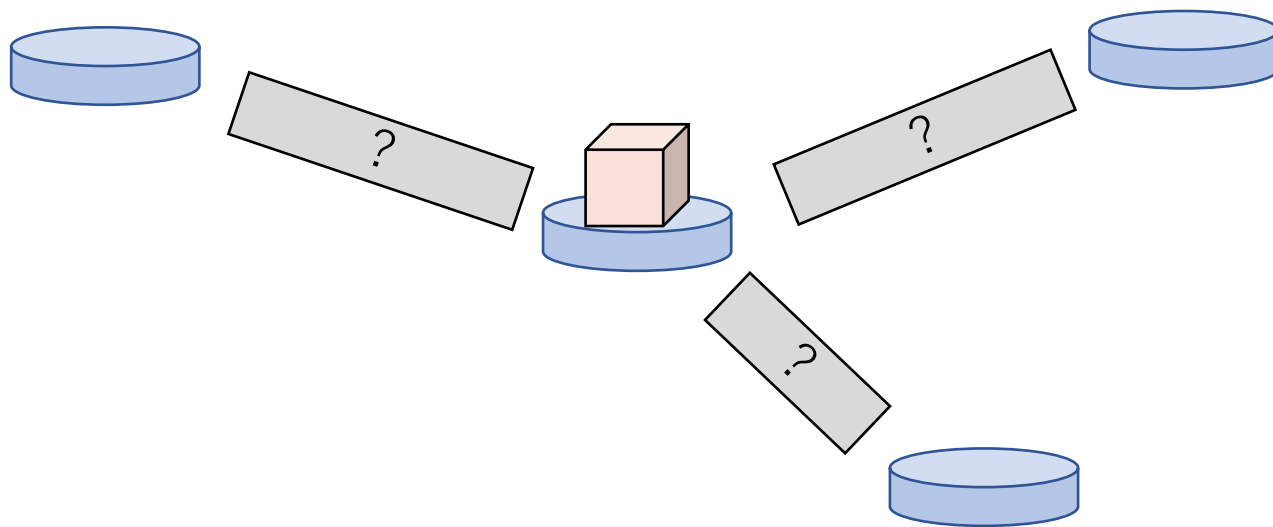
$$N = 30$$

製品を何個も置くと動きがよく分らないので、**1** 個だけ置いてみる



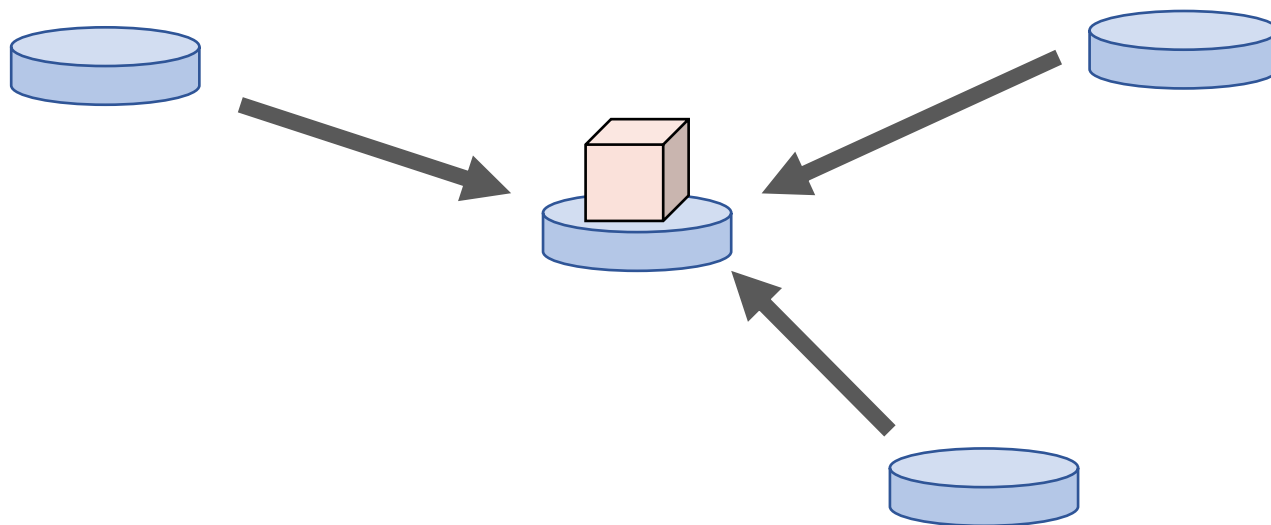
$$N = 30$$

製品を何個も置くと動きがよく分からないので、**1** 個だけ置いてみる



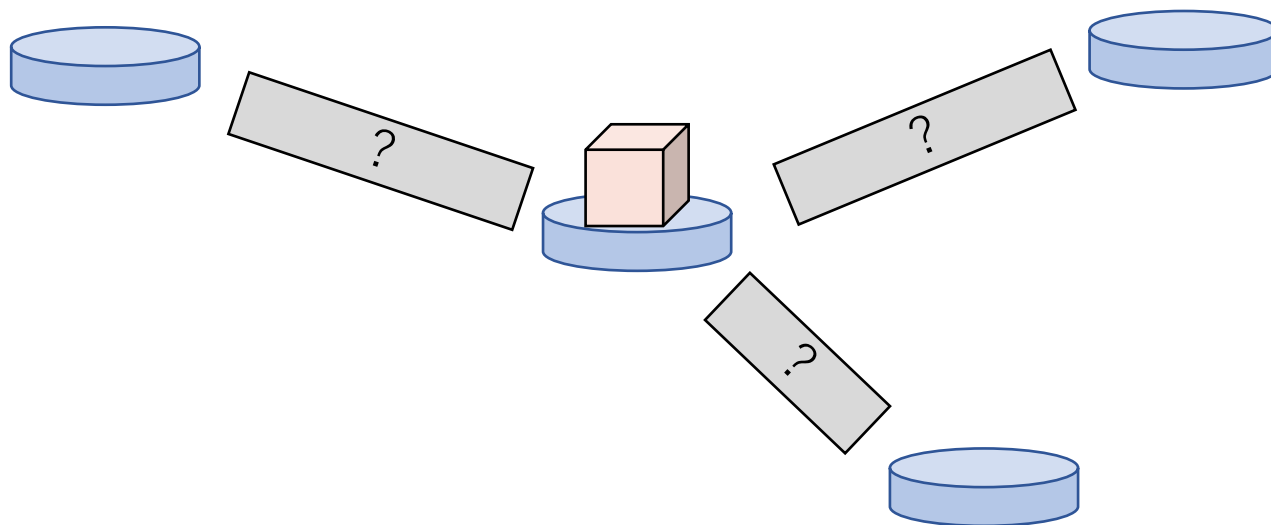
$$N = 30$$

製品を何個も置くと動きがよく分からないので、**1** 個だけ置いてみる



$$N = 30$$

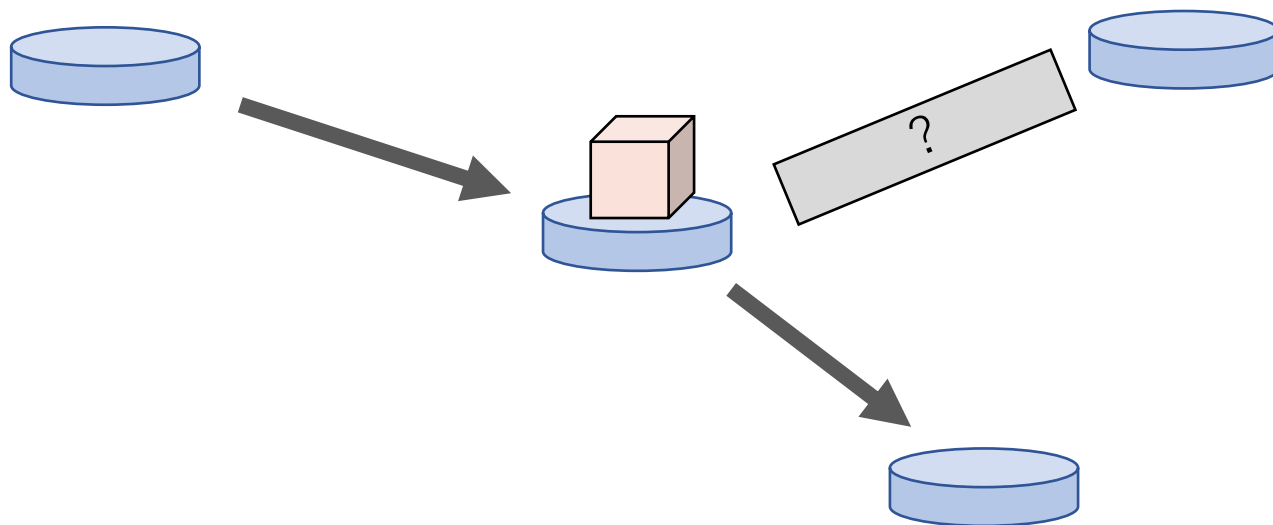
製品が動いても動かなくても、どこかの辺の向きは確定できた。
これを繰り返せば OK → 本当に？



$$N = 30$$

辺の向きが確定しないケース 1

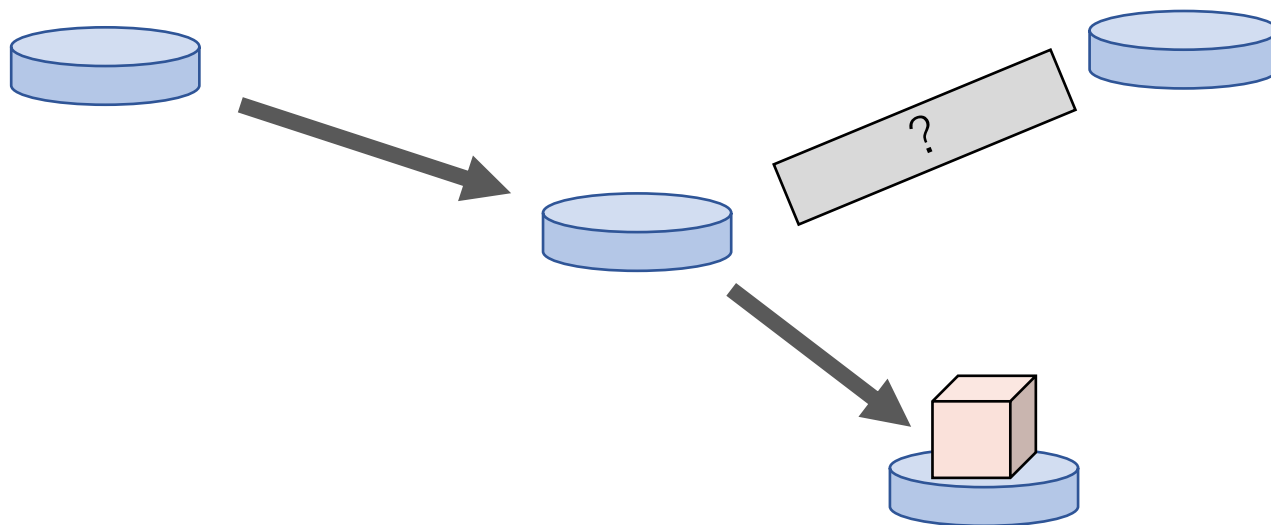
既に確定した辺に沿って流れてしまうと困る



$$N = 30$$

辺の向きが確定しないケース 1

既に確定した辺に沿って流れてしまうと困る

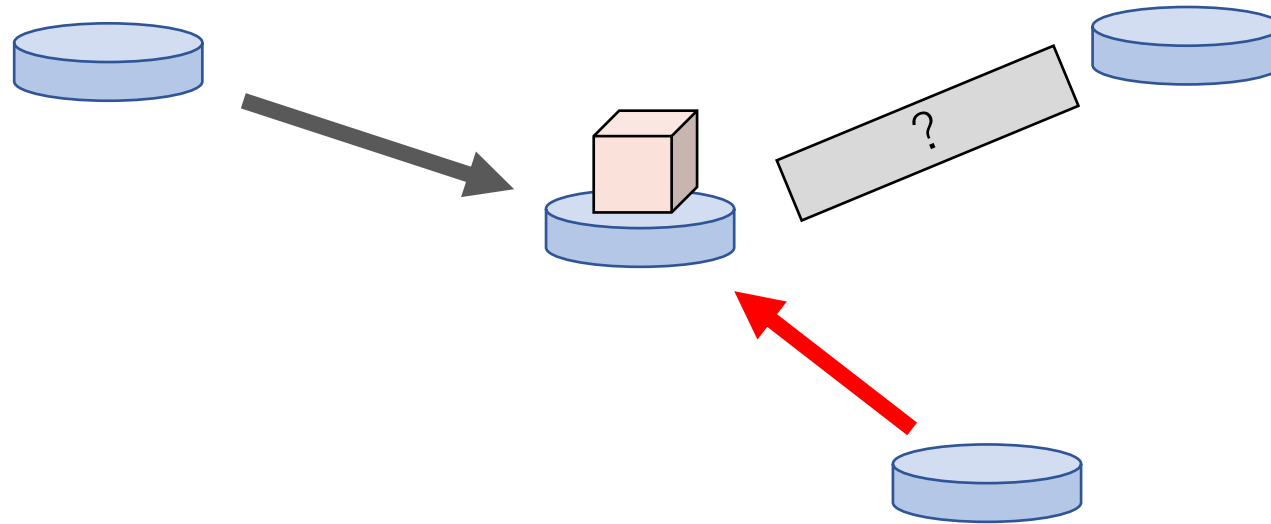


$$N = 30$$

辺の向きが確定しないケース 1

既に確定した辺に沿って流れてしまうと困る

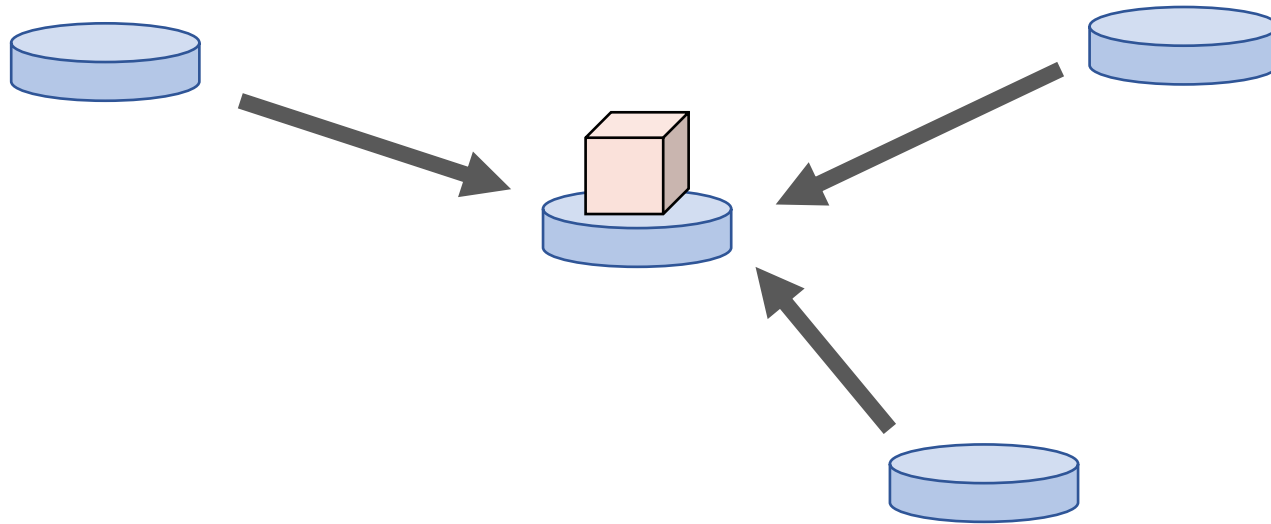
既知の辺は製品側に向けておく



$$N = 30$$

辺の向きが確定しないケース 2

既に確定した辺だけだと困る

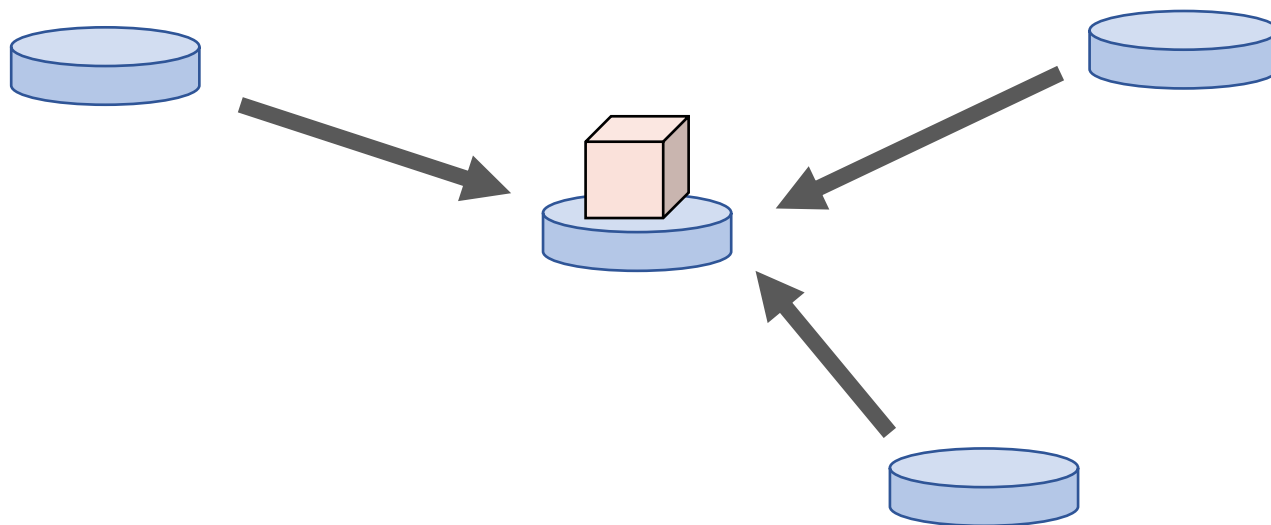


$$N = 30$$

辺の向きが確定しないケース 2

既に確定した辺だけだと困る

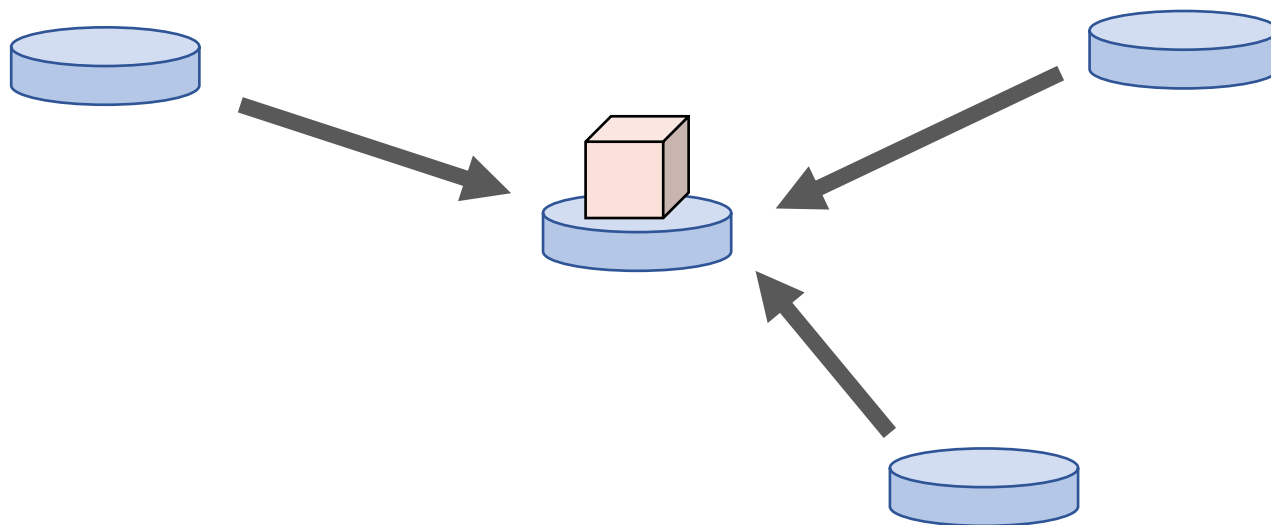
未確定の辺に接続する頂点を選んで製品を置く



$$N = 30$$

クエリの度に、少なくとも **1** 本の辺が確定する

辺は $N - 1 = 29$ 本 $\rightarrow$ クエリは **29** 回以内



$N = 100\,000$, パスグラフ

1 回のクエリで同時に何本もの辺を確定させる必要がある

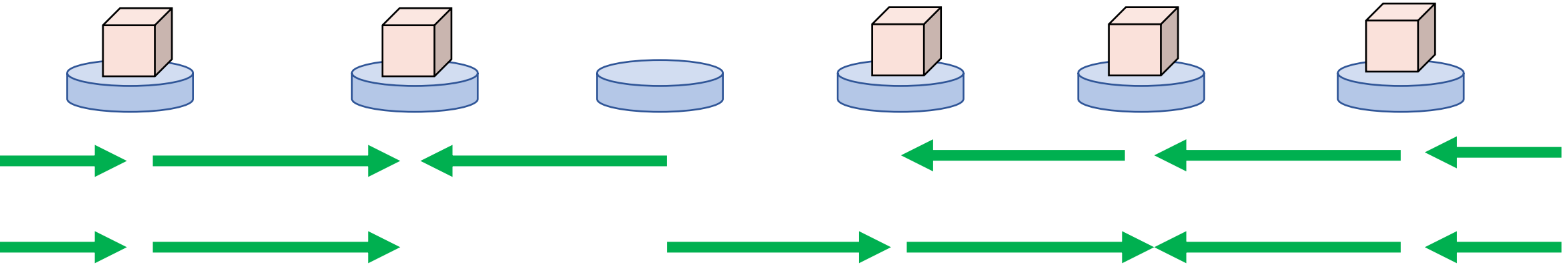
同時に複数の頂点に製品を置きたい



$N = 100\,000$, パスグラフ

1回のクエリで同時に何本もの辺を確定させる必要がある

同時に複数の頂点に製品を置きたい



どの製品がどう動いたのか分からない

$N = 100\ 000$, パスグラフ

1 回のクエリで同時に何本もの辺を確定させる必要がある

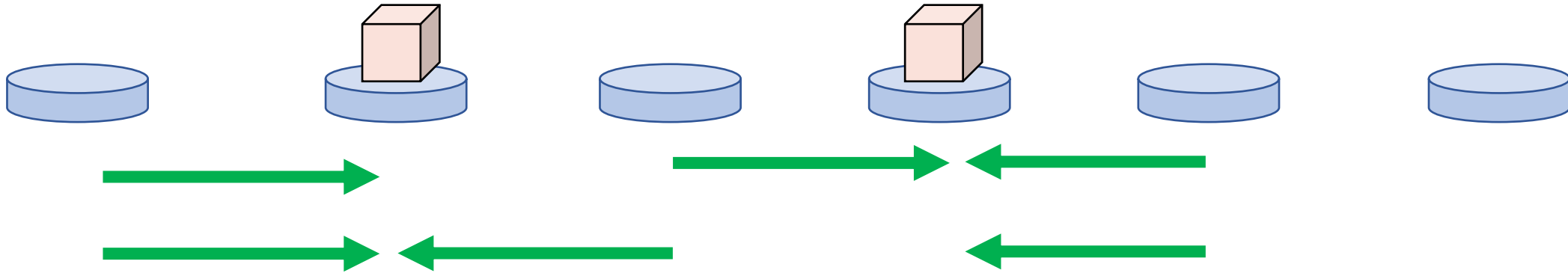
1 個おきに置くとどうか？



$N = 100\,000$, パスグラフ

1 回のクエリで同時に何本もの辺を確定させる必要がある

1 個おきに置くとどうか？



中央の製品がどちらに動いたのか分からない

$N = 100\,000$, パスグラフ

1 回のクエリで同時に何本もの辺を確定させる必要がある

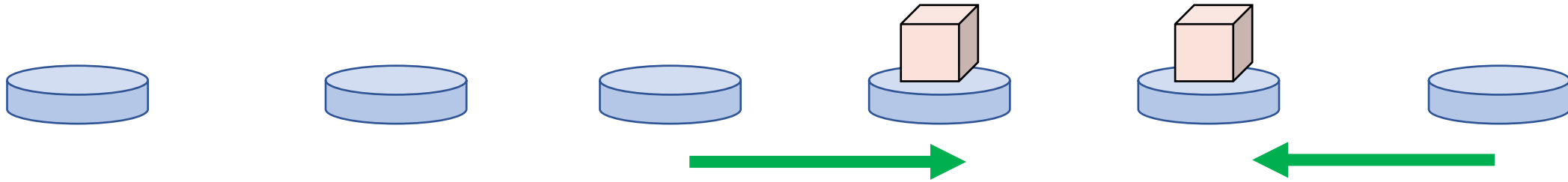
2 個おきに置くとどうか？



$N = 100\,000$, パスグラフ

1 回のクエリで同時に何本もの辺を確定させる必要がある

2 個おきに置くとどうか？

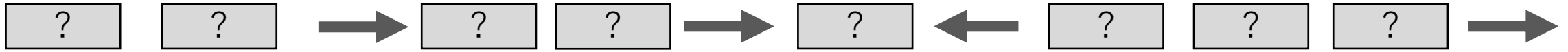


製品同士が干渉しないので、 $N/3$ 本の辺が確定する

$N = 100\,000$, パスグラフ

1 回のクエリで同時に何本もの辺を確定させる必要がある

2 個おきに置くとどうか？



これを繰り返して全ての辺を確定させる

$N = 100\,000$, パスグラフ

1 回のクエリで同時に何本もの辺を確定させる必要がある

2 個おきに置くとどうか？



未確定の辺に接続する頂点を、製品が **2** つおき以上離れるように貪欲に選ぶ

製品が隣接しないから、既知の辺を製品側に向ける処理も問題ない

$N = 100\,000$, パスグラフ

1 回のクエリで同時に何本もの辺を確定させる必要がある

2 個おきに置くとどうか？

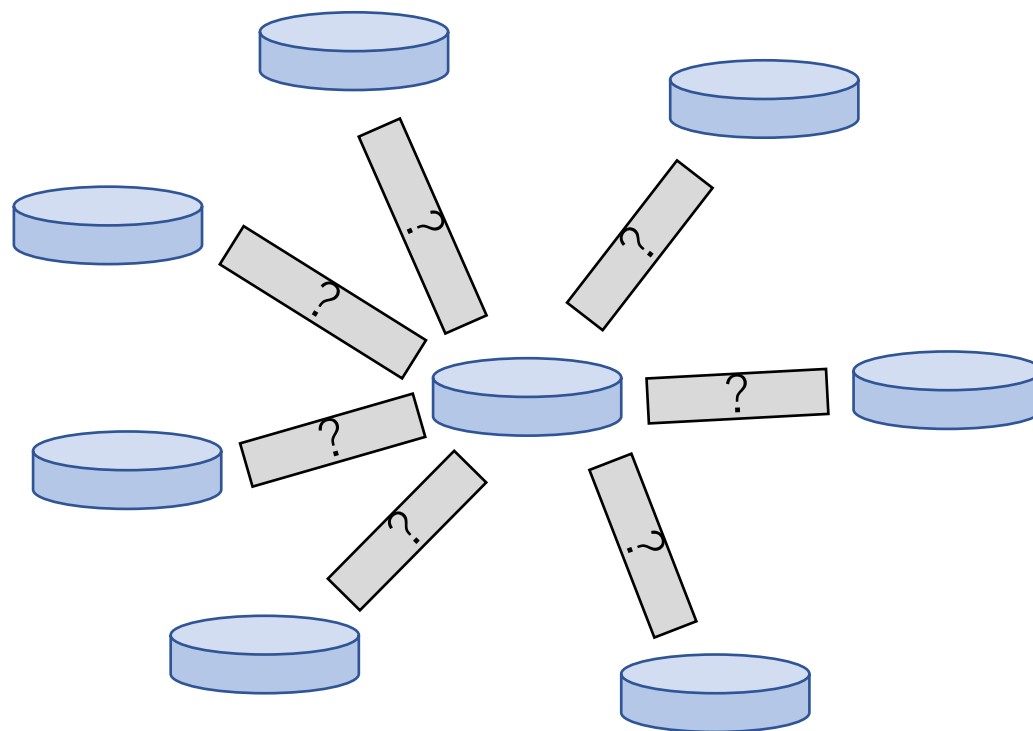


未確定の辺に接続する頂点を、製品が **2** つおき以上離れるように貪欲に選ぶ

4 回以内に全て確定する (証明略)

$$N = 100\,000$$

2 個おきに置く戦略を活用したい

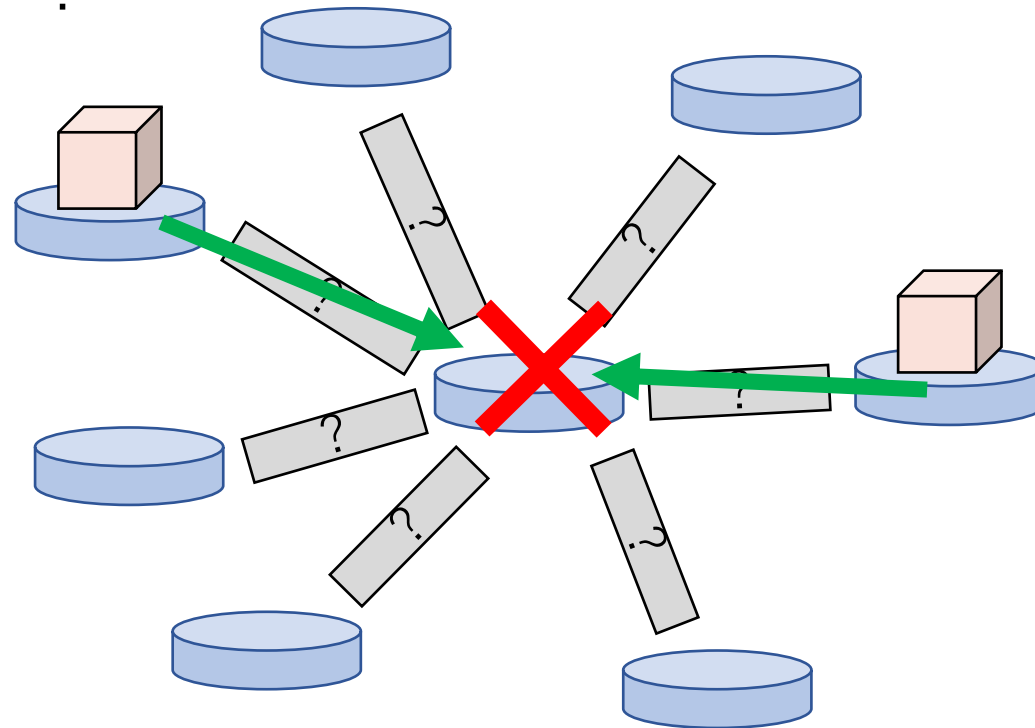


$N = 100\,000$

2 個おきに置く戦略を活用したい

製品の行き先が重ならないようにすると、スターグラフで困る

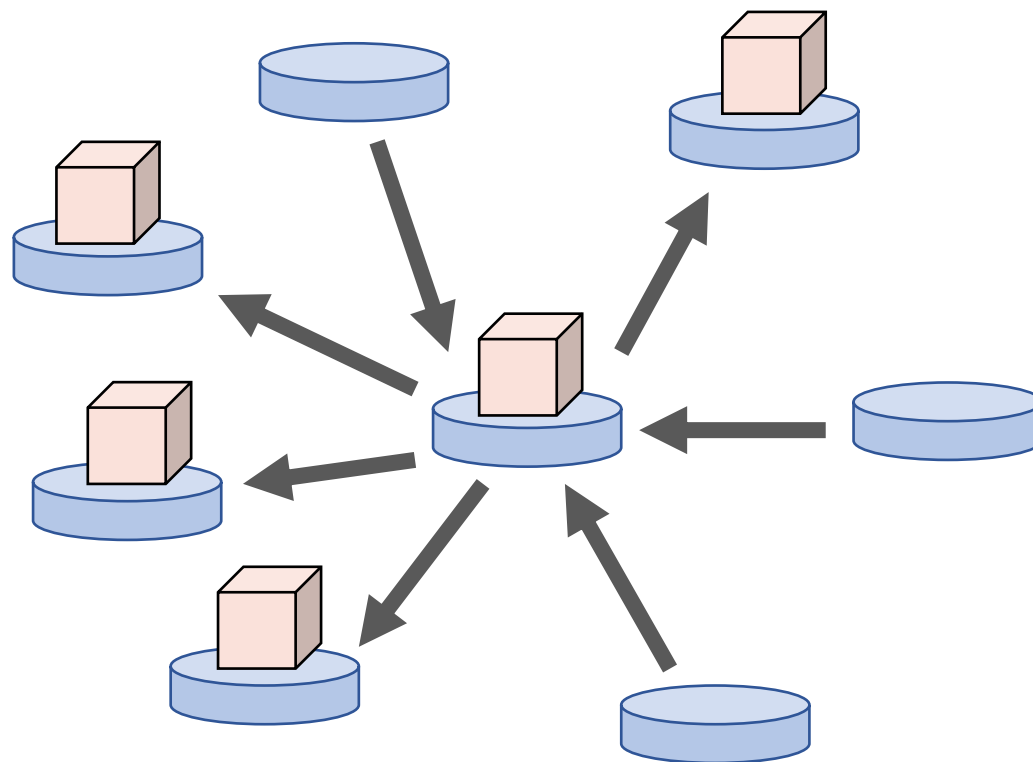
1 個ずつしか置けない！



$$N = 100\,000$$

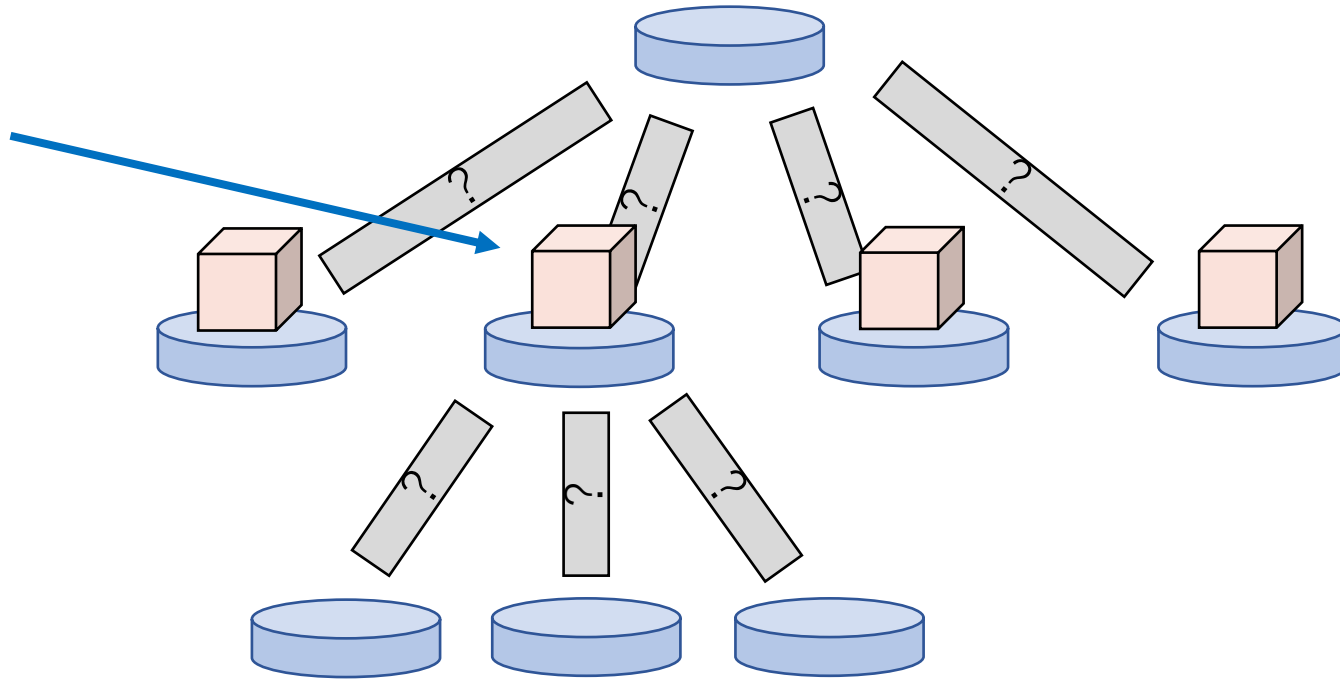
2 個おきに置く戦略を活用したい

スターグラフなら、葉に置いた製品が残っているか見れば、
中心で合流してしまっても大丈夫



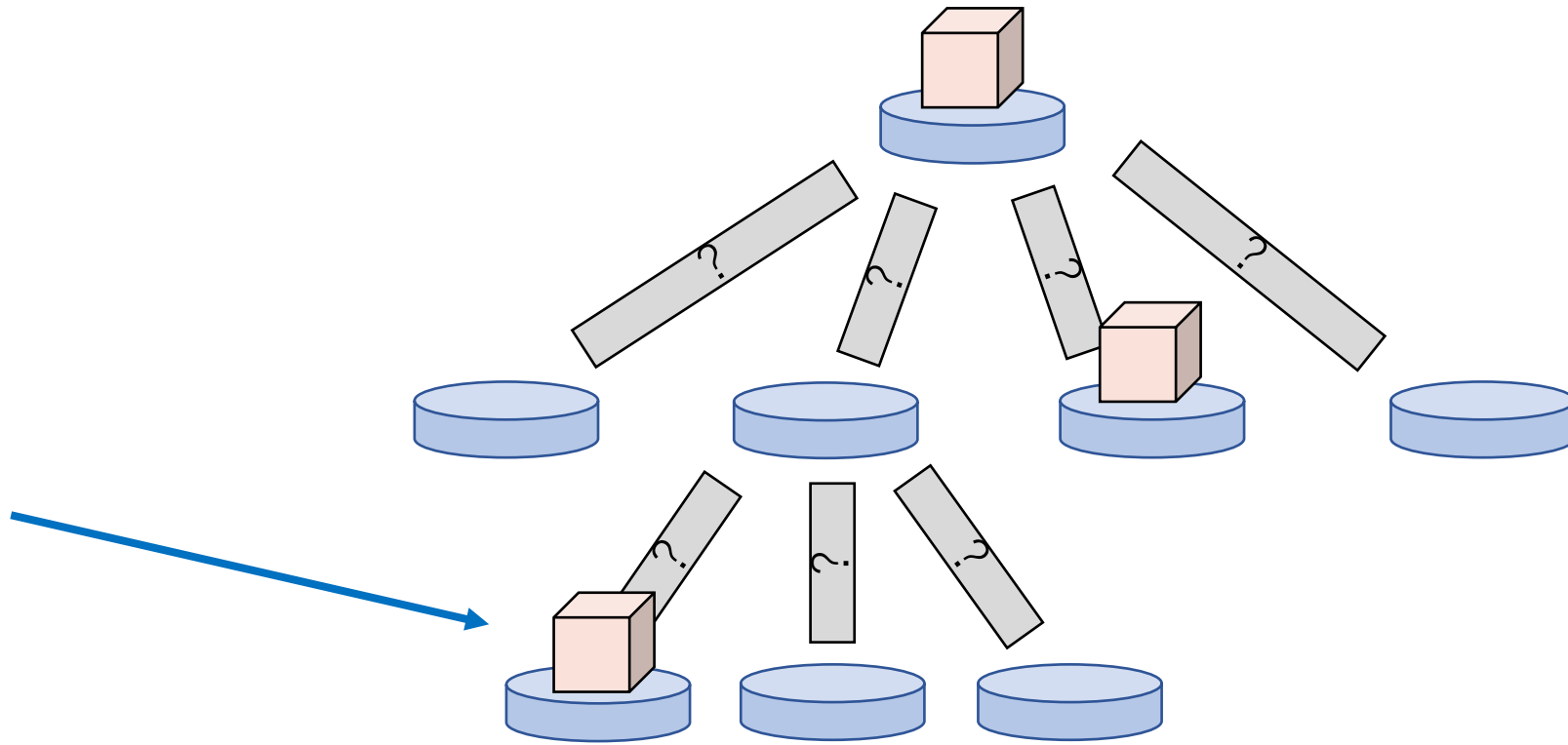
$$N = 100\,000$$

一般の木に対しても、似たような戦略が使えないだろうか？



$$N = 100\,000$$

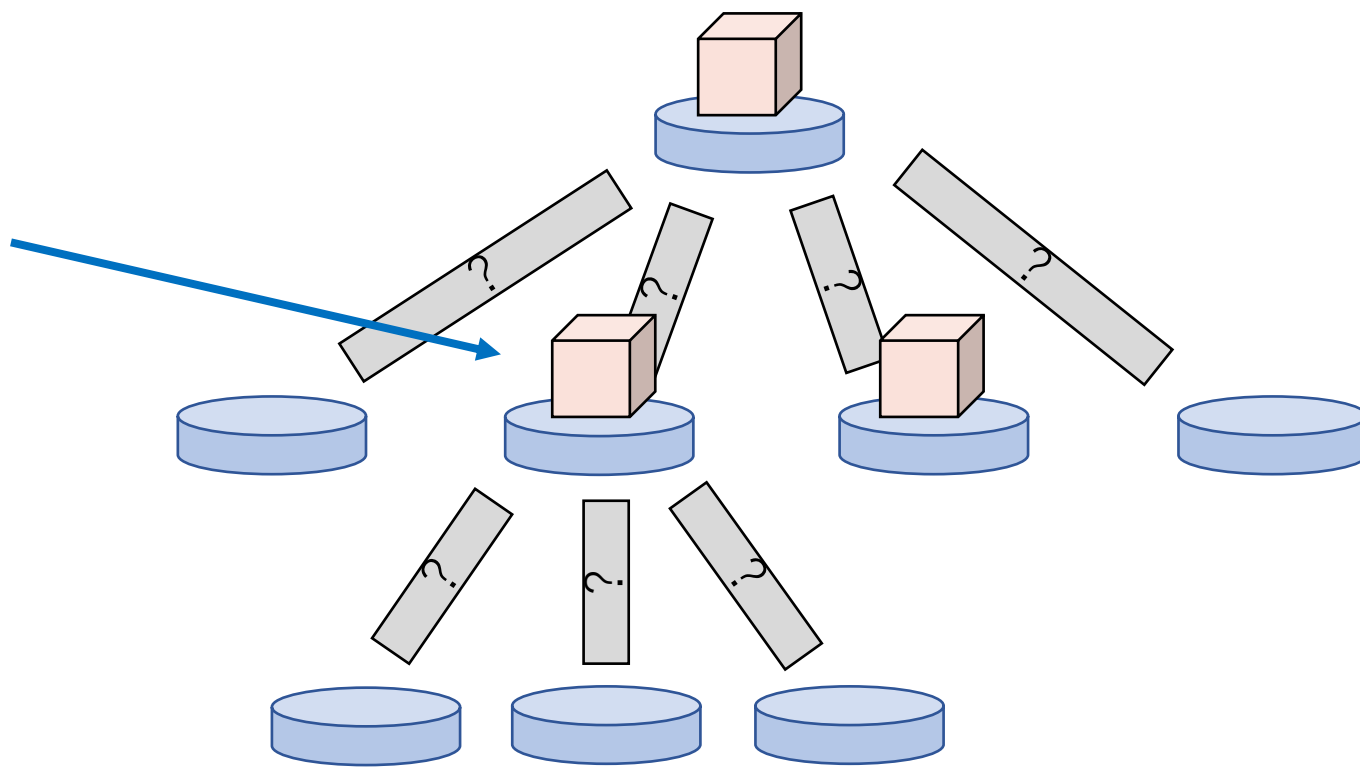
一般の木に対しても、似たような戦略が使えないだろうか？



下の台のどれかに移動していたら分かる。

$$N = 100\,000$$

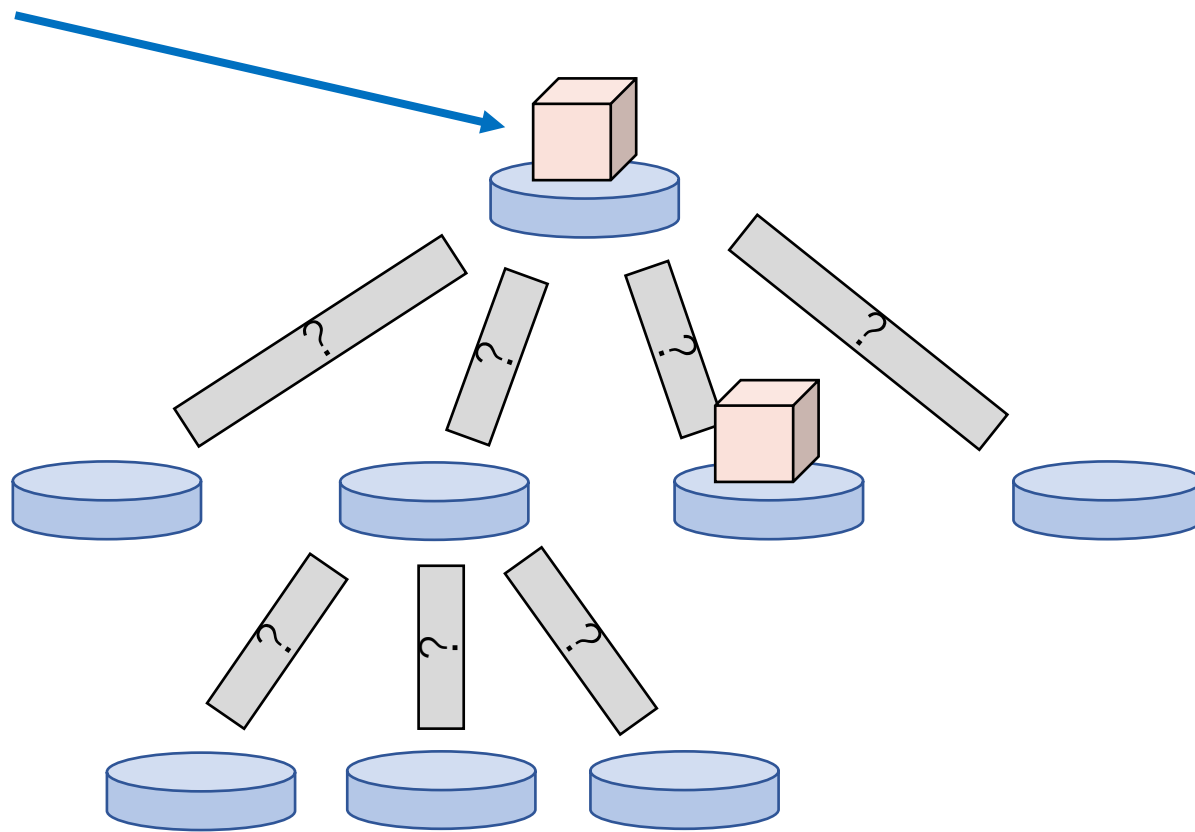
一般の木に対しても、似たような戦略が使えないだろうか？



置いた場所に留まっていた場合も分かる

$$N = 100\,000$$

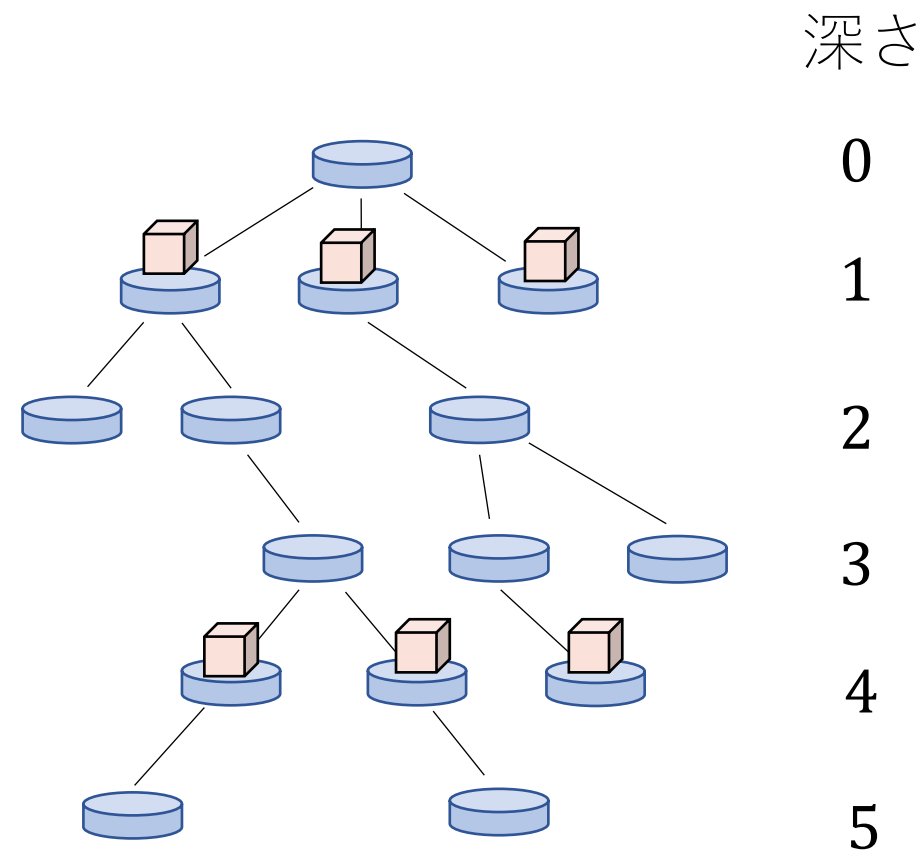
一般の木に対しても、似たような戦略が使えないだろうか？



上の台に行っていた場合、消去法で分かる。

$$N = 100\,000$$

根付き木にして、深さを **2** つおきに選び、製品を置く

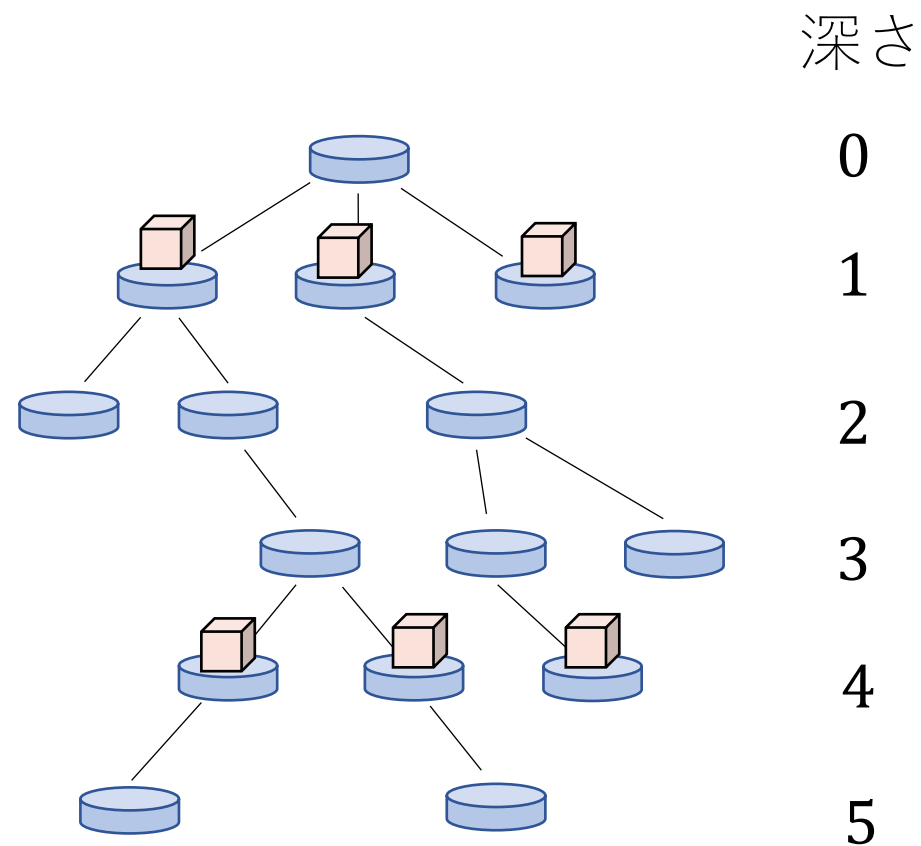


$$N = 100\,000$$

根付き木にして、深さを **2** つおきに選び、製品を置く

深さを mod **3** で分類し、最も大きいグループに製品を置く

$N/3$ 本以上の辺が確定する



$$N = 100\,000$$

未確定の辺に接続する頂点たちを深さを mod 3 で分類し、
最も大きいグループに製品を置く

- ・ 製品は隣接しない
 - ・ 既知の辺は邪魔にならない
 - ・ **1** 製品につき少なくとも **1** 本の辺が確定する
- ・ 未確定の辺を **E** 本とすると、接続する頂点は **E** 個以上ある
→ 少なくとも **$E/3$** 本の辺が新たに確定する

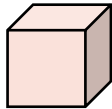
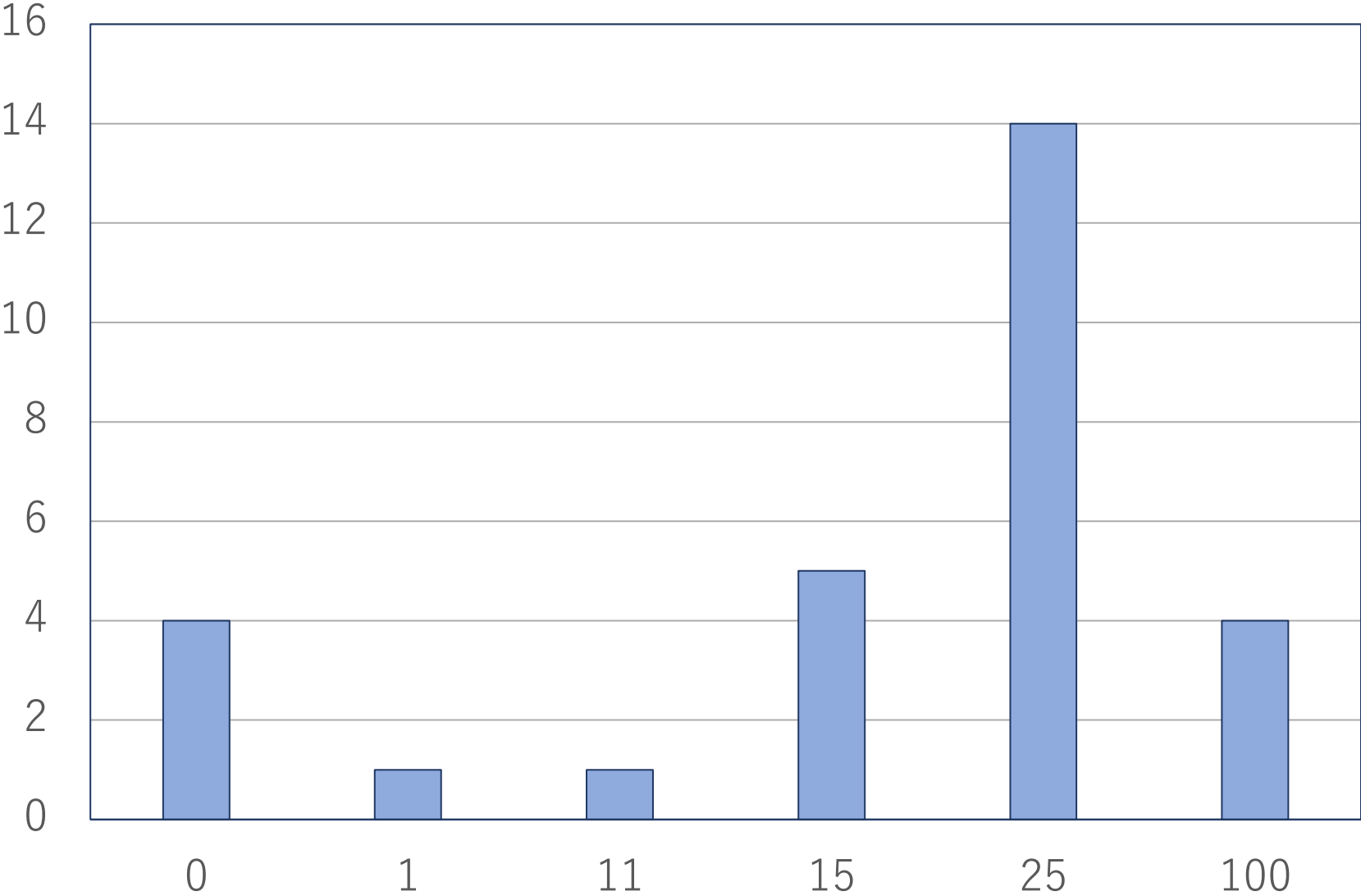
$$N = 100\,000$$

未確定の辺に接続する頂点たちを深さを mod 3 で分類し、
最も大きいグループに製品を置く

- ・ 製品は隣接しない
 - ・ 既知の辺は邪魔にならない
 - ・ 1 製品につき少なくとも 1 本の辺が確定する
- ・ 未確定の辺を E 本とすると、接続する頂点は E 個以上ある
→ 少なくとも $E/3$ 本の辺が新たに確定する

$\log_{1.5} N = 28.39\dots$ だから、30 回以内に全ての辺が確定する

統計情報



JOIG2022/2023 春季トレーニング 競技2日目

Smartphone解説



清水健吾 (Thistle)

問題概要

- N 台のスマホがあり、 i 番目のスマホは区間 $[A_i, B_i]$ のうち C_i 日間使える。
- $[1, K]$ の内最大何日間スマホを使えるか？

制約	小課題	特殊制約	N	K
	1	$B_i < A_j$ or $B_j < A_i$ ($i < j$)	1000	1000
	2	$C_i = B_i - A_i + 1$	300000	10^{15}
	3	$A_i = 1$	1000	1000
	4		1000	1000
	5		300000	300000
	6 (満点)		300000	10^{15}

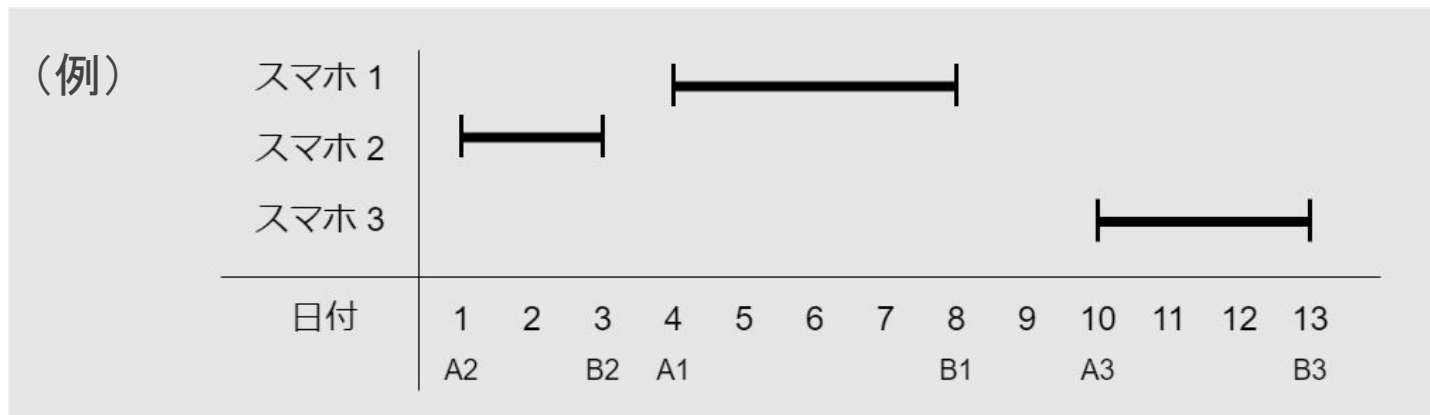
小課題 1

制約 $B_i < A_j$ or $B_j < A_i$ ($i < j$) を読み解こう。



$A_i < B_i$, $A_j < B_j$ と合わせると.....

$(A_i < B_i) < (A_j < B_j)$ or $(A_j < B_j) < (A_i < B_i) \Rightarrow$ 区間が被っていない！



小課題 1

他のスマホのことを考えずに使うことが出来る。

スマホを C_i 日未満使うメリットはないため、全て使うことにする。

⇒ C_i の総和を出力すればよい

⇒ **AC!**

小課題 2

制約は $C_i = B_i - A_i + 1 \Rightarrow$ 区間全てで使うのが最適

問題文の言い換えをすると、

N 個の区間が与えられる。1 つ以上の区間で被覆された部分の長さの和を求めよ

よくわからないので、とりあえず愚直解を考えてみよう。

- 長さ K の配列 D を用意し、全ての i に対して $A_i \leq x \leq B_i$ である $D[x]$ に 1 を足す $O(NK)$
- 値が 1 以上の要素の数を数える $O(K)$



小課題 2

愚直解を高速化していこう。

まず、1step目では、**区間**に対して**均一**に 1 を加算するということをしている。

これは、**imos法を使うことで高速にできる**。（セグ木でも出来る）

これで、

- 長さ K の配列 D を用意し全ての i に対して $A_i \leq x \leq B_i$ である $D[x]$ に 1 を足す **$O(K)$**
- 値が 1 以上の要素の数を数える $O(K)$

となった。

小課題 2

区間への加算の様子をよく見てみると、 **A_i, B_i 周辺以外では連続な場所は値が同じである**、ということが分かる。

例:

D : 000000		1111		22222		11111		33333		2222		1111		0000			
A0		A1		B1		A2,A3		B2		B0		B3					

⇒ 値が同じ区間については持っておく情報を 1 つに減らせる。

= **座標圧縮**によって K 日を $O(N)$ 区間にまで減らせる。

小課題 2

これによって愚直解の $O(K)$ 部分が N に依存する計算量に変わり、

- 座標圧縮をする $O(N \log N)$
- 長さ $O(N)$ の配列 D を用意し全ての i に対して $A_i \leq x < B_{i+1}$ である $D[x]$ に 1 を足す $O(N \log N)$
- 値が 1 以上の要素の $D[x+1]-D[x]$ の総和を求める $O(N)$

となる。

座標圧縮時の処理と合わせて $O(N \log N)$ で解くことが出来た。

小課題 2

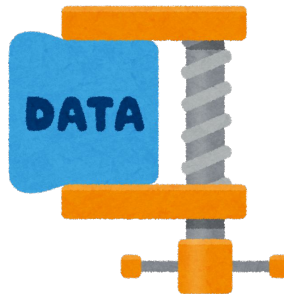
このように、座標圧縮は、

座標の制約の値が大きく、かつ、

入力で与えられた数値で分割した区間の中では操作や性質が同じ時

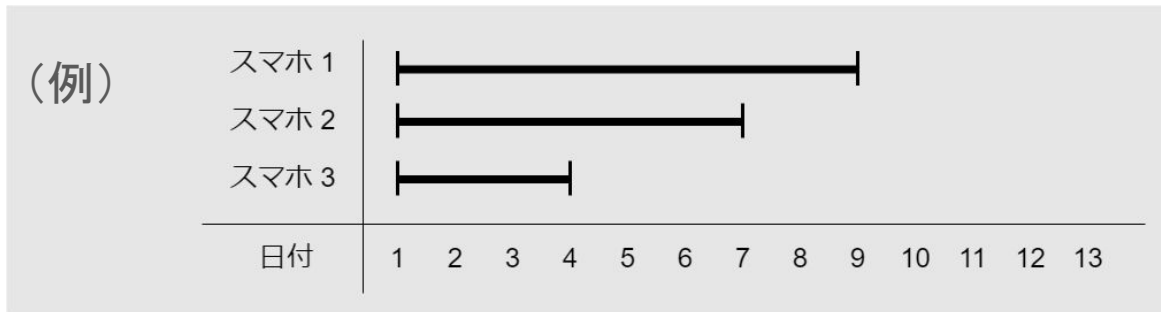
に使うことが出来る。

このような問題を見たら反射で使えるようにしておくといだろう。



小課題 3

制約 $A_i = 1$ の場合の状況を考える。 B_i が大きい順でソートすると、



のようになる。

ここで重要な考察は、

スマホの使用可能区間同士は完全な包含関係にあるということ。



小課題 3

この場合、各日について、使用日が余っているスマホのうち
早く使えなくなる = B_i が小さい順に使うという貪欲法が最適である。

直感的には、 B_i が大きいものは後でも使えるから後に回した方が
得しそうという感じ。

ある日に使えるスマホが2つあったとして、 B_i が大きいスマホは B_i が小さい
スマホの代わりに必ず使えるが、逆は出来ない日がある。

小課題 3

貪欲法の厳密な証明

スマホの使用日数が最大となるあるスマホの使い方において、

1. $B_i < B_j$ なスマホ i, j について、 k 日目にスマホ i の使用日数が残っているにも関わらずスマホ j を使ったとする
 - ・もし $k+1$ 日目以降でスマホ i を使っていないなら、 k 日目はスマホ j の代わりにスマホ i を使う
 - ・もし $k+1$ 日目以降でスマホ i を使っているなら、その日と使うスマホを交換する
2. k 日目にスマホ i の使用日数が残っているにも関わらずどのスマホも使わなかったとする
 - ・もし $k+1$ 日目以降でスマホ i を使っていないなら、なにもしない
 - ・もし $k+1$ 日目以降でスマホ i を使っているなら、スマホ i を使っている日をその日と交換する

この交換を繰り返すことで、 B_i が小さく使用日数が余っているスマホから貪欲に選ぶ使い方に帰着される

ソートがボトルネックとなり、 $O(K + N \log N)$ でこの小課題が解けた。

小課題 3

各日について考えるということをしなくても、この小課題だけなら

区間は終点でソート

という格言に従い、終点の早い区間から順に、前から C_i 日割り当てていく、
という方法でも解くことが出来る。

小課題 4 : $N, K \leq 1000$

特殊な制約はない。

小課題 3 で行った考察は $A_i = 1$ でなくても成立する。

よって、各日について、今使えるスマホの中で最も B_i が小さいものを選んで使用する、という貪欲法で解くことが出来る。

計算量は $O(NK)$



小課題 5 : $N, K \leq 300000$



N と K の値が大きくなり、 $O(NK)$ では通らなくなった。

各日について、というところは変えられないので、 $O(K)$ は削れない。

そこで、最も B_i が小さいスマホを選ぶ、というパートを削減することにしよう。

こういったときに使用できるデータ構造に、**priority_queue** というものがある。

- ・1 日目から順に探索し、 A_i 日目になったら prique にスマホ i を入れる
- ・その日使うスマホを選ぶため、prique 内で B_i 最小のスマホを確認する
もし使えなかったらそのスマホを取り出し、次のスマホを確認

とすることで全体で $O(K + N \log N)$ で実行できる。

小課題 6 (満点) : $N \leq 300000$, $K \leq 10^{15}$

K が大きくなったため、計算量に K が入ることも許されなくなった。

この問題の特徴は、

- ・ K の制約が大きすぎる
- ・ A_i, B_i で区切られた区間内の日の性質は均質である

ということで、**座標圧縮**をしよう。

計算量は **$O(N \log N)$** となり、この問題を解くことが出来た



得点分布



0点

15点

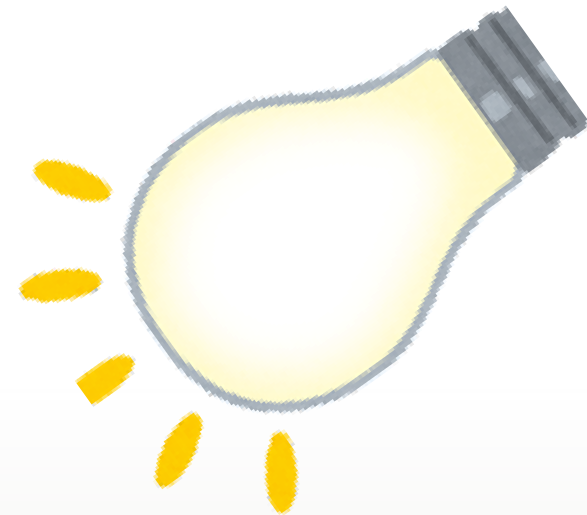
25点

45点

55点

70点

100点



白色光 (White Light)

JOIG 2022/2023 春合宿 競技2-2

解説：大野栞 / 萩原千晴

出典：いらすとや



問題概要

- ・ 赤(R)、緑(G)、青(B)のライトが計N個並んでいる
- ・ 1～K個の連続したライトを選び、1回の操作で消せる
- ・ R, G, Bの順に点灯し、Bで終了するようにしたい
- ・ 操作の最小回数を出力



小課題

- ◆ 小課題 1 $N \leq 16$, $K=1$...10点
- ◆ 小課題 2 $N \leq 200000$, $K=1$...26点
- ◆ 小課題 3 $N \leq 200000$, $K \leq 100$...26点
- ◆ 小課題 4 $K \leq N \leq 200000$...38点

小課題

- ◆ 小課題 1 $N \leq 16, K=1$...10点
- ◆ 小課題 2 $N \leq 200000, K=1$...26点
- ◆ 小課題 3 $N \leq 200000, K \leq 100$...26点
- ◆ 小課題 4 $K \leq N \leq 200000$...38点

小課題 1

◆ $N \leq 16$... $O(2^N)$ でも間に合う



bit 全探索

◆ 二進法で、各ライトを消すか消さないか管理

◆ 全通り調べて、消すライトの最小数を出力

($\because K=1$)

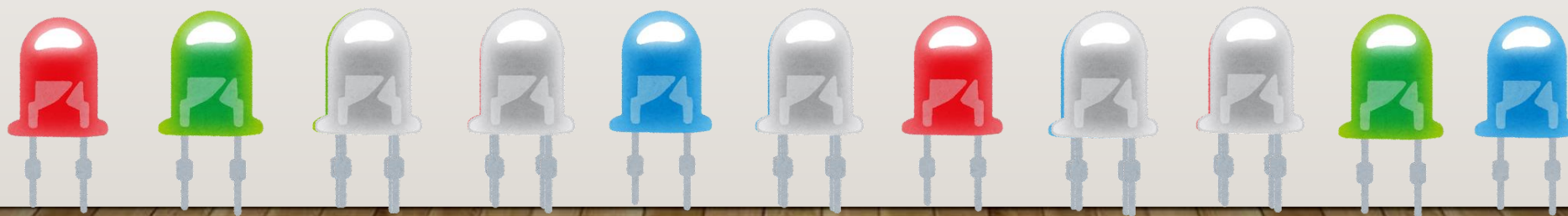
小課題

- ◆小課題 1 $N \leq 16$, $K=1$...10点
- ◆小課題 2 $N \leq 200000$, $K=1$...26点
- ◆小課題 3 $N \leq 200000$, $K \leq 100$...26点
- ◆小課題 4 $K \leq N \leq 200000$...38点

小課題 2

- ◆ $K=1 \rightarrow$ 消すライトの数をなるべく減らすだけ
- ◆ 左から順に見ていき、RGBの順になっている限り消さない

➡ 貪欲法 ... $O(N)$



小課題 2 <貪欲法>

- ◆ Bで終わらなければいけない



- ◆ RGBRGB...Bとなっているなら、点灯しているライトの数は3の倍数

→ (ライトの総数) - (消したライトの数) を3で割った余り

の分だけ後ろのライトを消灯すれば、RGBRGB...Bとなる

★出力だけ調整すればOK★

小課題 2 <DP編>

以下

- 文字列 RGBRGB...R : 文字列R
- 文字列 RGBRGB...RG : 文字列G
- 文字列 RGBRGB...RGB : 文字列B

と呼ぶ（長さは任意）

小課題 2 <アルゴリズム一例>

$dp[i][0]$

... $S[0] \sim S[i-1]$ の部分列を、文字列 **R** にするために消す文字数の最小値

$dp[i][1]$

... $S[0] \sim S[i-1]$ の部分列を、文字列 **G** に... 略)

$dp[i][2]$

... $S[0] \sim S[i-1]$ の部分列を、文字列 **B** に... 略)

※ $dp[i][j] = \text{INF}$
... 不可能を示唆

※ $dp[0][2] = 0$
(一つも消灯しないのもOK)

小課題 2 <アルゴリズム一例>

例えば $S[i]$ が **R** の時

① $S[0] \sim S[i-1]$ の部分文字列 **B** に、 $S[i]$ を加えて文字列 **R** にする

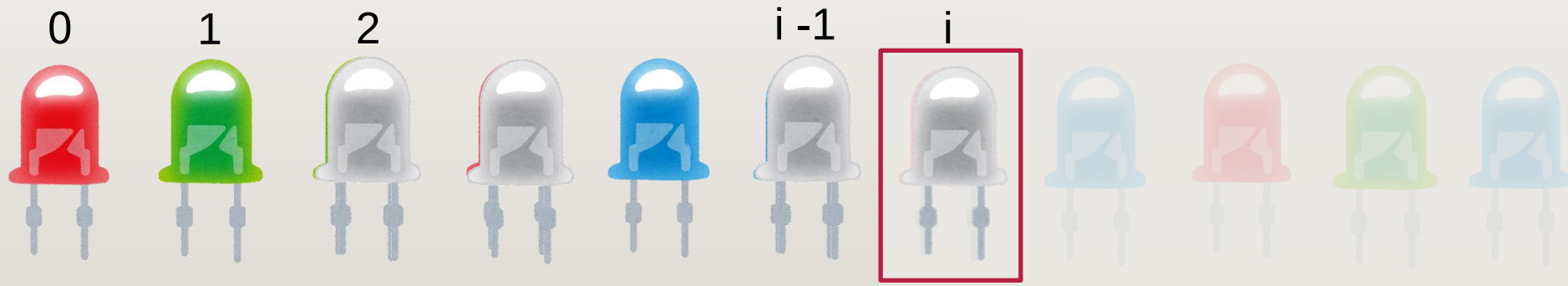
➡ $dp[i+1][0] = \min\{dp[i+1][0], dp[i][2]\}$



小課題 2 <アルゴリズム一例>

- ② $S[i]$ は消灯して、 $S[0] \sim S[i-1]$ の部分文字列Bをそのまま使う
(文字列G,Rでもやる)

→ $dp[i+1][2] = \min\{ dp[i+1][2], dp[i][2] + 1 \}$



小課題

- ◆小課題 1 $N \leq 16$, $K=1$...10点
- ◆小課題 2 $N \leq 200000$, $K=1$...26点
- ◆小課題 3 $N \leq 200000$, $K \leq 100$...26点
- ◆小課題 4 $K \leq N \leq 200000$...38点

小課題 3

$O(N * K)$ で間に合いそう！

➡ K文字前までさかのぼりながらDPできる

☆小課題 2 のDPをそのまま使える☆

①はそのまま...

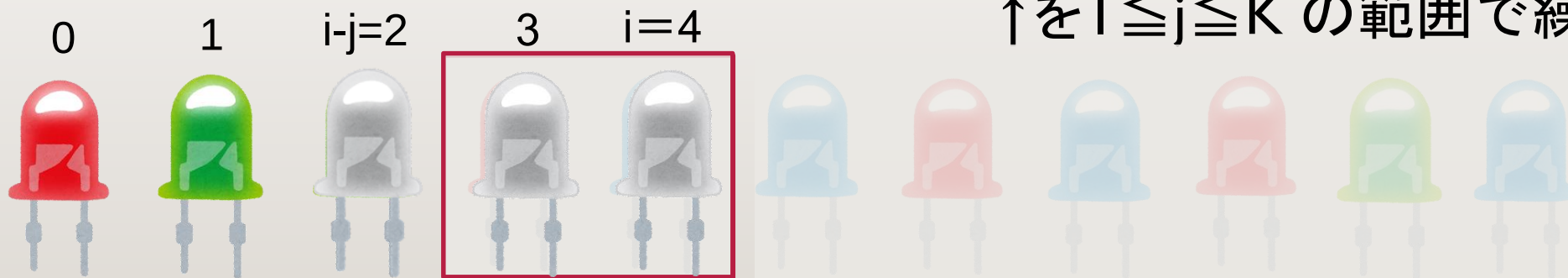
小課題 3

j 文字分さかのぼる

② $S[i-j+1] \sim S[i]$ は消灯して、 $S[0] \sim S[i-j]$ の部分文字列 B をそのまま使う
(文字列 B, R でもやる)

$$\rightarrow dp[i+1][2] = \min\{ dp[i+1][2], dp[i-j+1][2] + 1 \}$$

↑を $1 \leq j \leq K$ の範囲で繰り返す



小課題

- ◆小課題 1 $N \leq 16, K=1$...10点
- ◆小課題 2 $N \leq 200000, K=1$...26点
- ◆小課題 3 $N \leq 200000, K \leq 100$...26点
- ◆小課題 4 $K \leq N \leq 200000$...38点

小課題 4

$O(N * K)$ はTLE

➡ K回の for 文をなくしたい

`for( j=1; j≤K; j++ ) dp[ i+1 ][ 2 ] = min{ dp[ i+1 ][ 2 ] , dp[ i-j+1 ][ 2 ] +1 }`

は結局、

`dp[ i+1 ][ 2 ] = min{ dp[ i+1 ][ 2 ] , min{ dp[ i-j+1 ][ 2 ] | 1≤j≤K } +1 }`

小課題 4

スライド最小値
という有名問題

$\min\{ dp[i-j+1][2] \mid 1 \leq j \leq K \}$ を $O(1) / O(\log N)$ で得たい！

★ deque (que[0]~que[2]) で記憶しておく

i から K 以上離れた位置のデータは使えない = 位置のデータが必要

➡ 更新した $dp[i][j]$ は、 $\{i, dp\text{値}\}$ のペアにして $que[j]$ に後ろから追加

$i_1 < i_2, dp[i_1] \geq dp[i_2]$ となったら、 $dp[i_1]$ のデータは捨てる

➡ que は、i が昇順、dp 値が降順になる ➡ $que[j]$ の最後尾が、求める最小値

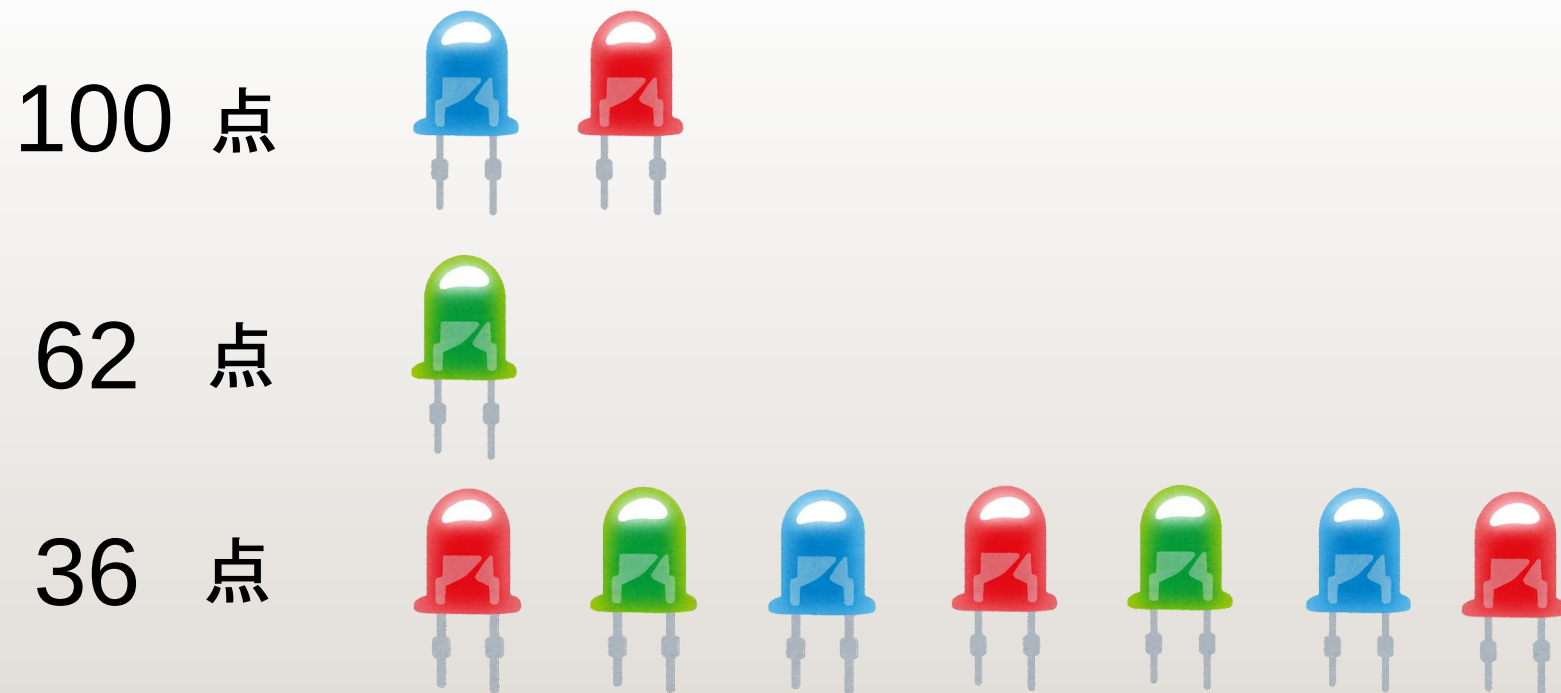
小課題 4 （別解）

$\min\{ dp[i-j+1][l] \mid l \leq j \leq K \}$ を $O(1)$ / $O(\log N)$ で得たい！

区間の最小値を求める  セグメント木！

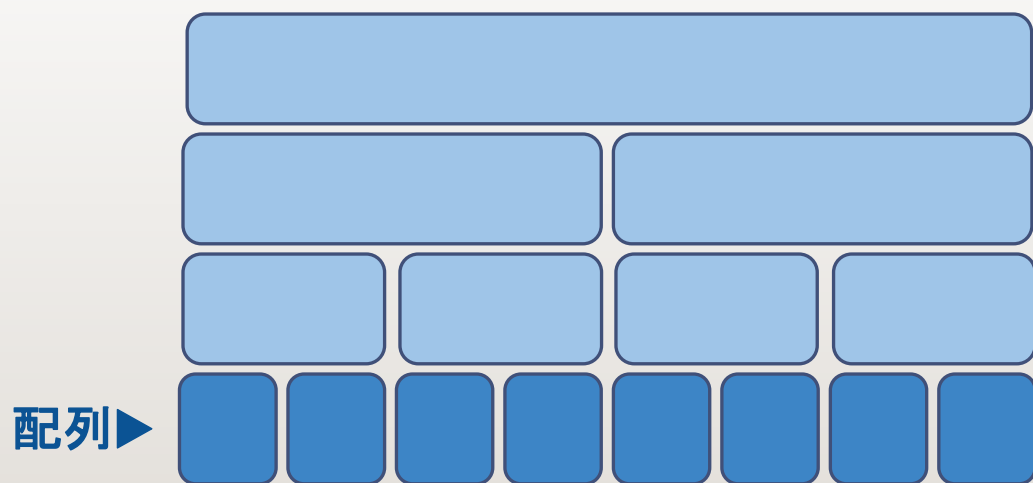
($O(\log N)$ でデータを更新、 $O(\log N)$ で任意の区間の最小値を求める)

得点分布



おまけ：セグメント木って何??

→ 配列を高速に処理できる便利なデータ構造！



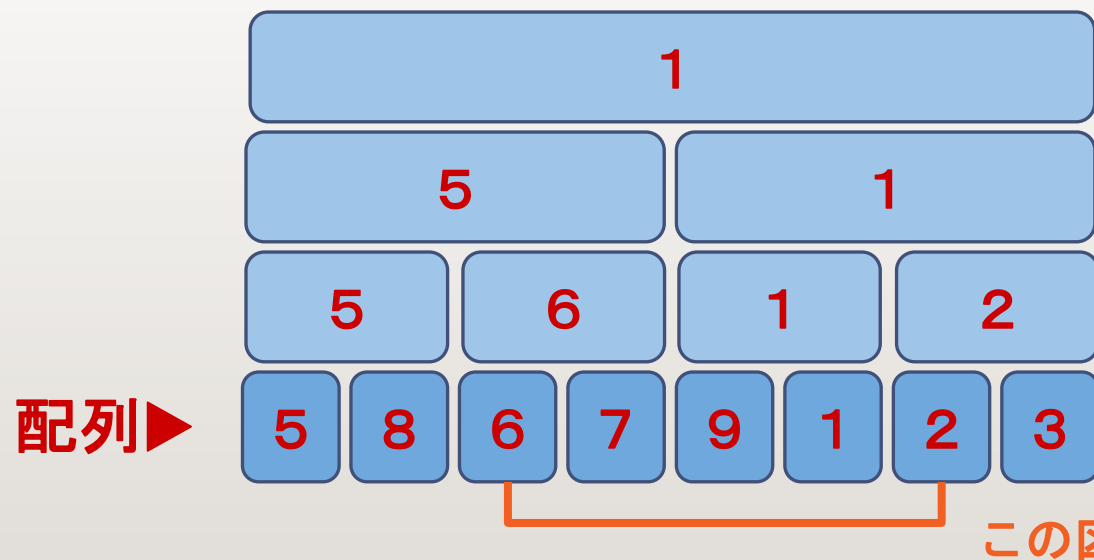
左のように
半々ずつを管理していく木を作る

ある区間での最小値を求めたり
要素の総和を求めたりするこ
とが

$O(\log N)$ の計算量でできる！

おまけ：セグメント木って何??

具体的にやってみよう！

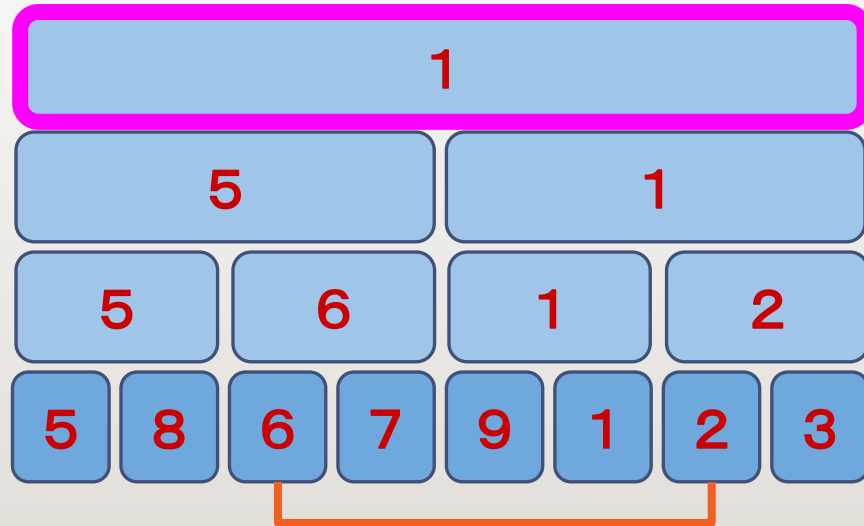


区間最小値を求める場合はこんな感じ

区間最小値クエリ：
RMQ (Range Minimum Query)

おまけ：セグメント木って何??

具体的にやってみよう！



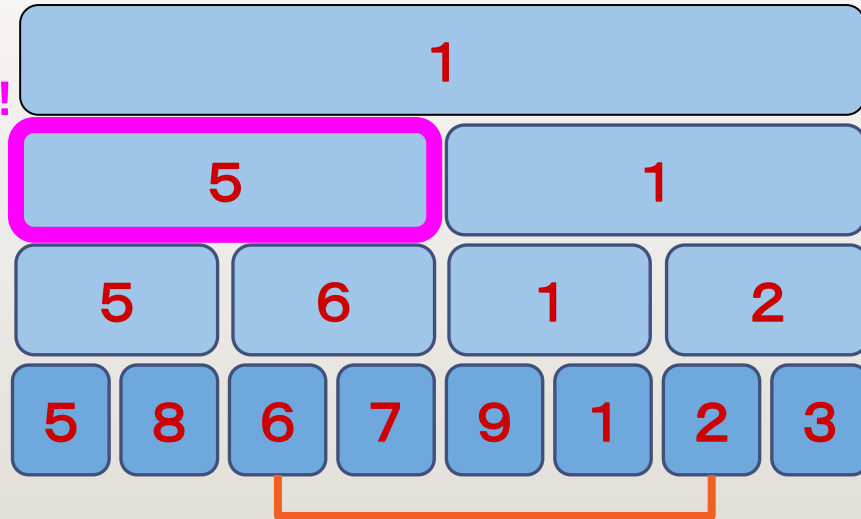
はみ出てる！

根から見てゆき、
各頂点が管理する区間と
今考えている区間を見比べる

おまけ：セグメント木って何??

具体的にやってみよう！

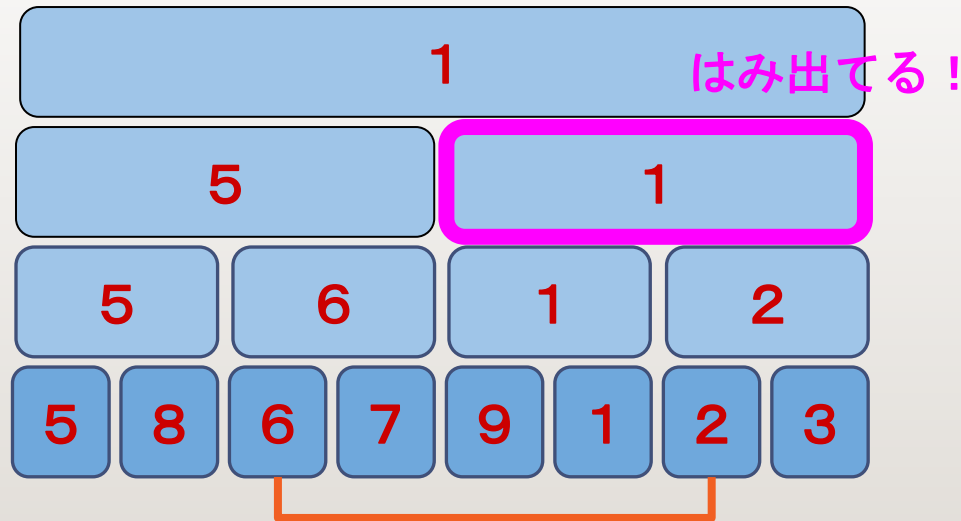
はみ出てる！



根から見てゆき、
各頂点が管理する区間と
今考えている区間を見比べる

おまけ：セグメント木って何??

具体的にやってみよう！



根から見てゆき、
各頂点が管理する区間と
今考えている区間を見比べる

おまけ：セグメント木って何??

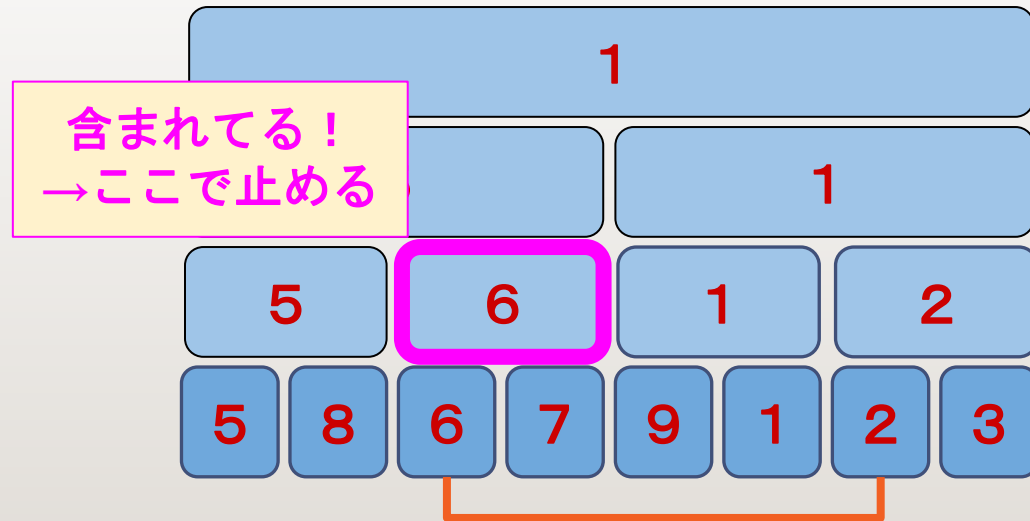
具体的にやってみよう！



根から見てゆき、
各頂点が管理する区間と
今考えている区間を見比べる

おまけ：セグメント木って何??

具体的にやってみよう！



根から見てゆき、
各頂点が管理する区間と
今考えている区間を見比べる

おまけ：セグメント木って何??

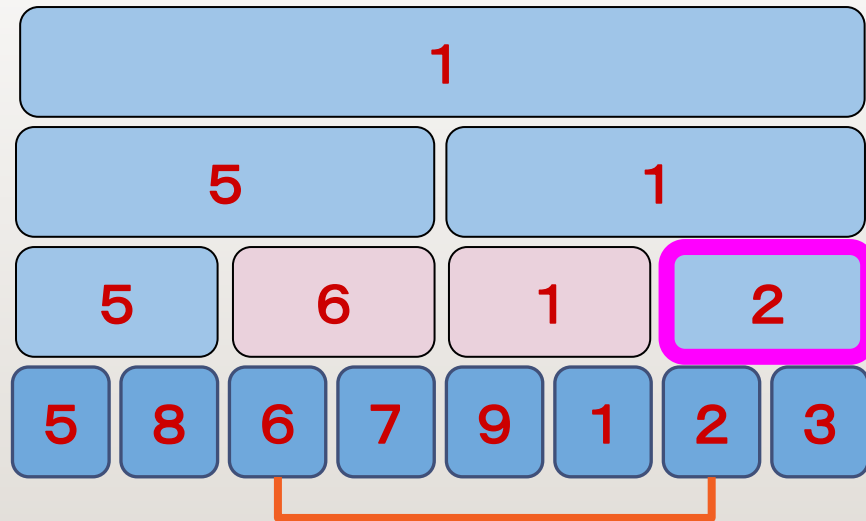
具体的にやってみよう！



根から見てゆき、
各頂点が管理する区間と
今考えている区間を見比べる

おまけ：セグメント木って何??

具体的にやってみよう！

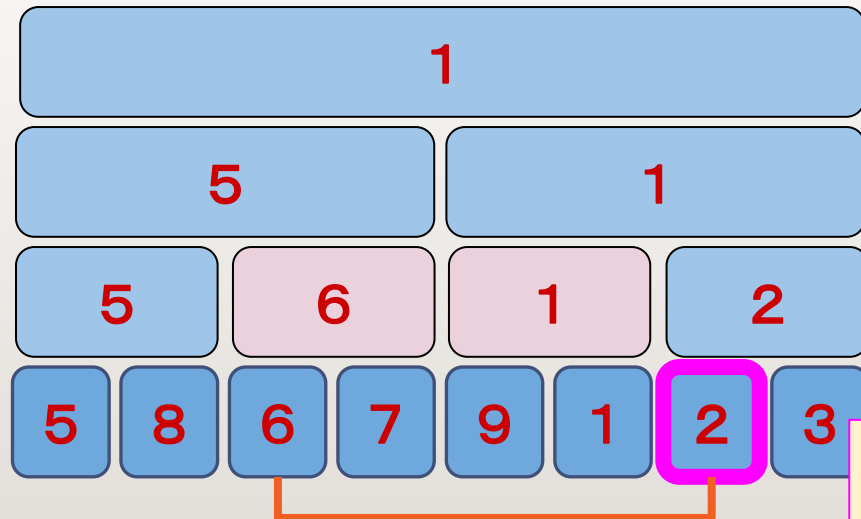


根から見てゆき、
各頂点が管理する区間と
今考えている区間を見比べる

はみ出てる！

おまけ：セグメント木って何??

具体的にやってみよう！

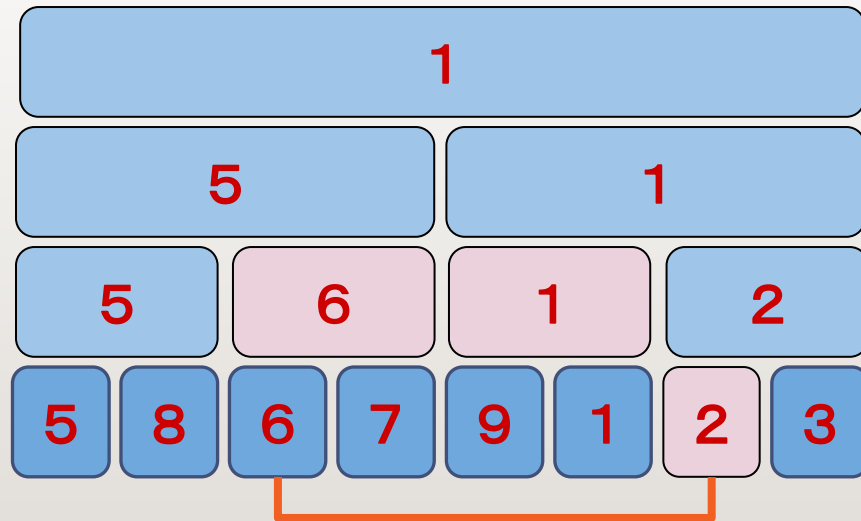


根から見てゆき、
各頂点が管理する区間と
今考えている区間を見比べる

含まれてる！
→ここで止める

おまけ：セグメント木って何??

具体的にやってみよう！



この頂点を見ればOK！

おまけ：セグメント木って何??

具体的にやってみよう！

すごく大きなデータでも
細かく見ずに済む！



おまけ：セグメント木って何??

RMQの実装

値を1箇所更新する時

→ 葉を更新し、親を辿りつつ更新していく

区間最小値を求める時

→ 根から辿り再帰的に見ていく

おまけ：セグメント木って何??

RM

値

一

区

→ 根から辿り再帰的に見ていく

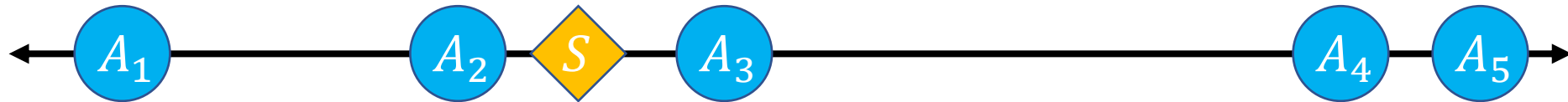
さらに先の実装については割愛します
ぜひ調べてみましょう！

Bitaro's Travel 解説

ynymxiaolongbao

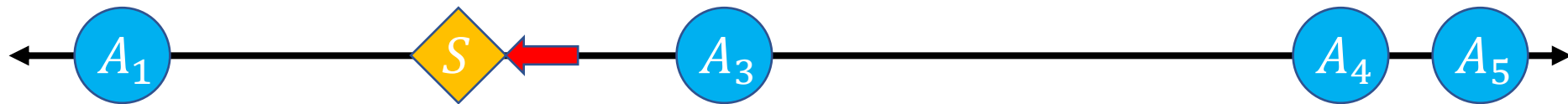
問題概要

- 数直線上に N 個の点 $A_1, A_2, \dots, A_N$ がある
- S からスタートして、「最も近い点に移動して、その点を消す」ことを点がなくなるまで繰り返す
- 移動距離の総和はいくつか？
- これを色々な S について Q クエリ答える



問題概要

- 数直線上に N 個の点 $A_1, A_2, \dots, A_N$ がある
- S からスタートして、「最も近い点に移動して、その点を消す」ことを点がなくなるまで繰り返す
- 移動距離の総和はいくつか？
- これを色々な S について Q クエリ答える



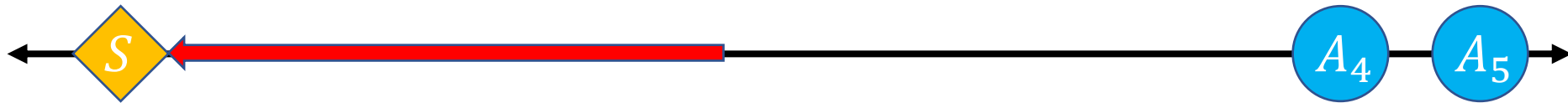
問題概要

- 数直線上に N 個の点 $A_1, A_2, \dots, A_N$ がある
- S からスタートして、「最も近い点に移動して、その点を消す」ことを点がなくなるまで繰り返す
- 移動距離の総和はいくつか？
- これを色々な S について Q クエリ答える



問題概要

- 数直線上に N 個の点 $A_1, A_2, \dots, A_N$ がある
- S からスタートして、「最も近い点に移動して、その点を消す」ことを点がなくなるまで繰り返す
- 移動距離の総和はいくつか？
- これを色々な S について Q クエリ答える



問題概要

- 数直線上に N 個の点 $A_1, A_2, \dots, A_N$ がある
- S からスタートして、「最も近い点に移動して、その点を消す」ことを点がなくなるまで繰り返す
- 移動距離の総和はいくつか？
- これを色々な S について Q クエリ答える



問題概要

- 数直線上に N 個の点 $A_1, A_2, \dots, A_N$ がある
- S からスタートして、「最も近い点に移動して、その点を消す」ことを点がなくなるまで繰り返す
- 移動距離の総和はいくつか？
- これを色々な S について Q クエリ答える



小課題 1 ($N \leq 2000, Q = 1$)

- 点が全てなくなるまで、毎回一番近い点を $O(N)$ で求める
- $O(N^2Q)$



小課題 2 ($N \leq 200000, Q = 1$)

- 途中の状態を見してみる
- 左からいくつかと、右からいくつかの点だけが残っている
- 左側で一番右にある点を A_l 、右側で一番左にある点を A_r とする
- S から一番近いのは A_l か A_r $\rightarrow$ これだけ見ればok
- $O(NQ)$



小課題 2 ($N \leq 200000, Q = 1$)

- 途中の状態を見してみる
- 左からいくつかと、右からいくつかの点だけが残っている
- 左側で一番右にある点を A_l 、右側で一番左にある点を A_r とする
- S から一番近いのは A_l か A_r → これだけ見ればok
- $O(NQ)$



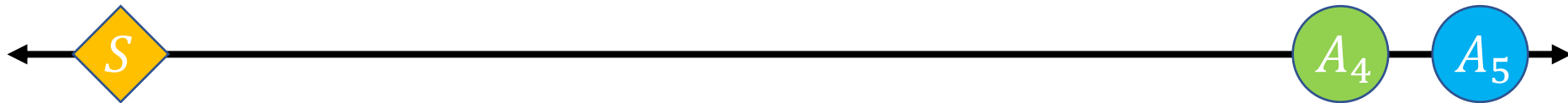
小課題 2 ($N \leq 200000, Q = 1$)

- 左からいくつかと、右からいくつかの点だけが残っている
- **左側で一番右にある点**を A_l 、右側で一番左にある点を A_r とする
- 「左側で一番右にある点」はいつも存在するんですか？



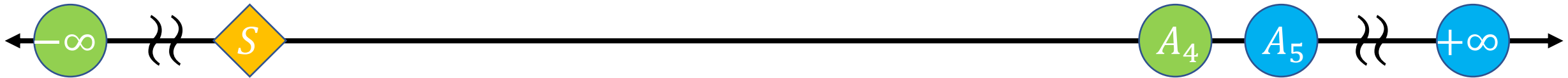
小課題 2 ($N \leq 200000, Q = 1$)

- 左からいくつかと、右からいくつかの点だけが残っている
- **左側で一番右にある点**を A_l 、右側で一番左にある点を A_r とする
- 「左側で一番右にある点」はいつも存在するんですか？
- → **No**



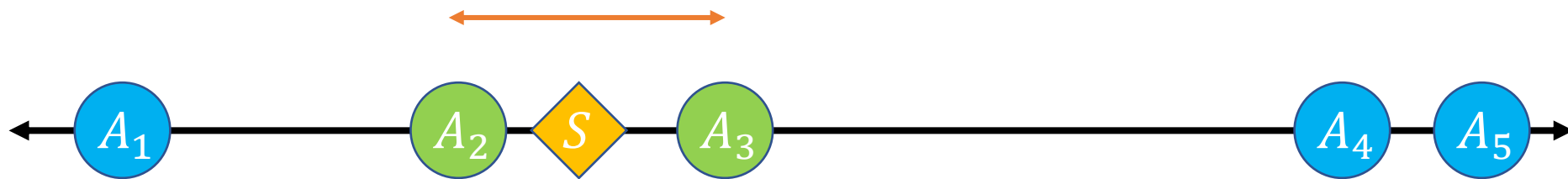
小課題 2 ($N \leq 200000, Q = 1$)

- $-\infty$ と $+\infty$ に点を追加する (**番兵**)
- これらの点は必ず最後に取りられる
- これらの点を取る直前に終了すればよい



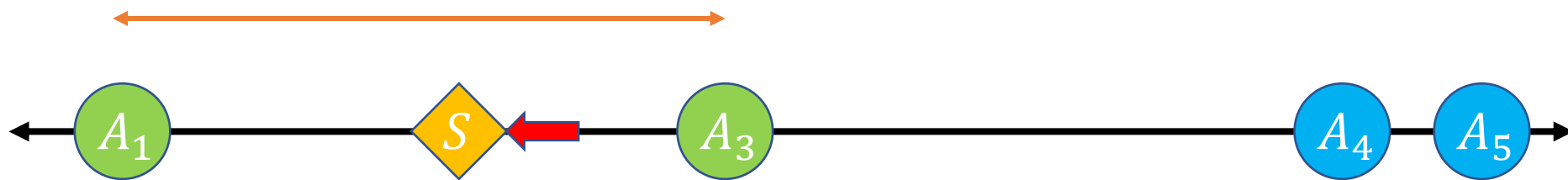
小課題 3 ($A_{i+1} - A_i \leq 100$)

- もう一度経過を見てみる
- A_l と A_r の距離は大きくなっていく



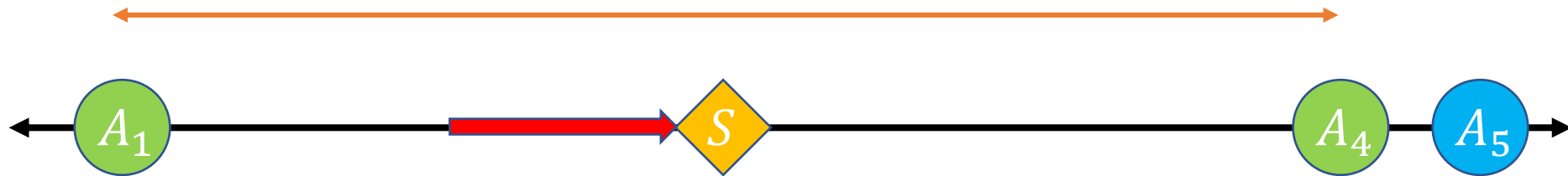
小課題 3 ($A_{i+1} - A_i \leq 100$)

- もう一度経過を見てみる
- A_l と A_r の距離は大きくなっていく



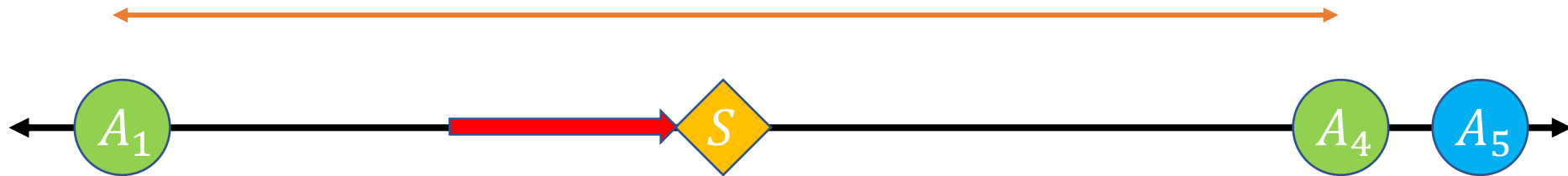
小課題 3 ($A_{i+1} - A_i \leq 100$)

- もう一度経過を見てみる
- A_l と A_r の距離は大きくなっていく



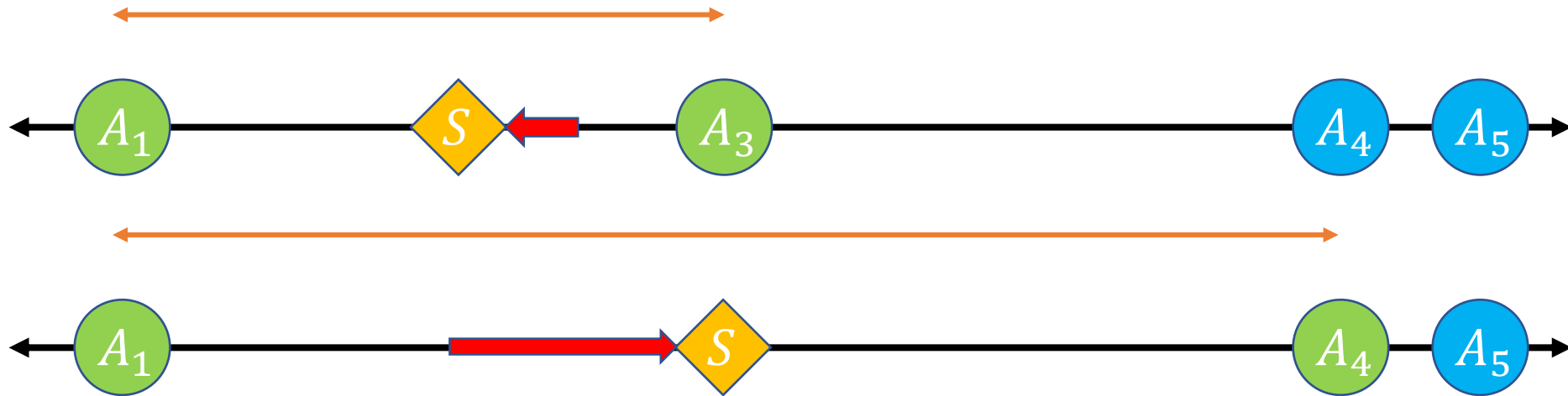
小課題 3 ($A_{i+1} - A_i \leq 100$)

- もう一度経過を見てみる
- A_l と A_r の距離は大きくなっていく
- どれくらい？



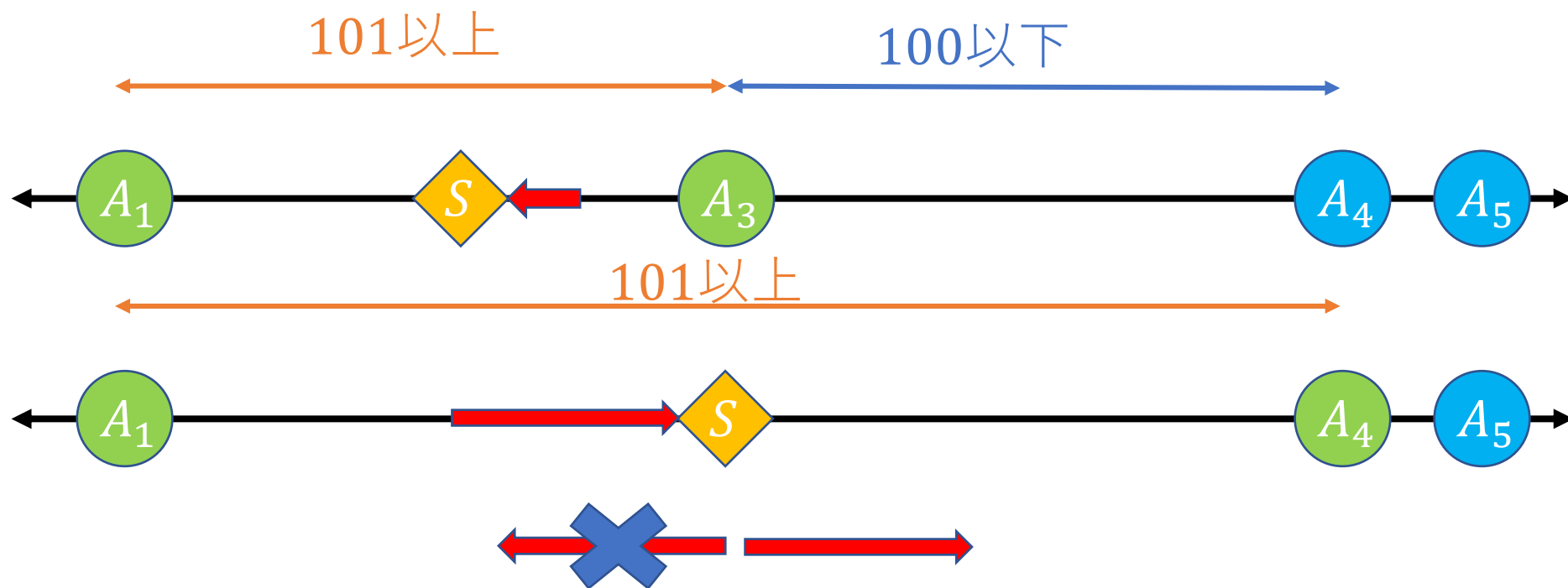
小課題 3 ($A_{i+1} - A_i \leq 100$)

- どれくらい A_l と A_r の距離が大きくなっていくか
- 一回で区間一つ分増える
- $A_i < A_{i+1}$ なので区間の長さは1以上→1以上増える



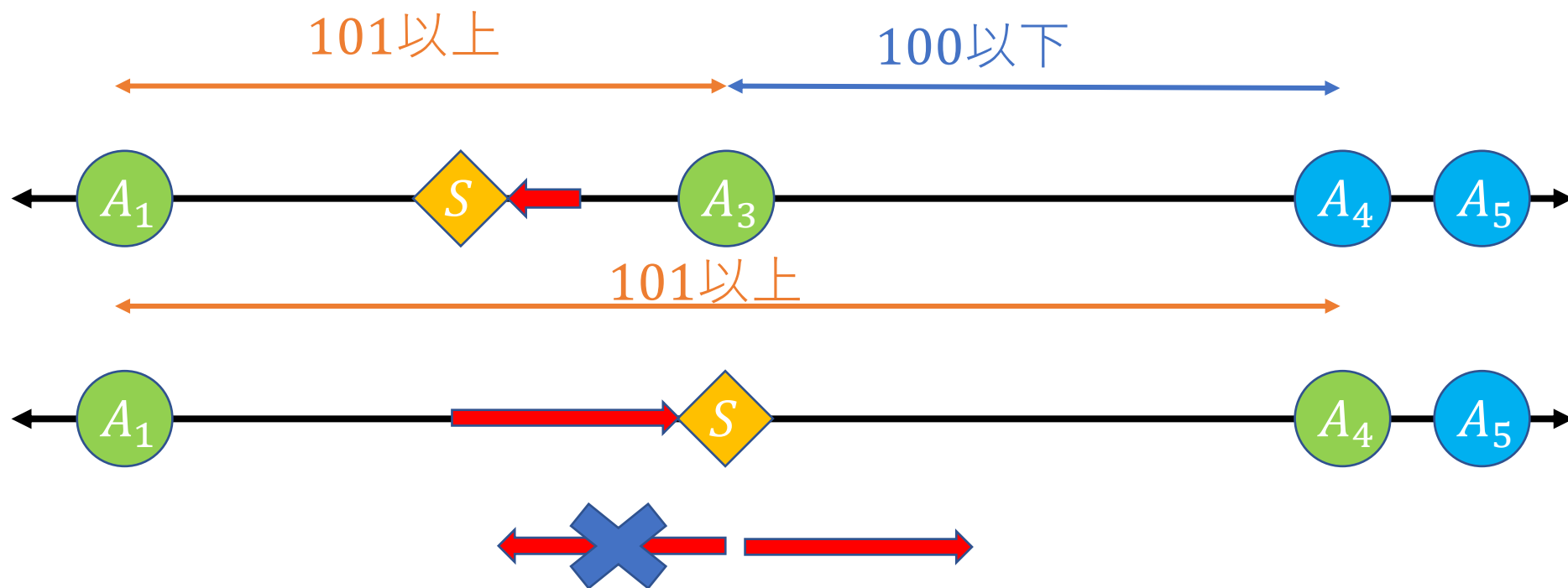
小課題 3 ($A_{i+1} - A_i \leq 100$)

- 100 回移動すると A_l と A_r の距離は101以上になる
- →それ以降は最後以外 A_l と A_r の間を飛び越えられない



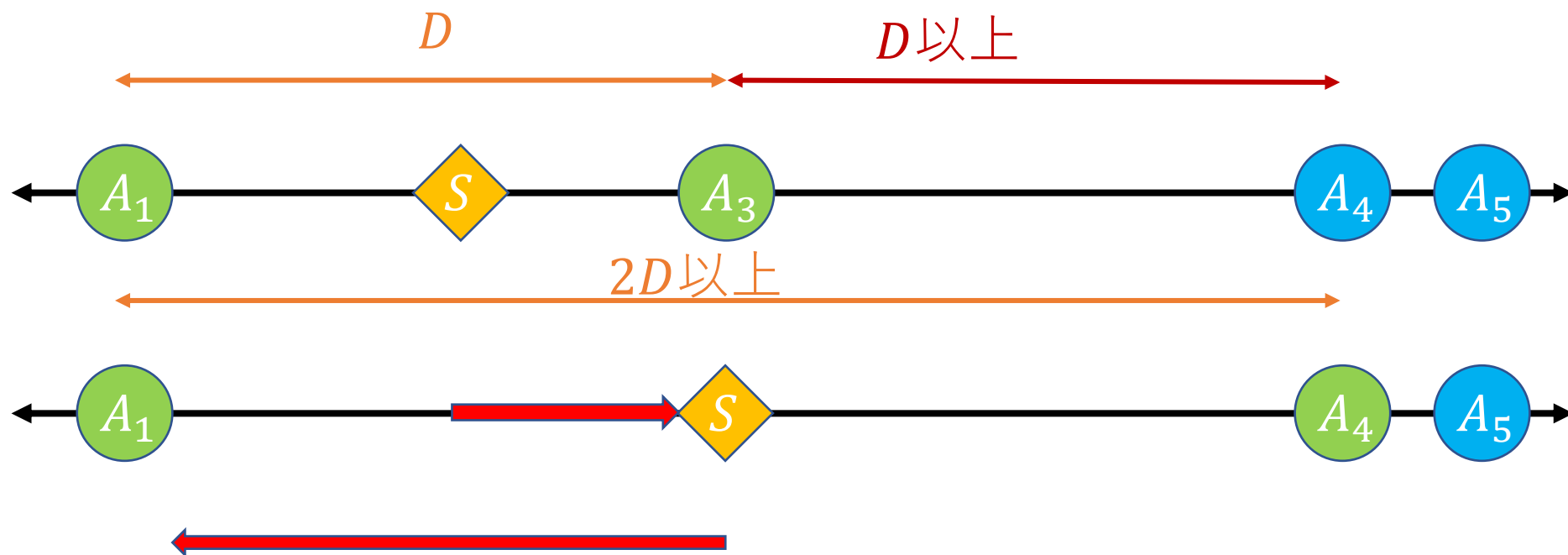
小課題 3 ($A_{i+1} - A_i \leq 100$)

- 100 回移動 $\rightarrow A_1$ か A_N まで同じ向きで移動 ($\rightarrow$ 逆の端まで移動)
- 計算回数は $100 \times Q$ くらい



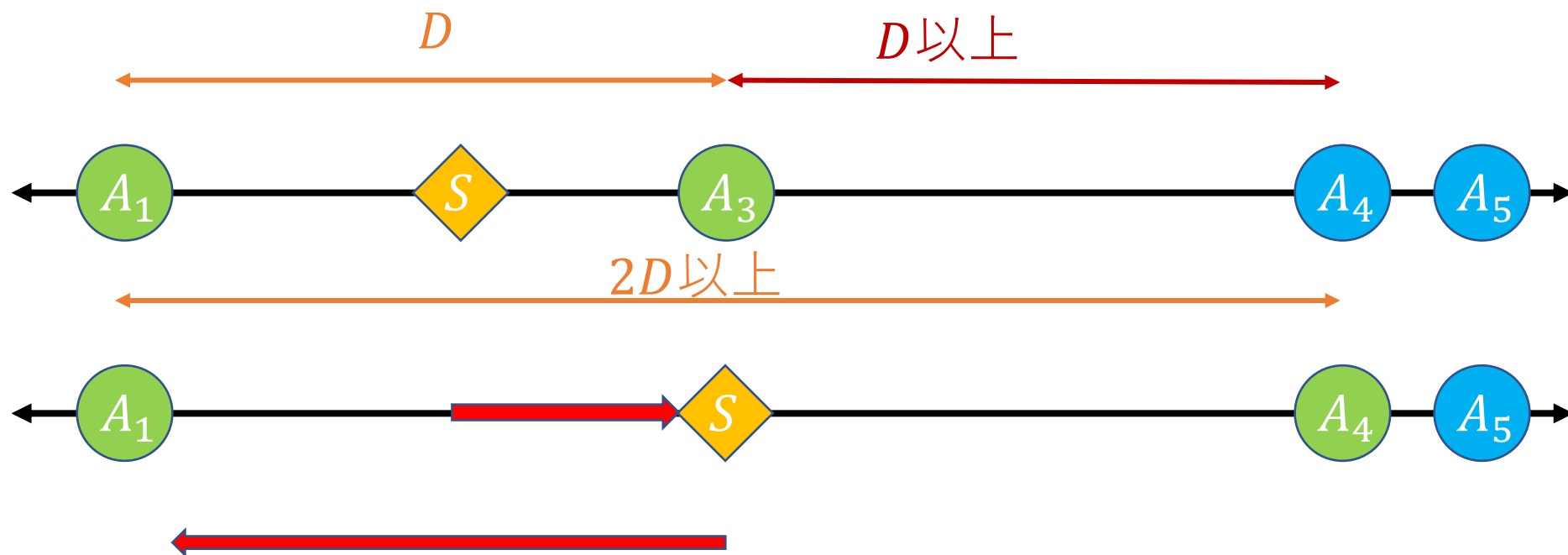
満点解法

- どれくらい A_l と A_r の距離が大きくなっていくか
- 進む向きが変わるときには距離 D が2倍以上になる



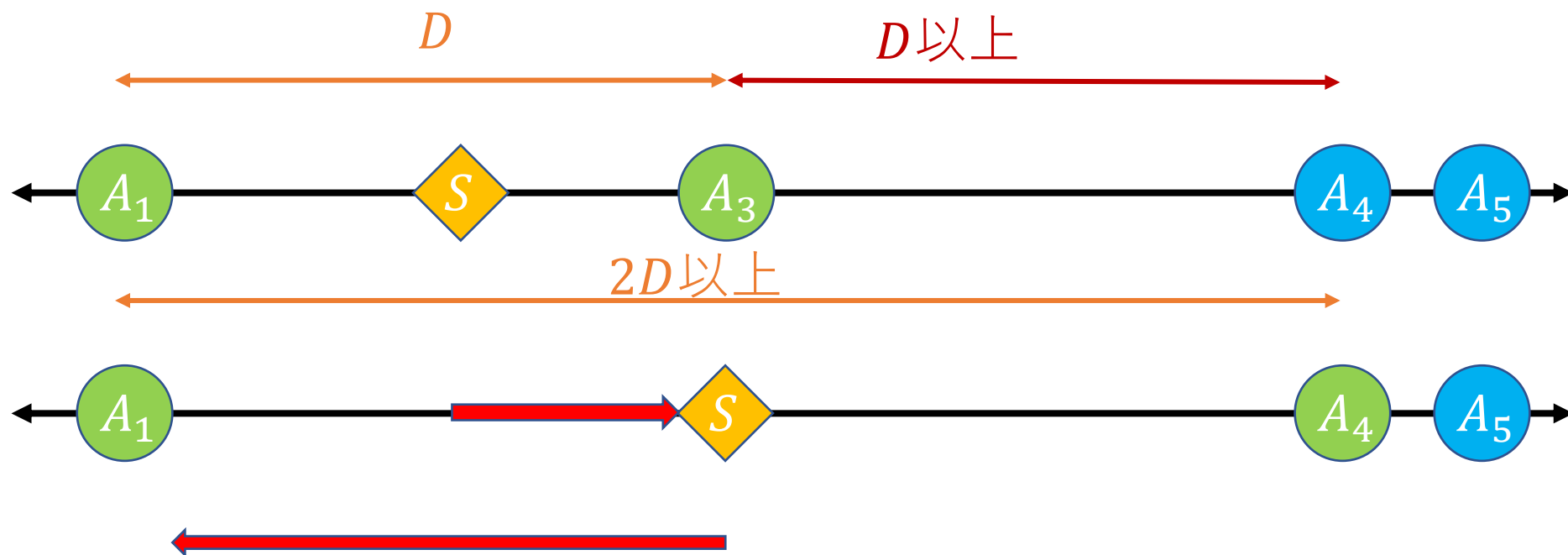
満点解法

- 距離 D が 2 倍以上にならないなら進む向きは変わらない
- 最初左右どちらに進むか調べ、その方向で距離 D が 2 倍以上になる初めての点を二分探索で求めて、そこまで同じ向きで進む



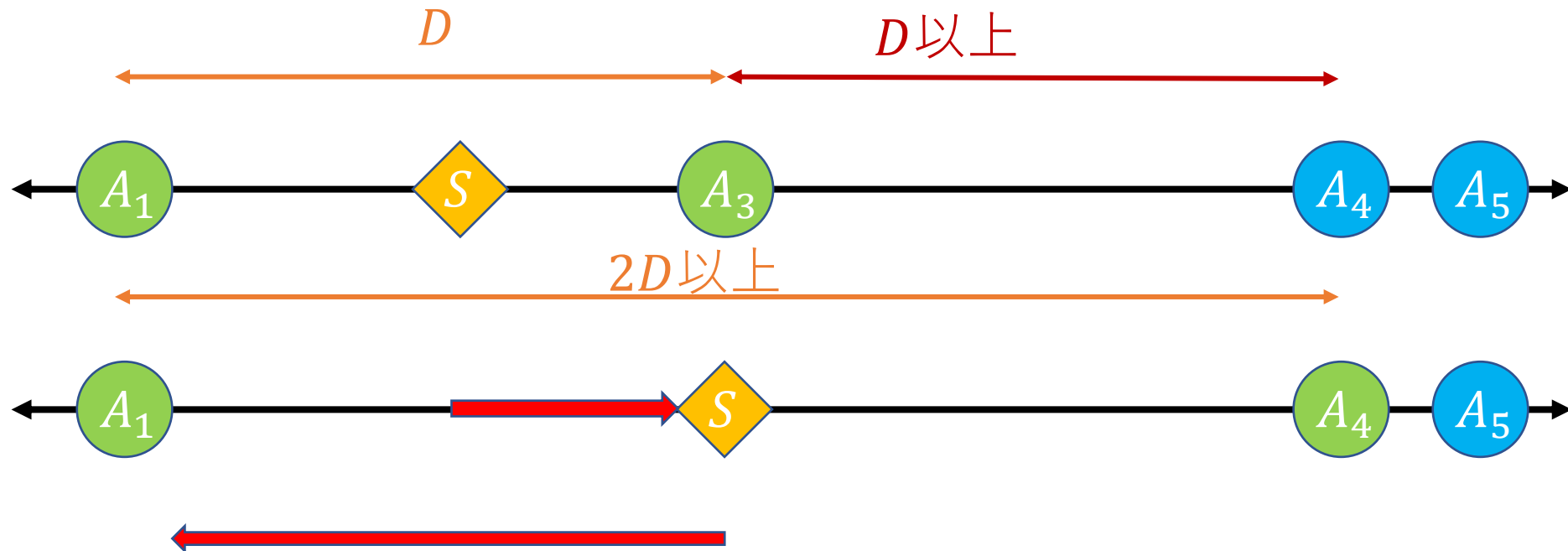
満点解法

- 距離 D は毎回 2 倍以上になるので、これを $O(\log \max A_i)$ 回繰り返せばよい
- 計算量は $O(N + Q \log N \log \max A_i)$

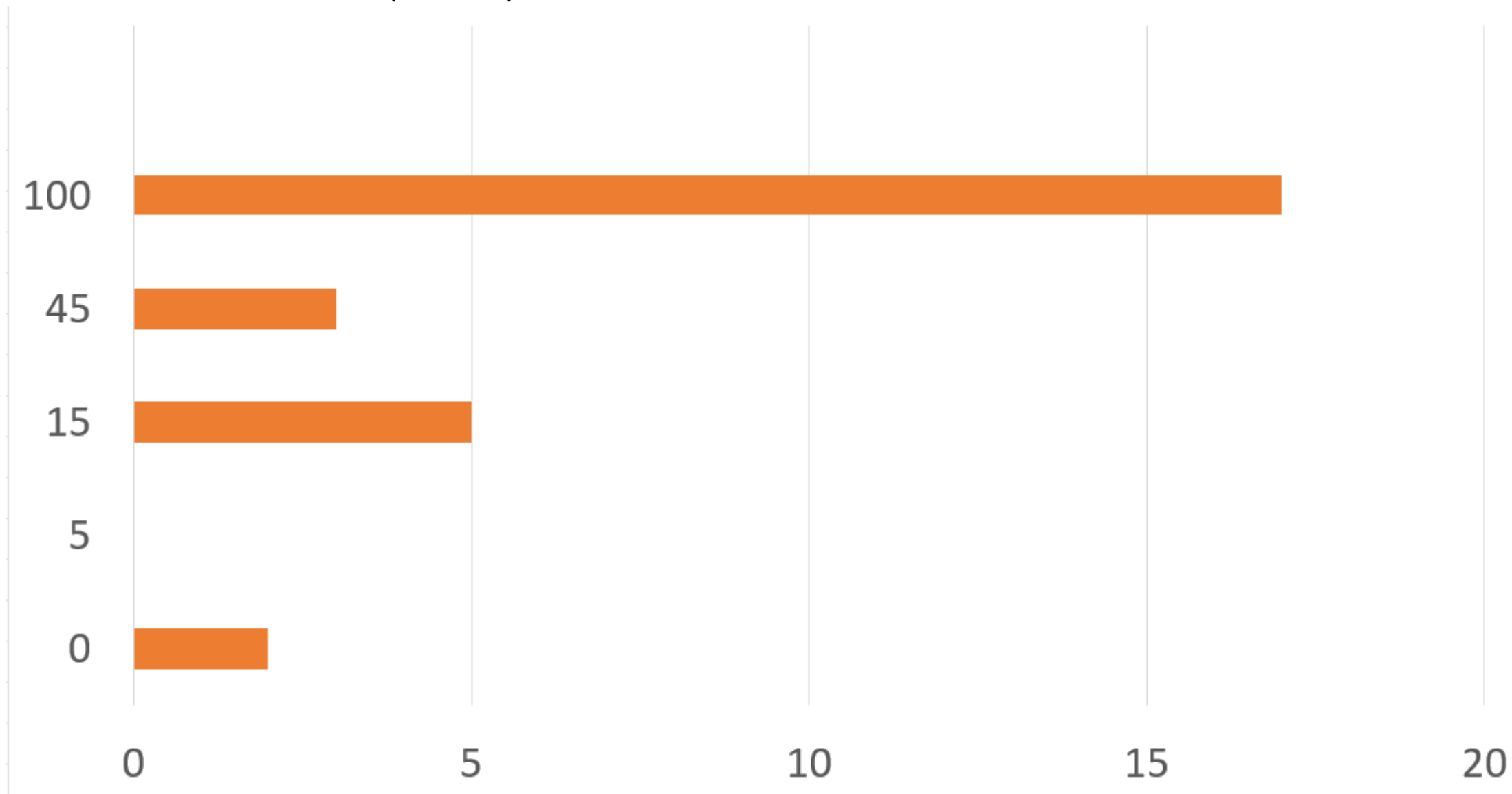


満点解法（別解）

- $A_{i+1} - A_i$ が距離 D 以上になる初めての場所をRMQ(Sparse Table)+二分探索で求めてもよい
- 計算量は $O(N + Q \log N \log \max A_i)$

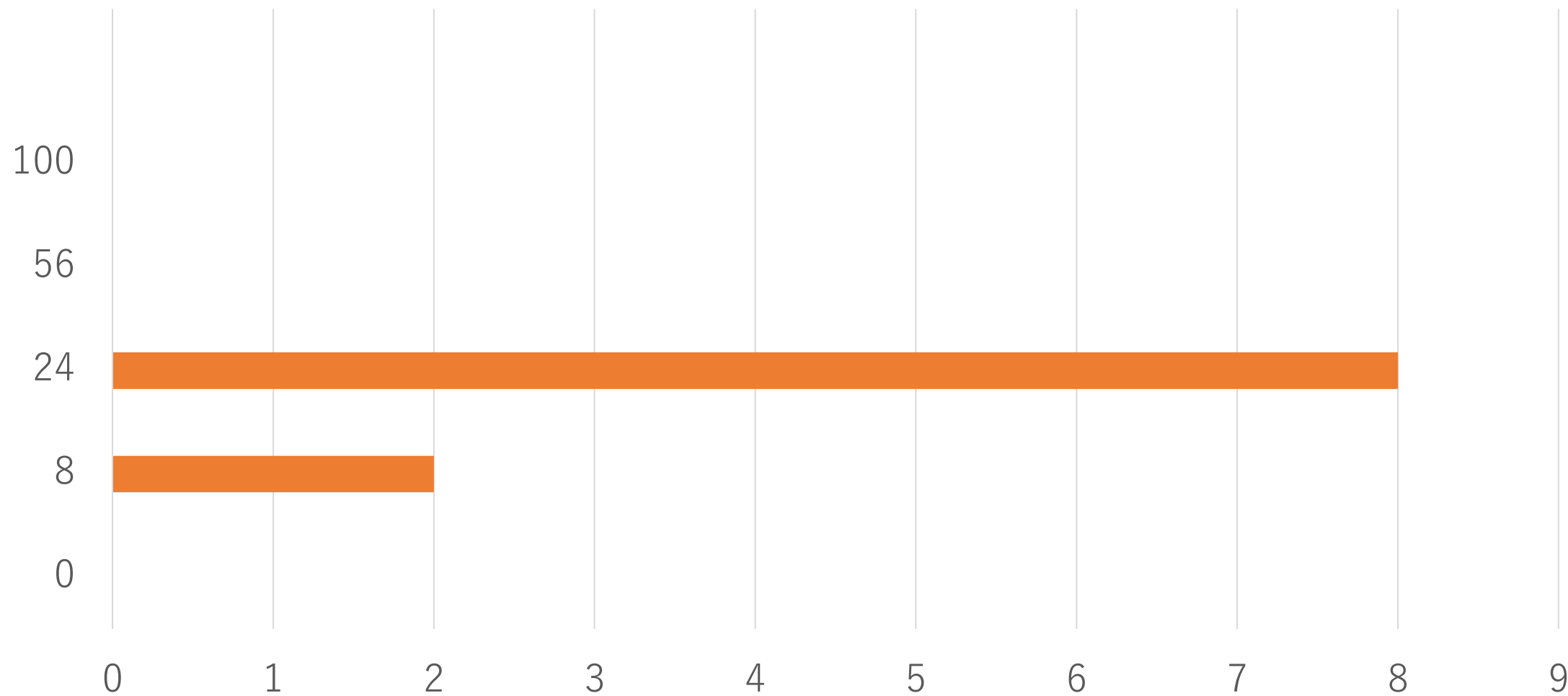


得点分布 (JOI)



得点分布 (JOIG)

得点分布





最後の戦い 解説

情報オリンピック春季トレーニング 4日目

2023 年 3 月 22 日

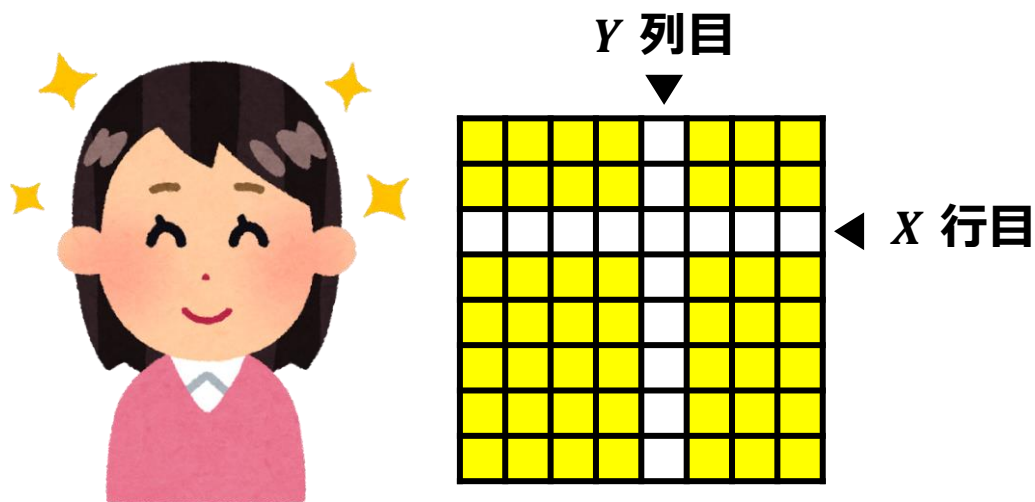
米田優峻 (@e869120)



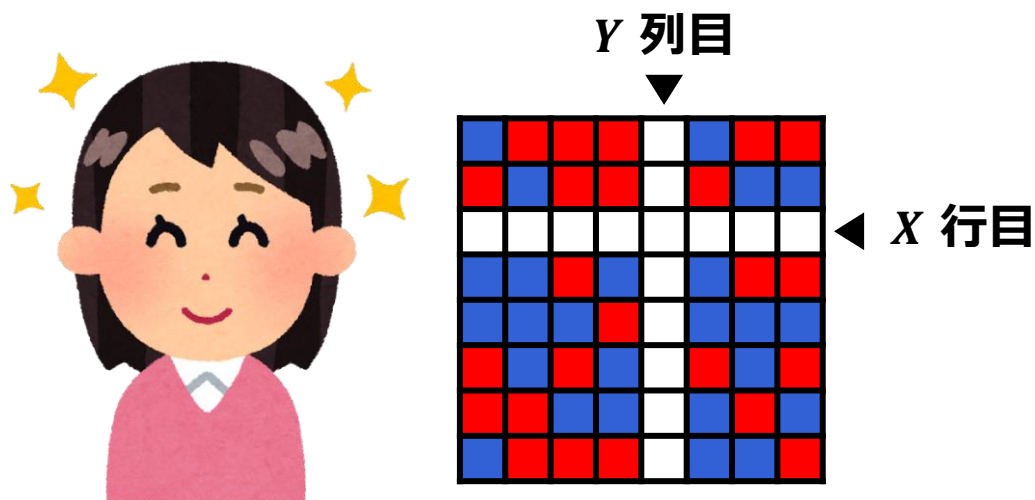
AAAABBBBBBAB...



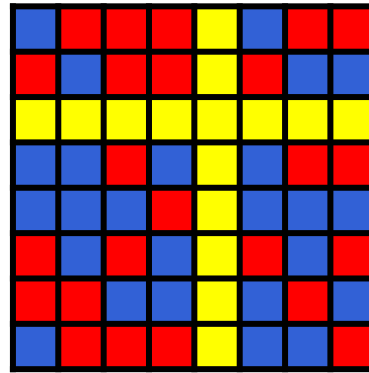
Anna は Bruno に 「長さ N の AB 文字列」 を伝えたいです
ただし、決められた方法で伝えなければなりません



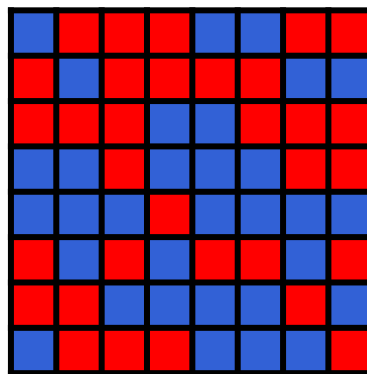
まず、Anna は 8×8 のマス目の X 行目・ Y 列目以外の
49 個のマスに、青か赤で色を塗ります



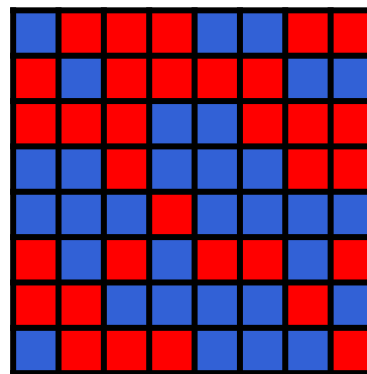
まず、Anna は 8×8 のマス目の X 行目・ Y 列目以外の
49 個のマスに、青か赤で色を塗ります



次に、ビ太郎が残りの 15 マスに青か赤で色を塗ります
(Anna にも Bruno にも見えない場所で塗ります)



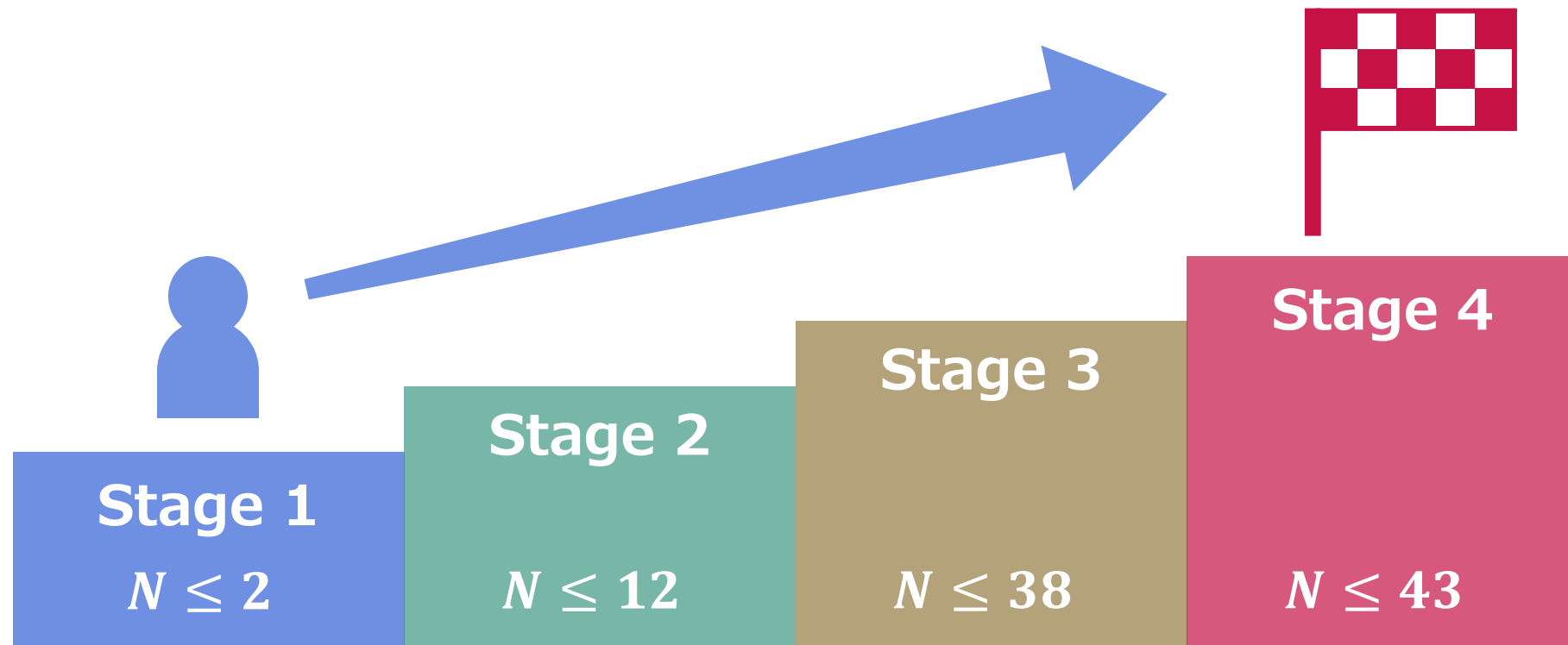
**次に、ビ太郎が残りの 15 マスに青か赤で色を塗ります
(Anna にも Bruno にも見えない場所で塗ります)**



最後に、Bruno がマス目を見ます (x, y の値は見えません)
長さ N の文字列を正しく当ててください

より長い文字列でも当てられた場合、高得点が得られます

あなたはどこまで行けますか？



Stage 1

アイデア：赤の個数で識別する

Anna

- 文字列が A ならば、全部青で塗る
- 文字列が B ならば、全部赤で塗る

Bruno

- 赤が 32 個以下のときに限り「A」と判定

1

$N = 1$ の解法 : 具体例

12 / 105

例

文字列 A を伝えたい場合

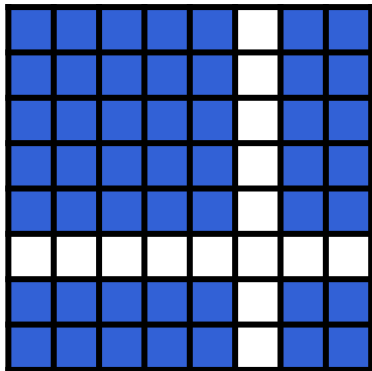
1

$N = 1$ の解法 : 具体例

13 / 105

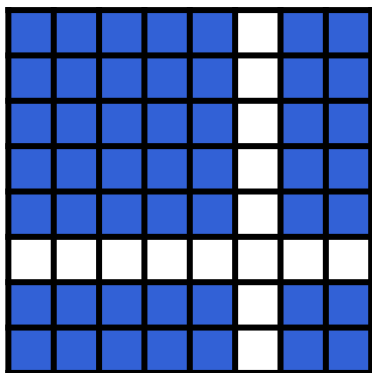
例

文字列 A を伝えたい場合

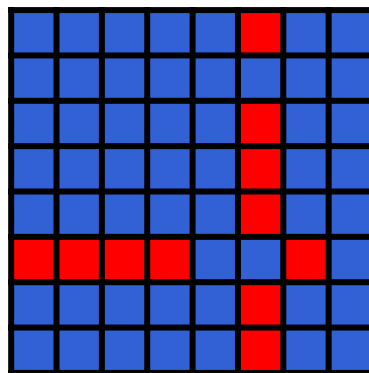


Annaは全部
青で塗る

例

文字列 A を伝えたい場合

Annaは全部
青で塗る



ビ太郎が残りの
マスを塗る

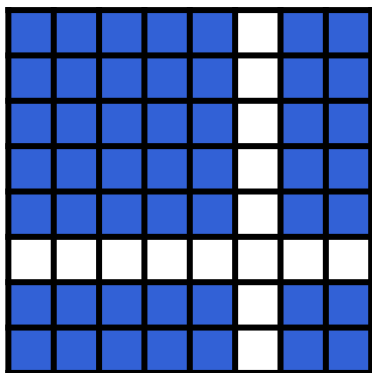
1

$N = 1$ の解法 : 具体例

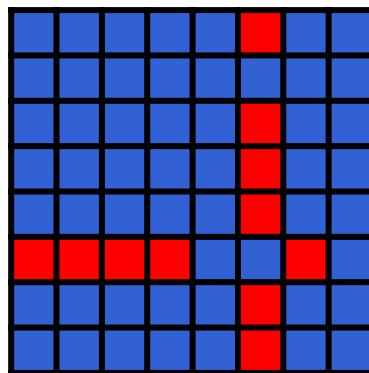
15 / 105

例

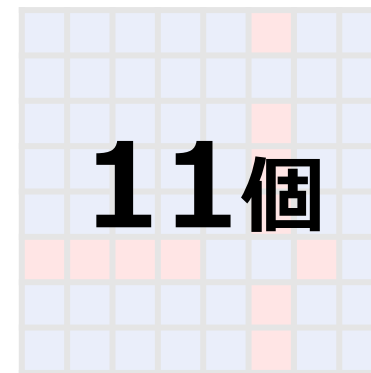
文字列 A を伝えたい場合



Annaは全部
青で塗る



ビ太郎が残りの
マス塗る



赤の個数 ≤ 32
Brunoは「A」と判定

1

$N = 1$ の解法：具体例

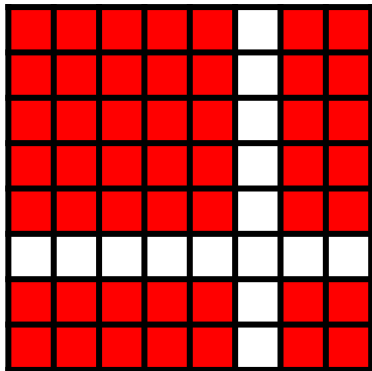
16 / 105

例

文字列 B を伝えたい場合

例

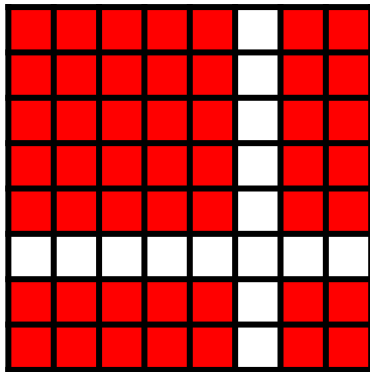
文字列 B を伝えたい場合



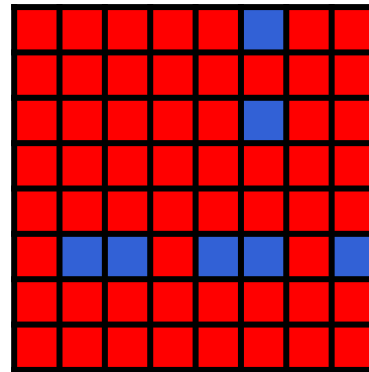
Annaは全部
赤で塗る

例

文字列 B を伝えたい場合



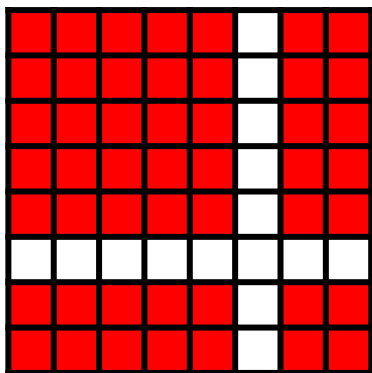
Annaは全部
赤で塗る



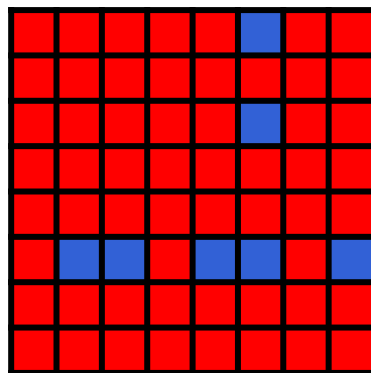
ビ太郎が残りの
マスを塗る

例

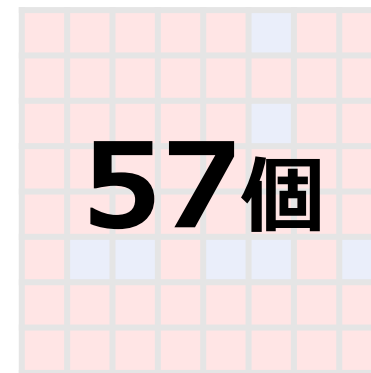
文字列 B を伝えたい場合



Annaは全部
赤で塗る

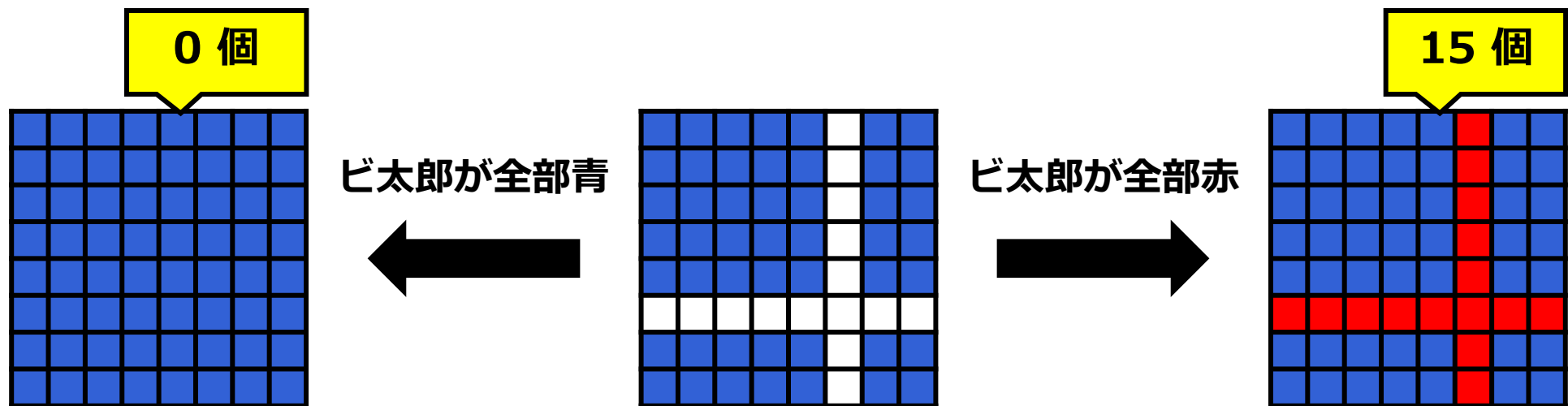


ビ太郎が残りの
マスを塗る



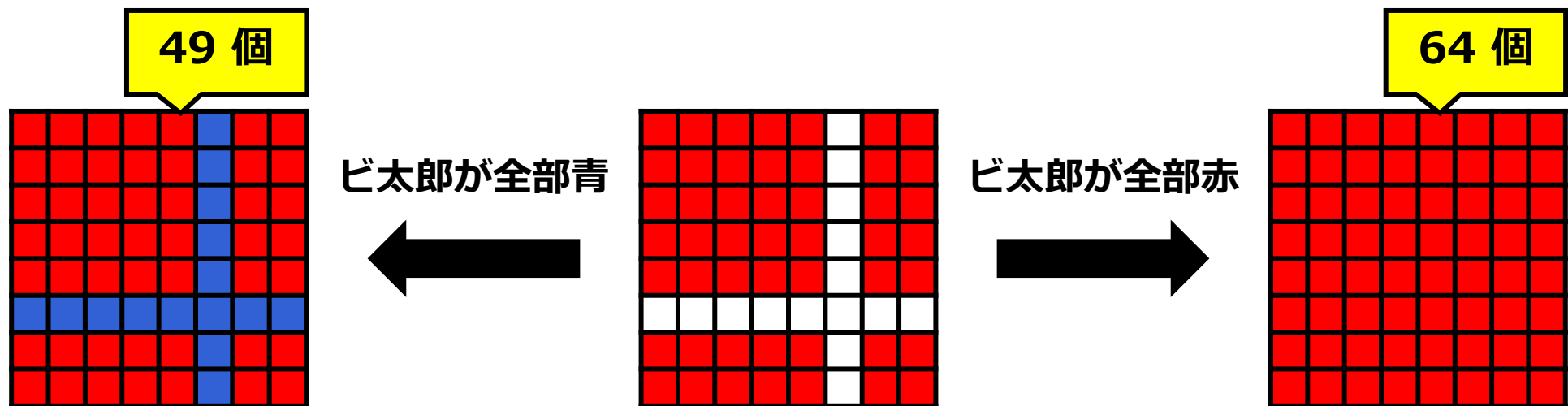
赤の個数 > 32
Brunoは「B」と判定

なぜこの方法が上手くいくか？



Anna が塗れるマスを全部青で塗った場合

赤の個数は 0~15 個の範囲になる (つまり絶対に 32 個以下)



Anna が塗れるマス全部赤で塗った場合

赤の個数は 49~64 個の範囲になる (つまり絶対に 32 個より多い)

この方法を少し工夫すると
 $N = 2$ でも解ける

Anna

以下の数だけ赤で塗る

- AA : 0個
- AB : 16個
- BA : 32個
- BB : 48個

Bruno

赤の個数で場合分け

- 0~15個 : AA
- 16~31個 : AB
- 32~47個 : BA
- 48~63個 : BB

※Anna は、自分が赤で塗るマスの個数を 0~49 個の範囲で調整できる

1

$N = 2$ の解法：具体例

25 / 105

例

文字列 AB を伝えたい場合

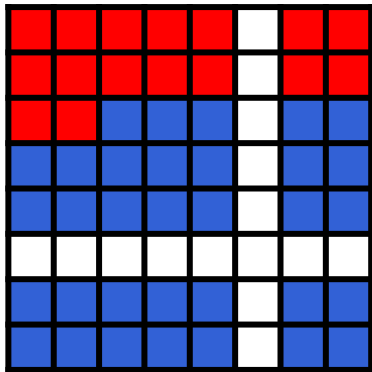
1

$N = 2$ の解法：具体例

26 / 105

例

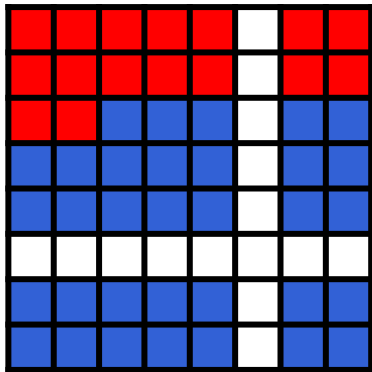
文字列 AB を伝えたい場合



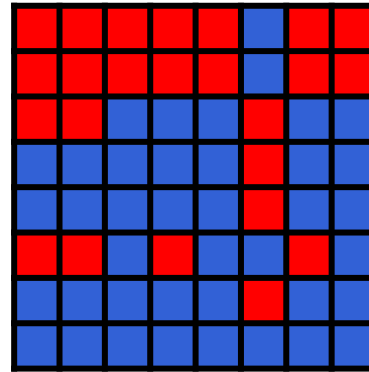
Annaは16マス
赤で塗る

例

文字列 AB を伝えたい場合



Annaは16マス
赤で塗る



ビ太郎が残りの
マスを塗る

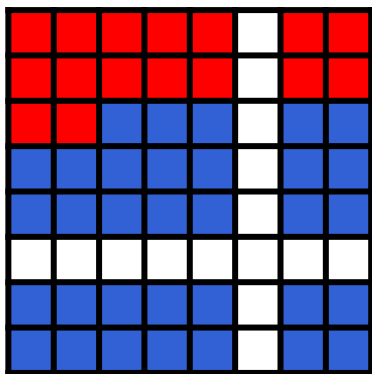
1

 $N = 2$ の解法 : 具体例

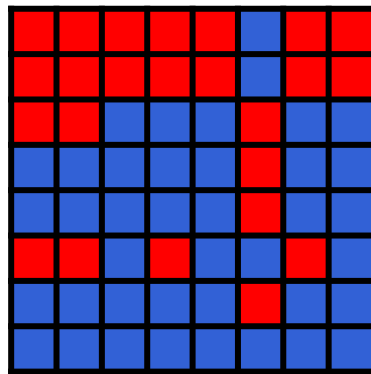
28 / 105

例

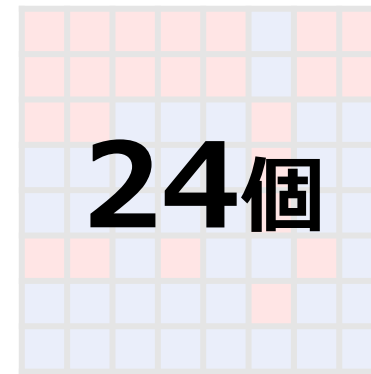
文字列 AB を伝えたい場合



Annaは16マス
赤で塗る



ビ太郎が残りの
マスを塗る



24個
 $16 \leq \text{赤の個数} \leq 31$
Brunoは「AB」と判定

1

$N = 2$ の解法：具体例

29 / 105

例

文字列 BB を伝えたい場合

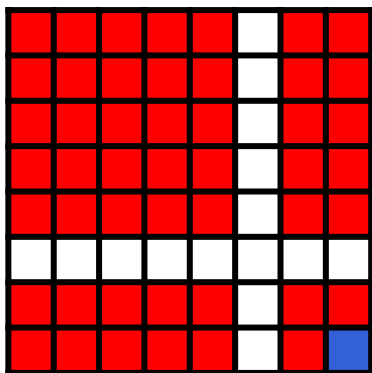
1

$N = 2$ の解法：具体例

30 / 105

例

文字列 BB を伝えたい場合



Annaは48マス
赤で塗る

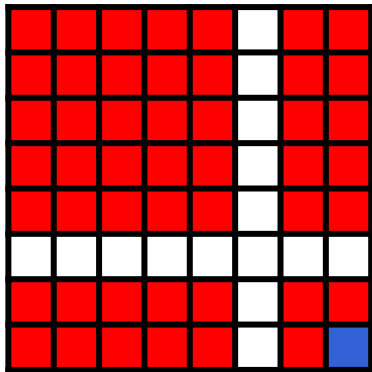
1

$N = 2$ の解法 : 具体例

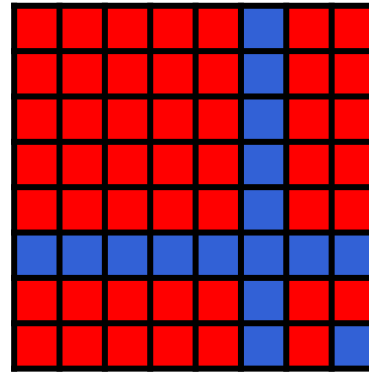
31 / 105

例

文字列 BB を伝えたい場合



Annaは48マス
赤で塗る



ビ太郎が残りの
マスを塗る

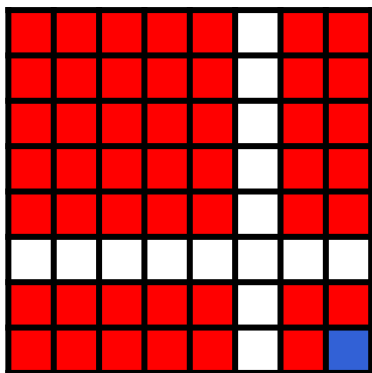
1

 $N = 2$ の解法：具体例

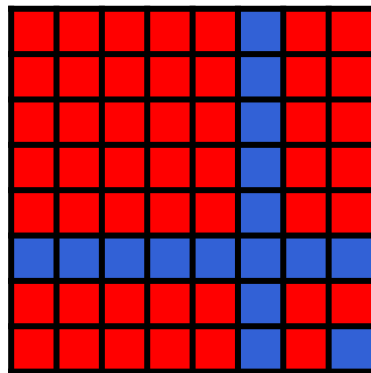
32 / 105

例

文字列 BB を伝えたい場合



Annaは48マス
赤で塗る



ビ太郎が残りの
マスを塗る



$48 \leq \text{赤の個数} \leq 63$
Brunoは「BB」と判定

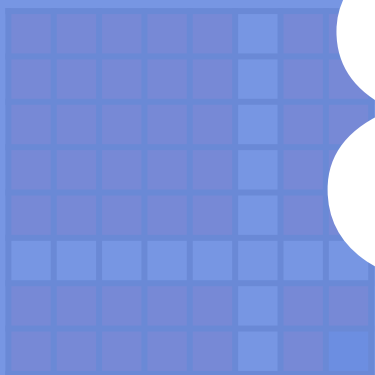
1

$N = 2$ の解法 : 具体例

33 / 105

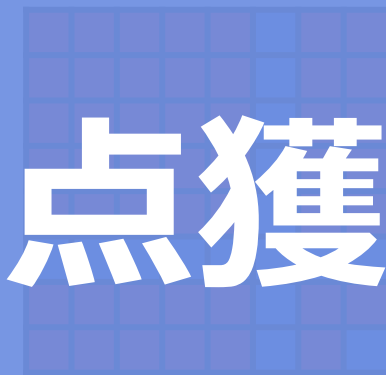
例

文字列 BB を伝えたい場合



Annaは48マス
赤で塗る

8



ビ太郎が残りの
マスを塗る

点獲得!

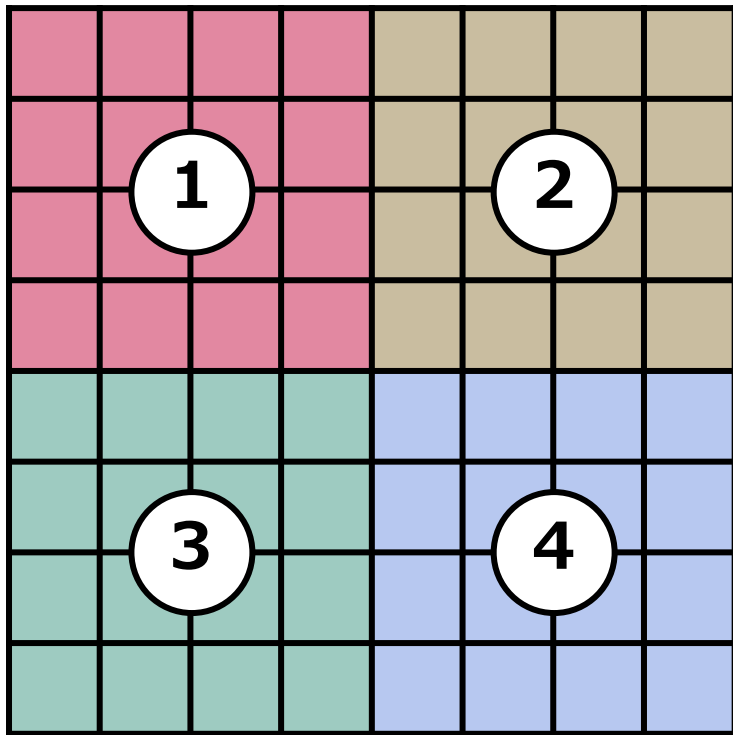
48個



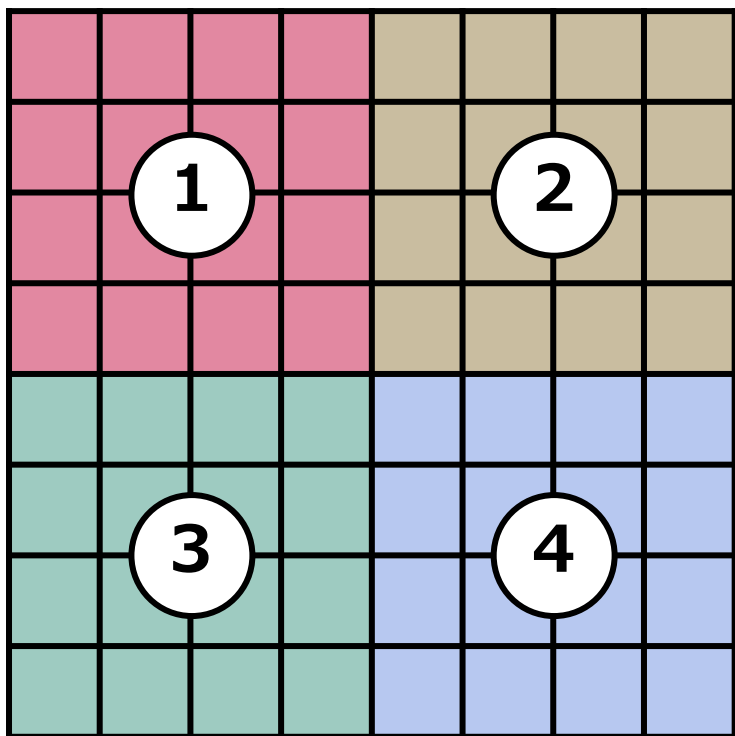
$48 \leq \text{赤の個数} \leq 63$
Brunoは「BB」と判定

Stage 2

アイデア：区画ごとに多数決



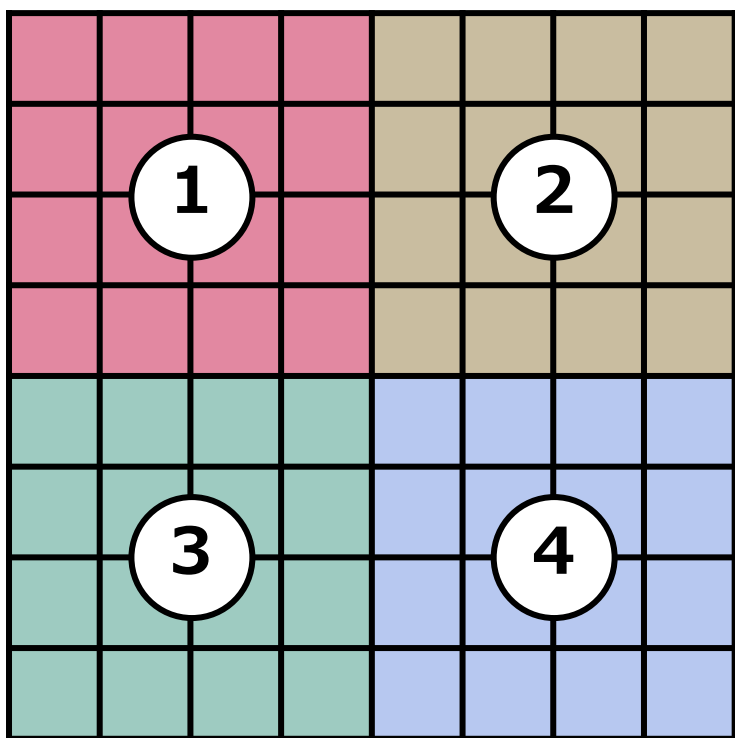
マス目を左図のように 4 分割する



マス目を左図のように 4 分割する



i 文字目が A のとき、Anna は区画 i の塗れる
マスを青で塗る (B のとき赤)



マス目を左図のように 4 分割する



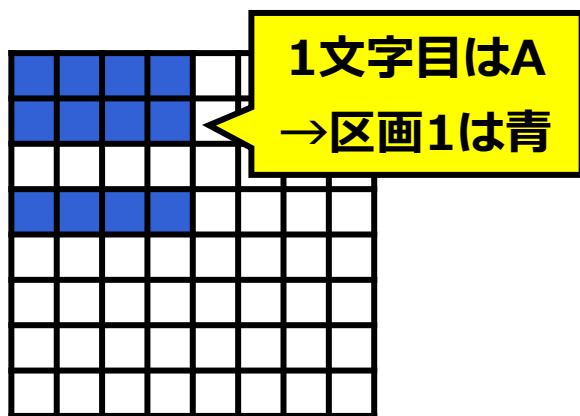
i 文字目が A のとき、Anna は区画 i の塗れる
マスを青で塗る (B のとき赤)



区画 i 内で青の方が赤より多い場合、Bruno は
 i 文字目を A だと判断する

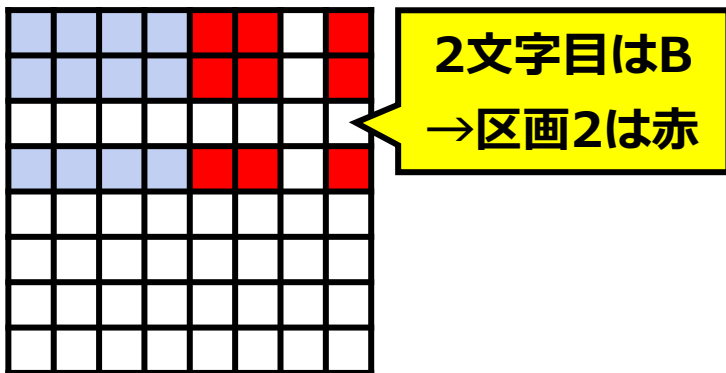
例**文字列 ABBA を伝えたい場合**

例

文字列 **ABBA** を伝えたい場合

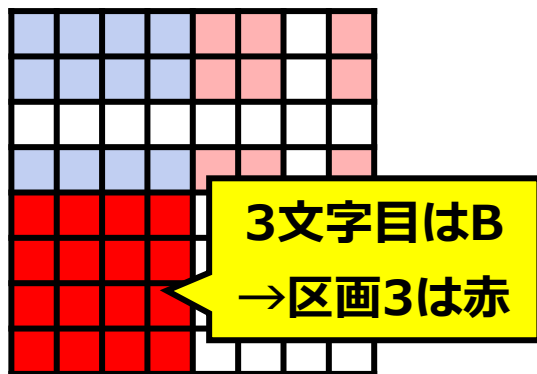
Annaは各区画に
色を塗る

例

文字列 **AB**BA を伝えたい場合

Annaは各区画に
色を塗る

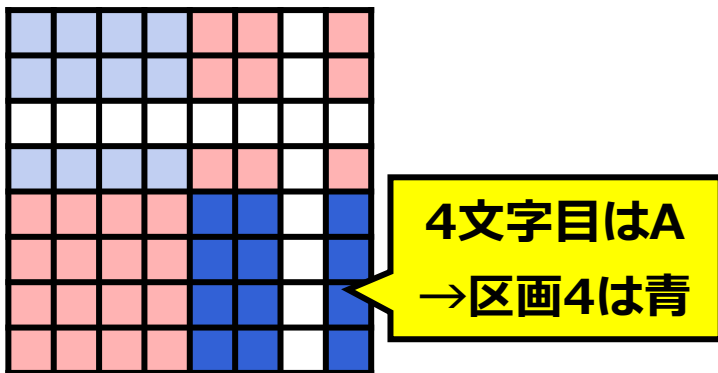
例

文字列 **ABBA** を伝えたい場合

Annaは各区画に
色を塗る

例

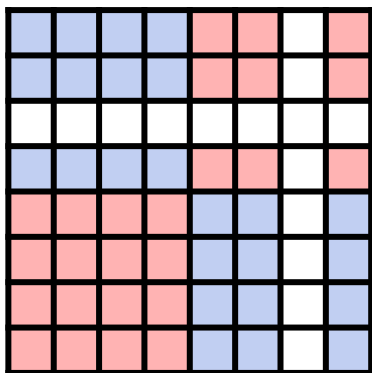
文字列 ABBA を伝えたい場合



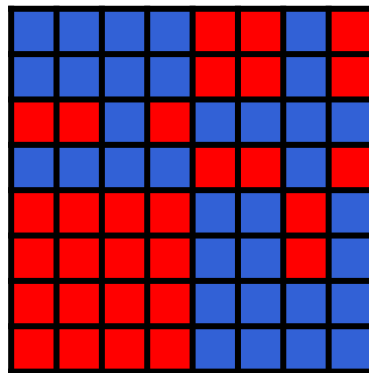
Annaは各区画に
色を塗る

例

文字列 ABBA を伝えたい場合



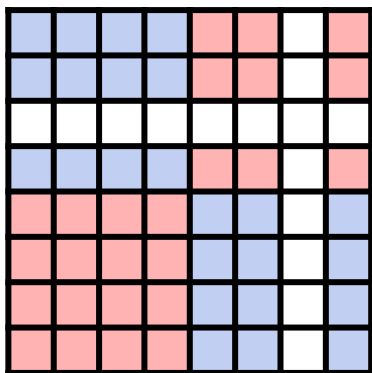
Annaは各区画に
色を塗る



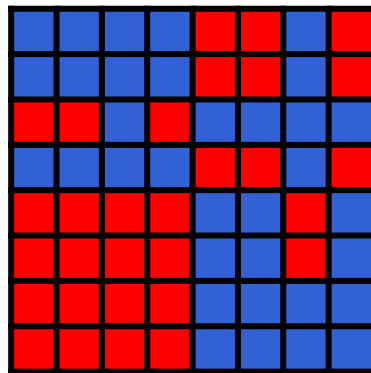
ビ太郎が残りの
マス塗る

例

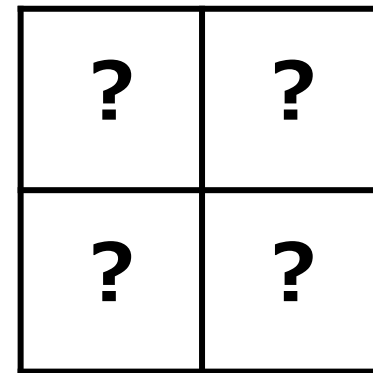
文字列 ABBA を伝えたい場合



Annaは各区画に
色を塗る



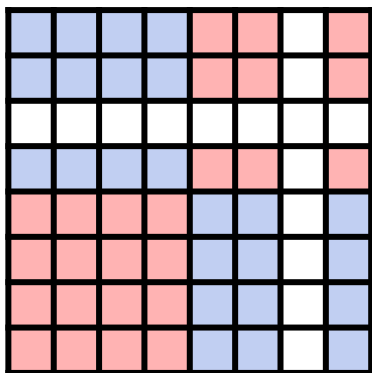
ビ太郎が残りの
マス进行塗る



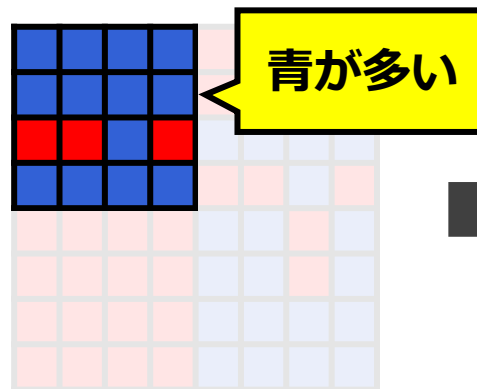
Brunoは区画ごとの
多数決を取る

例

文字列 ABBA を伝えたい場合



Annaは各区画に
色を塗る



ビ太郎が残りの
マスを塗る

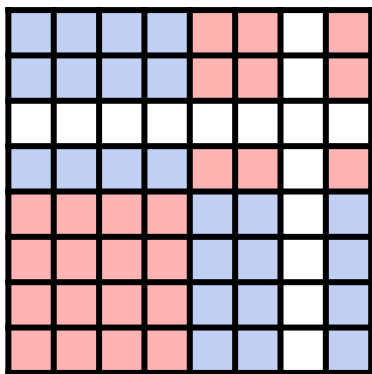


A	?
?	?

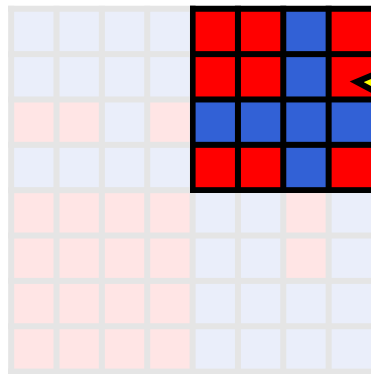
Brunoは区画ごとの
多数決を取る

例

文字列 ABBA を伝えたい場合



Annaは各区画に
色を塗る



ビ太郎が残りの
マス塗る

赤が多い

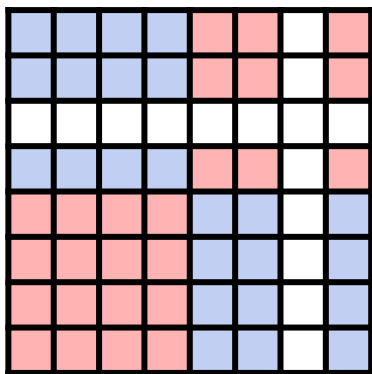


A	B
?	?

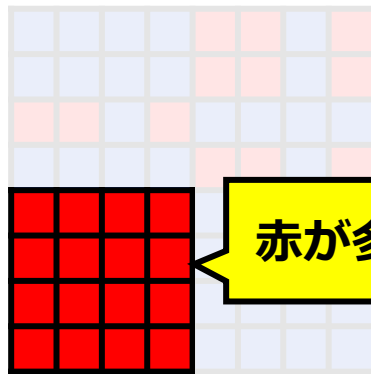
Brunoは区画ごとの
多数決を取る

例

文字列 ABBA を伝えたい場合



Annaは各区画に
色を塗る



ビ太郎が残りの
マス塗る

赤が多い

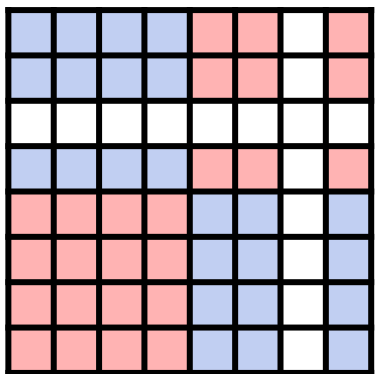


A	B
B	?

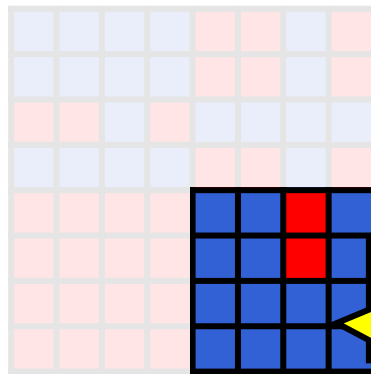
Brunoは区画ごとの
多数決を取る

例

文字列 ABBA を伝えたい場合



Annaは各区画に
色を塗る



ビ太郎が残りの
マスを塗る

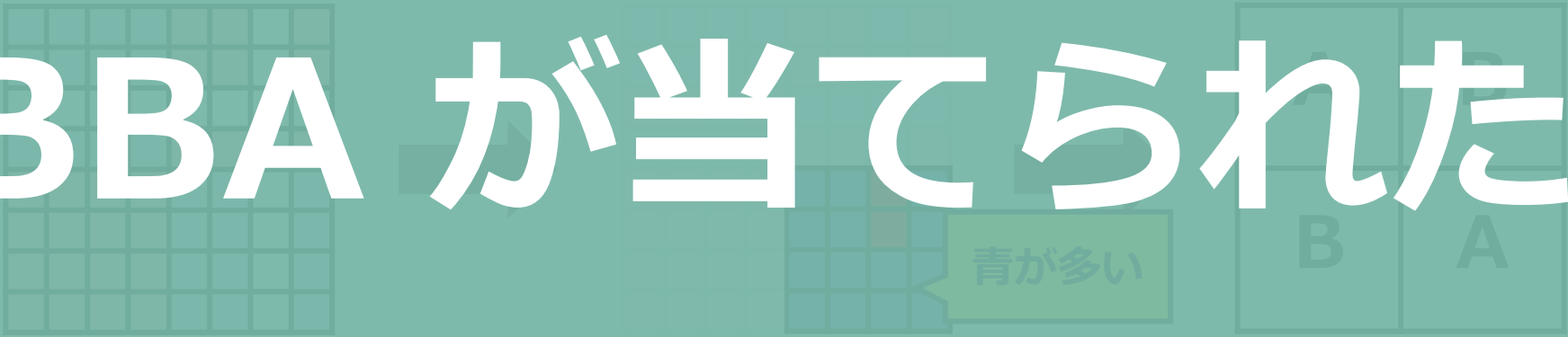


A	B
B	A

Brunoは区画ごとの
多数決を取る

例

文字列 ABBA を伝えたい場合



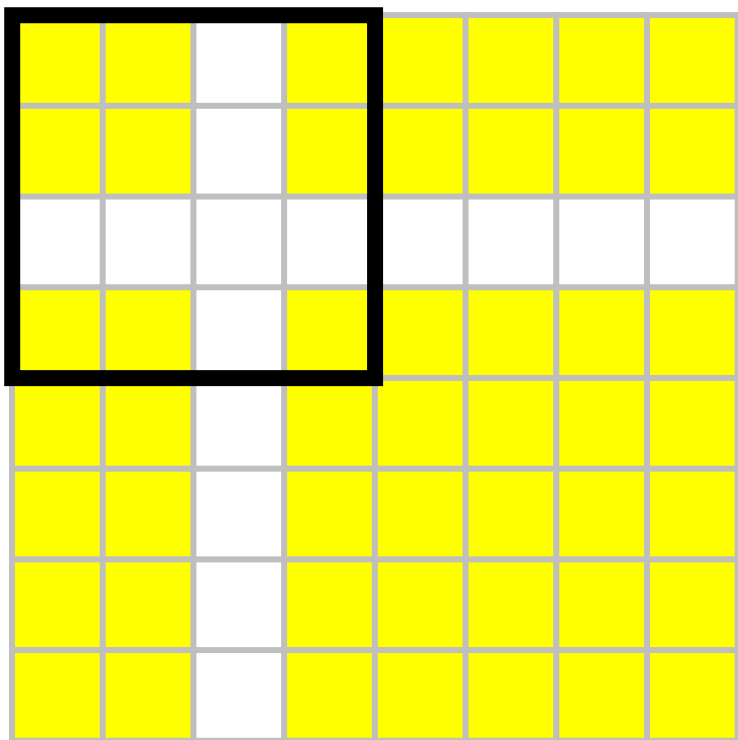
The first grid shows a 4x4 grid with the first two columns filled with blue squares. The second grid shows the same 4x4 grid with the first two columns filled with blue squares and the last two columns filled with green squares. A green speech bubble with the text '青が多い' (More blue) points to the blue squares. The third grid shows a 4x4 grid with the first two columns filled with blue squares and the last two columns filled with green squares. The letters 'B' and 'A' are written in the first and second rows of the last two columns, respectively.

ABBA が当てられた！

Annaは各区画に
色を塗る

ビ太郎が残りの
マスを塗る

Brunoは区画ごとの
多数決を取る



どんな場合でも、Anna が塗れる黄色マスは
各区画に (16 個中) 9 個以上存在する



Anna が過半数をコントロールできる !

この方法を少し工夫すると
 $N = 8$ でも解ける

1	1	1	1	2	2	2	2
1	1	1	1	2	2	2	2
1	1	1	1	2	2	2	2
1	1	1	1	2	2	2	2
3	3	3	3	4	4	4	4
3	3	3	3	4	4	4	4
3	3	3	3	4	4	4	4
3	3	3	3	4	4	4	4

これまでは、区画が正方形だったが...

1	8	7	6	5	4	3	2
2	1	8	7	6	5	4	3
3	2	1	8	7	6	5	4
4	3	2	1	8	7	6	5
5	4	3	2	1	8	7	6
6	5	4	3	2	1	8	7
7	6	5	4	3	2	1	8
8	7	6	5	4	3	2	1

これまでは、区画が正方形だったが…
斜めに分けるとどうなるか？



どの区画も (8 個中) 6 個以上 Anna が塗れる
過半数獲得！

この方法をさらに工夫すると
 $N = 12$ でも解ける

1	12	11	9	8	6	5	4
2	1	12	11	9	8	7	5
4	2	1	12	11	10	8	7
5	4	3	1	-	11	10	8
7	6	4	3	1	-	11	10
9	7	6	4	3	2	-	12
10	9	7	6	5	3	2	-
12	10	9	8	6	5	3	2

左図のようになると...

3/5 以上のマスに Anna が塗れるような
12 個の区画に分割できる！

1	12	11	9	8	6	5	4
2	1	12	11	9	8	7	5
4	2	1	12	11	8	7	
5	4	3	1	-	10		
7	6	4	3	-	11	10	
9	7	6	4	3	2	-	12
10	9	7	6	5	3	2	-
12	10	9	8	6	5	3	2

24

左図のようになると...

3/5 以上のマスに Anna が塗れるような
12 個の区画に分割できる！

点獲得！

Stage 3

アイデア : X, Y を読めるようにする

もし Bruno が x, y の値を見ることができたら
Anna が塗った場所もわかる



かなりの文字数を読み取れそう

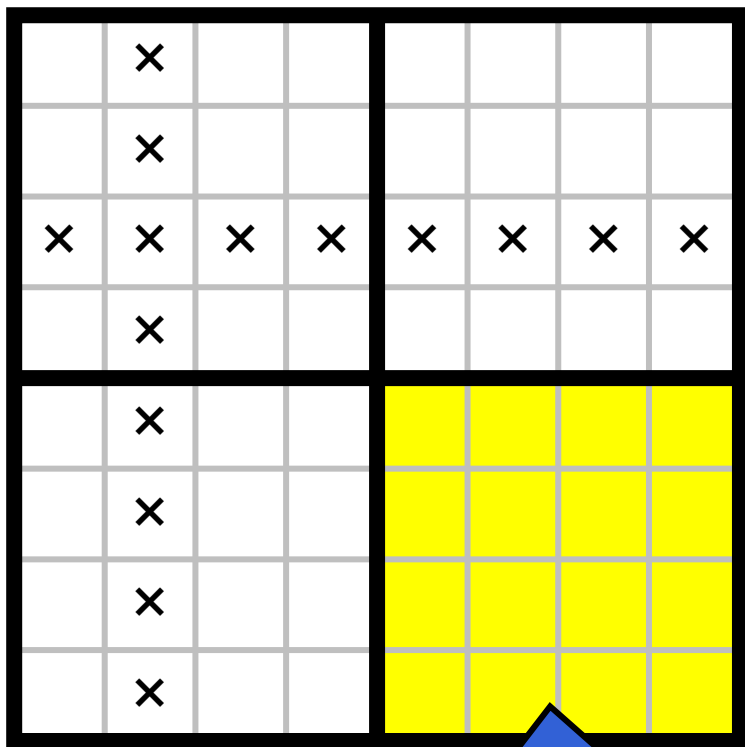
でも、Bruno は直接 X, Y の値を
見ることができない

一体どうすれば、 X, Y の値を
読み取ることができるのか？

3

 $N = 36$ の解法 (1/3)

61 / 105



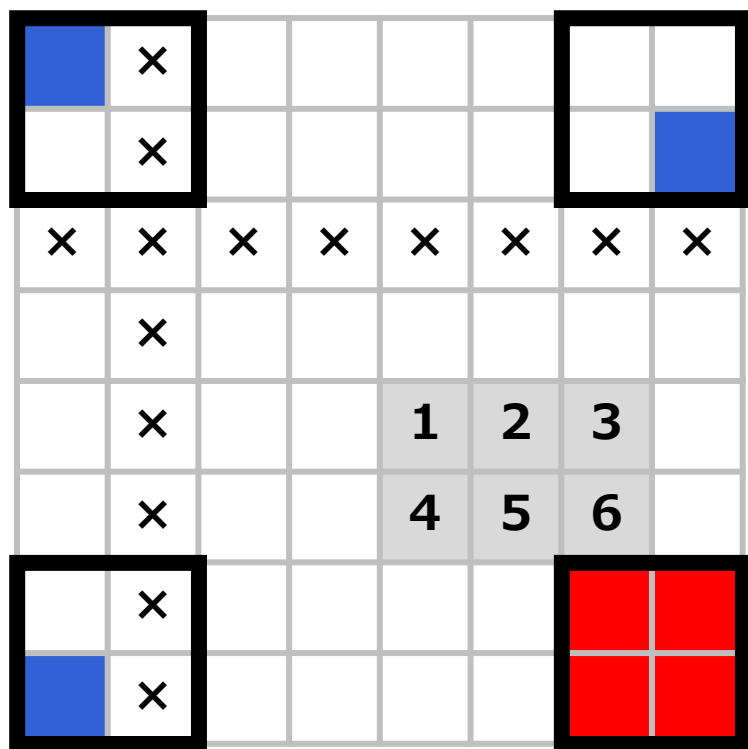
全マス置ける

まず、 4×4 の区画の中のどれか 1 つは
全マスに Anna が置くことができる

※Anna が塗れないマス を × 印で示している

	×						
	×						
×	×	×	×	×	×	×	×
	×						
	×			1	2	3	
	×			4	5	6	
	×						
	×						

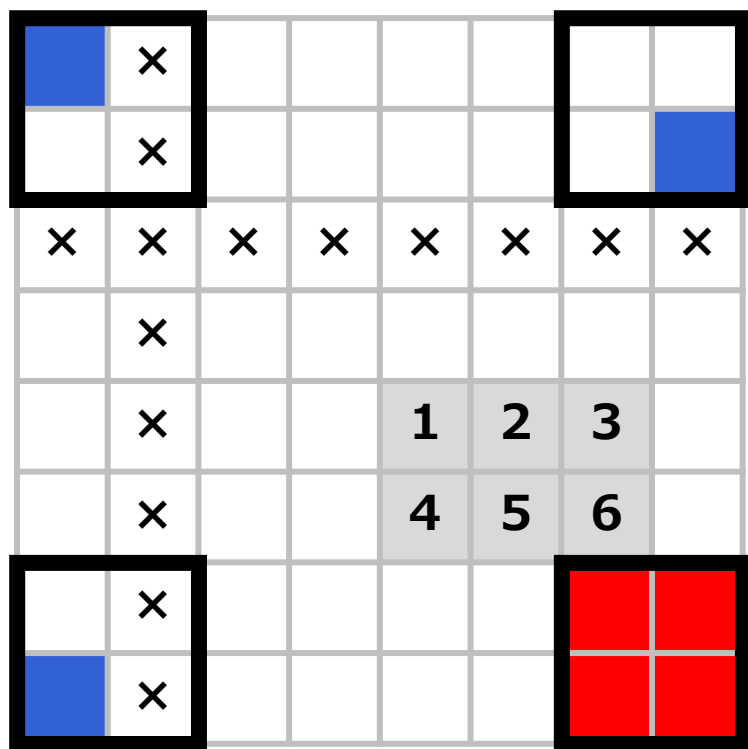
そこで、全マス置ける区画の適当な場所に
 X, Y の情報を記録することを考える
 (64 通りあるので $\log_2 64 = 6$ マス必要)



さらに目印として、全マス置ける区画の 2×2 隅を赤で塗ることを考える

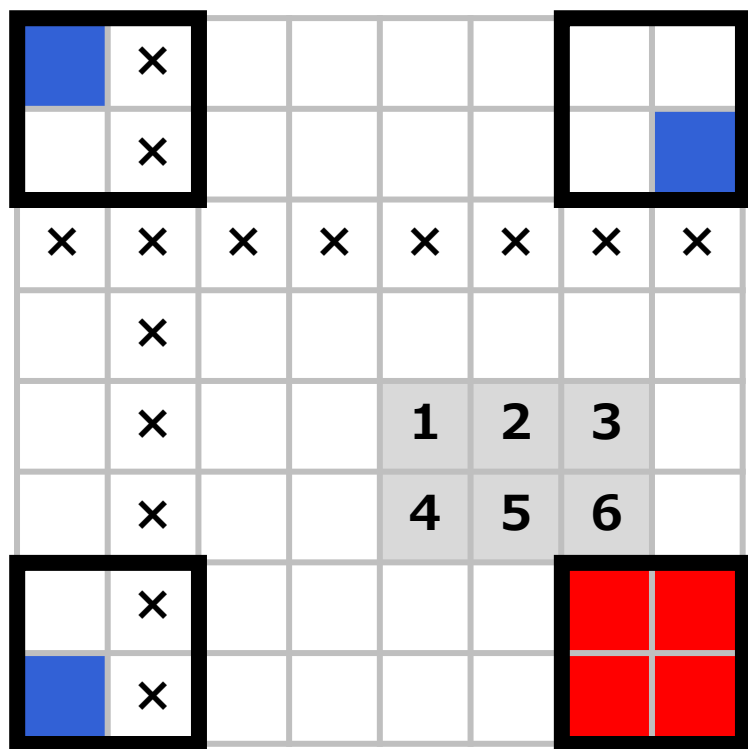
(他の区画の 2×2 隅はどれか 1 つを青で塗る)

このとき、Bruno は
 X, Y の値を読み取れるのか？



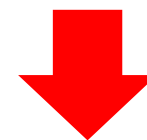
2×2 の隅のうち、全部赤になっているものは
必ず 1 つだけ存在する

(しかもこの周辺に x, y が書き込まれている)



2×2 の隅のうち、全部赤になっているものは
必ず 1 つだけ存在する

(しかもこの周辺に x, y が書き込まれている)



Bruno は隅を見れば良い !

X, Y を読み取った後は？

Blue	X	Yellow	Yellow	Yellow	Yellow	Yellow	Yellow
Yellow	X	Yellow	Yellow	Yellow	Yellow	Yellow	Blue
X	X	X	X	X	X	X	X
Yellow	X	Yellow	Yellow	Yellow	Yellow	Yellow	Yellow
Yellow	X	Yellow	Yellow	Gray	Gray	Gray	Yellow
Yellow	X	Yellow	Yellow	Gray	Gray	Gray	Yellow
Yellow	X	Yellow	Yellow	Yellow	Yellow	Red	Red
Blue	X	Yellow	Yellow	Yellow	Yellow	Red	Red

Anna が塗れるマスの中で
現時点で使われていないものは 36 個

	×	1	2	3	4	5	6
7	×	8	9	10	11	12	
×	×	×	×	×	×	×	×
13	×	14	15	16	17	18	19
20	×	21	22				23
24	×	25	26				27
28	×	29	30	31	32		
	×	33	34	35	36		

Anna が塗れるマスの中で
現時点で使われていないものは 36 個



左上から順に 1, 2, ..., 36 文字目を記録すれば、 $N = 36$ で解ける

具体例を考えてみよう

例 : AAAAAAAAAAAAAAAAAAABBBBBBBBBBBBBBBBBBBB を伝えたい

(ただし $X = 4, Y = 7$)

3 $N = 36$ の解法 : 具体例 (Anna)

71 / 105

							×
							×
							×
							×
×	×	×	×	×	×	×	×
							×
							×
							×

まず、左上の 4×4 隅が完全に空いている

3 $N = 36$ の解法：具体例 (Anna)

72 / 105

							×
							×
	X	X	X				×
	Y	Y	Y				×
×	×	×	×	×	×	×	×
							×
							×
							×

まず、左上の 4×4 隅が完全に空いている



この中の適切な場所に
 $X=4$ (赤青青)、 $Y=7$ (赤赤赤) を記録

※赤青青は 2 進法の 100 [=4]、赤赤赤は 2 進法の 111 [=7] に対応。

3 $N = 36$ の解法：具体例 (Anna)

73 / 105

★	★					★	×
★	★						×
							×
							×
×	×	×	×	×	×	×	×
							×
							×
★						★	×

まず、左上の 4×4 隅が完全に空いている



この中の適切な場所に
 $X=4$ (赤青青)、 $Y=7$ (赤赤赤) を記録

次に、左上の 2×2 隅を全部赤で塗り

それ以外の隅を 1 マス青で塗る

3 $N = 36$ の解法：具体例 (Anna)

74 / 105

		1	2	3	4		×
		5	6	7	8	9	×
10				11	12	13	×
14				15	16	17	×
×	×	×	×	×	×	×	×
18	19	20	21	22	23	24	×
25	26	27	28	29	30	31	×
	32	33	34	35	36		×

その後、残った塗れるマスに
左上から順に「文字に対応する色」で塗る
(A は青、B は赤)

※AA…ABB…B なので、18 文字目までが青、19 文字目からが赤

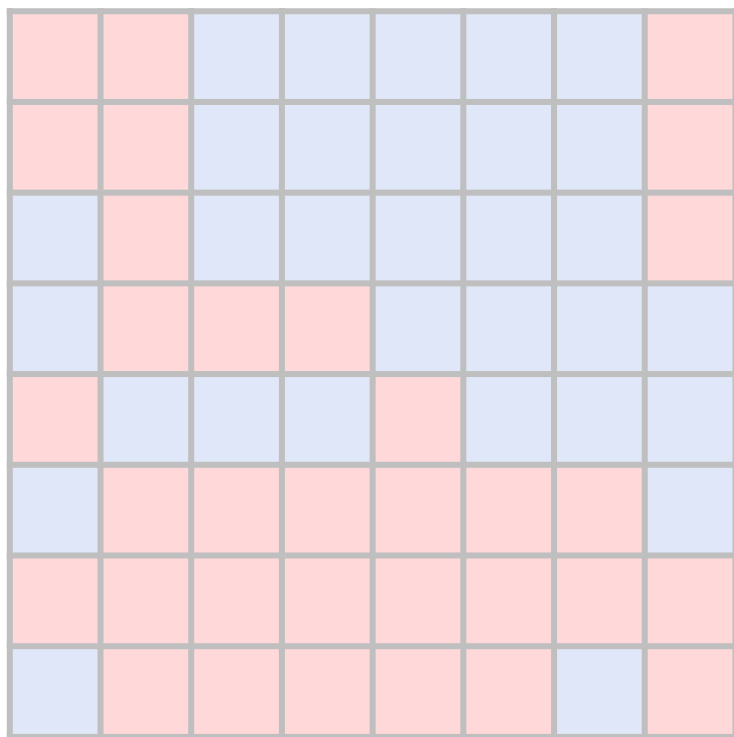
3 $N = 36$ の解法 : 具体例 (Anna)

75 / 105

							×
							×
							×
							×
×	×	×	×	×	×	×	×
							×
							×
							×

**このとき、Bruno は文字列を
復元できるか？**

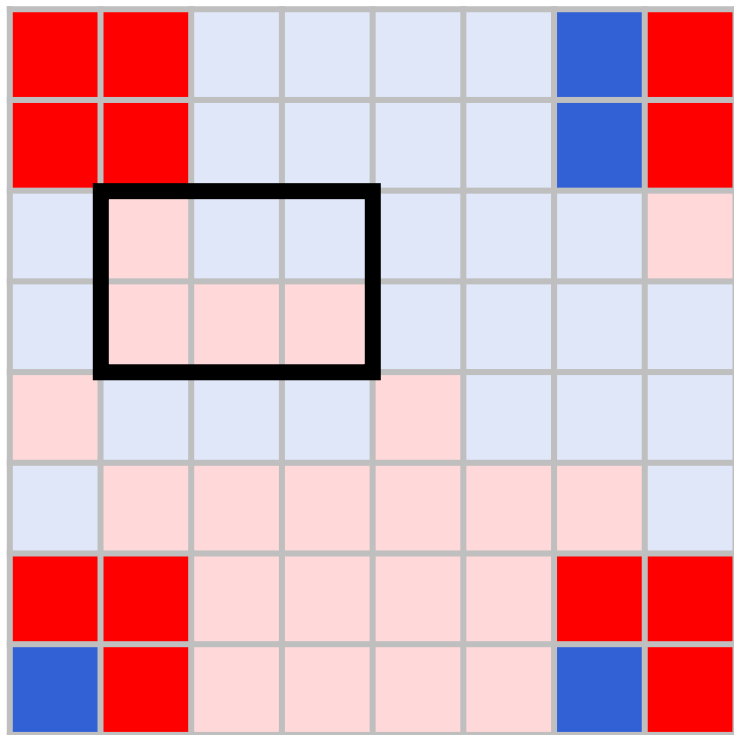
3

 $N = 36$ の解法 : 具体例 (Bruno) $\frac{77}{105}$ 

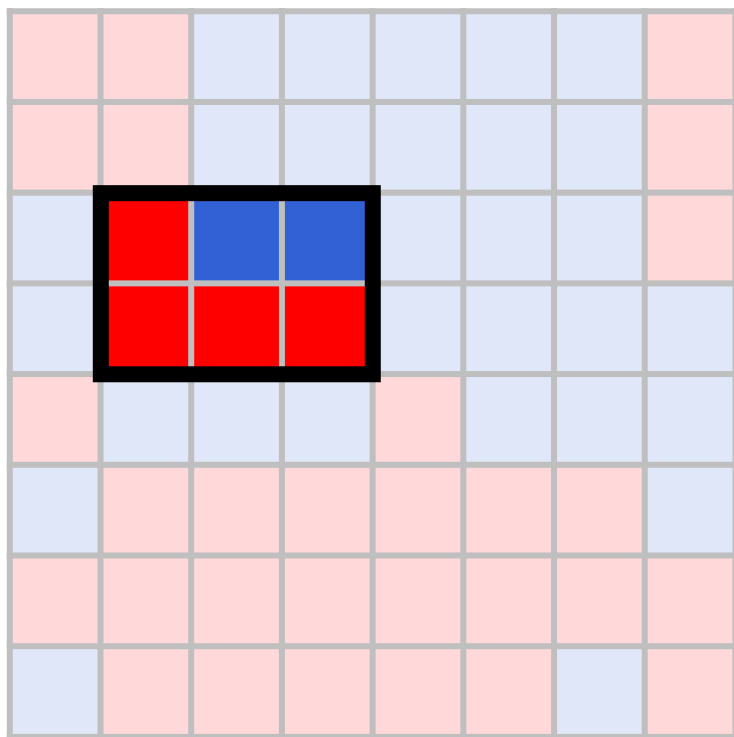
3

 $N = 36$ の解法 : 具体例 (Bruno)

78 / 105



まず、 2×2 の隅の中で全部赤のものは左上だけ
→太枠の中に X, Y の情報が書かれている



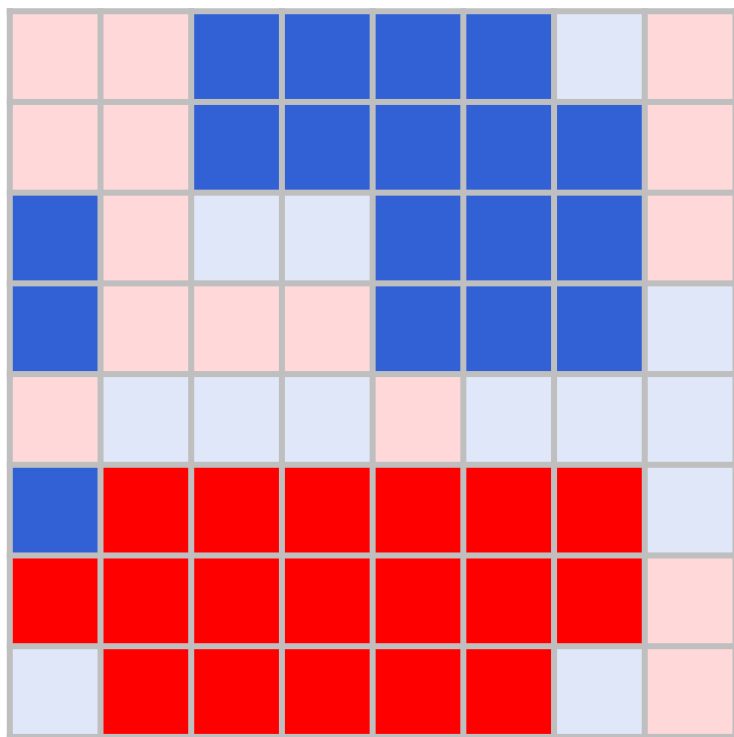
まず、 2×2 の隅の中で全部赤のものは左上だけ
→太枠の中に X, Y の情報が書かれている

情報を読むと、 $(X, Y) = (4, 7)$

3

 $N = 36$ の解法：具体例 (Bruno)

80 / 105



まず、 2×2 の隅の中で全部赤のものは左上だけ
→太枠の中に X, Y の情報が書かれている

情報を読むと、 $(X, Y) = (4, 7)$

そして (X, Y) が決まれば、文字列の情報がどこに
書かれているかがわかる

→答え「AAAA...ABBBB...B」が復元できる



36 文字を正しく

答えることができた！

まず、 2×2 の隅の中で全部赤のものは左上だけ

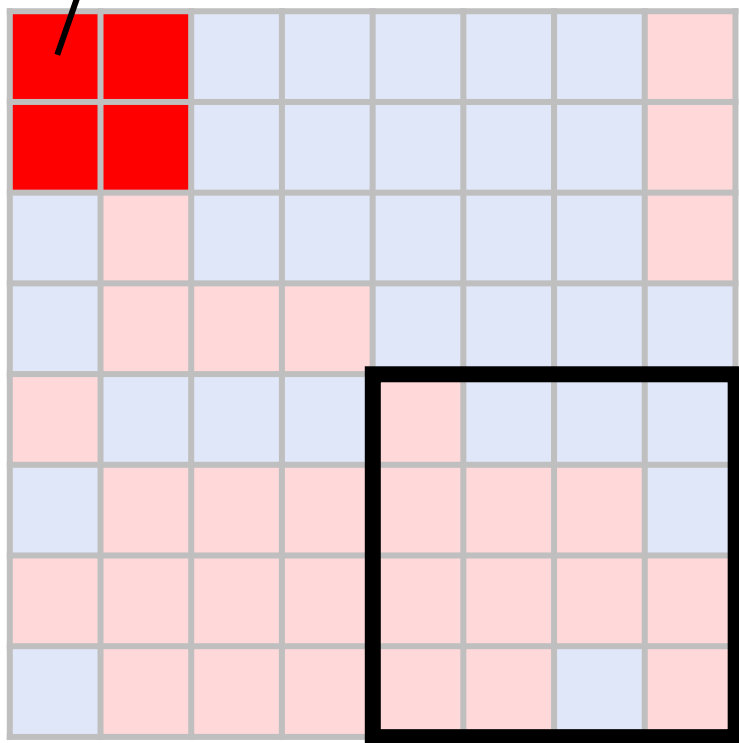
→大枠の中に X, Y の情報が書かれている

そして、 1 が残ったのは 1 文字列の情報がどこに
書かれているかがわかる

→答え「AAAA...ABBBB...B」が復元できる

この方法を少し工夫すると
 $N = 38$ でも解ける

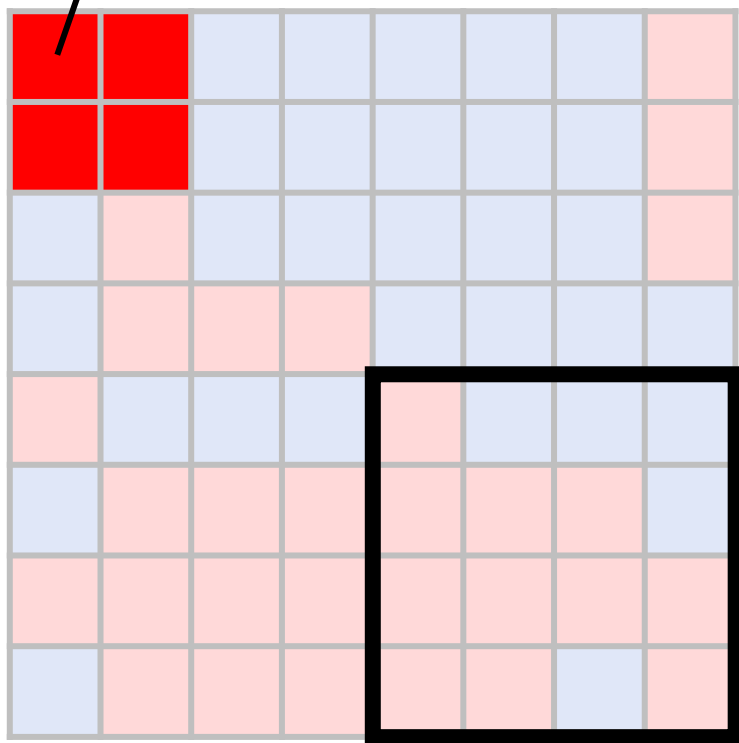
2×2 隅



元々は、 (X, Y) を記録するのに 6 マス使ったが...

2×2 隅が全部赤になっている部分がわかれば
 (X, Y) は太枠の 16 通りに絞れる

2×2 隅



元々は、 (X, Y) を記録するのに 6 マス使ったが...

2×2 隅が全部赤になっている部分がわかれば
 (X, Y) は太枠の 16 通りに絞れる



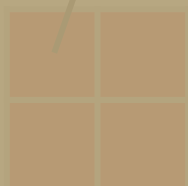
(X, Y) の記録が 4 マスで済み
2 文字分多く記録できる！

3

$N = 38$ の解法

85 / 105

2×2 隅



68 点獲得！

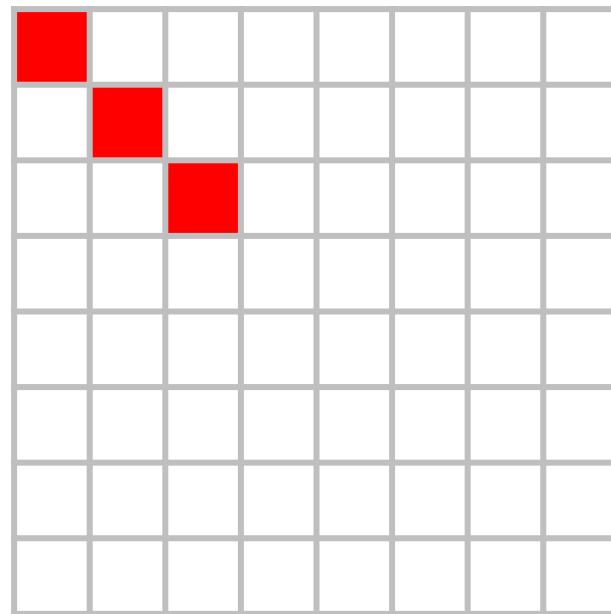
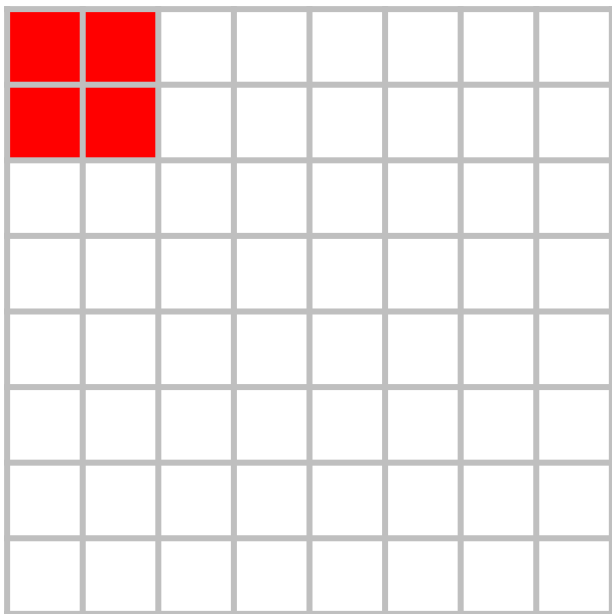
元々は、 (X, Y) を記録するのに 6 マス使ったが...

2×2 隅が全部赤になっている部分がわかれば

(X, Y) は太枠の 16 通りに絞れる

(X, Y) の記録が 4 マスで済み

2 文字分多く記録できる！

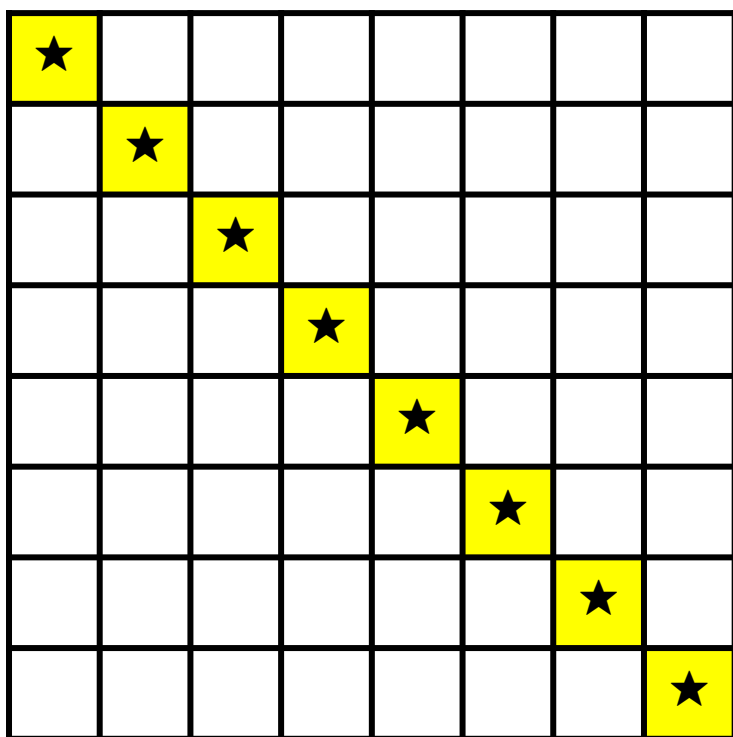


実は、識別は 2×2 隅ではなく斜め 3 つでも出来る

→ 1 マス減るので $N = 39$ が解ける (71 点)

Stage 4

アイデア：対角線を活用する



対角線の 8 マスだけで

(X, Y) の情報を読み取ることができる

×	×	×	×	×	×	×	×
×	★						
×		★					
×			★				
×				★			
×					★		
×						★	
×							★

42 マス使える

対角線の 8 マスだけで

 (X, Y) の情報を読み取ることを考える

もしこれが可能であれば...

- $X = Y$ のとき 42 文字分記録できる
- $X \neq Y$ のとき 43 文字分記録できる

★	×						
	×						
	×	★					
	×		★				
	×			★			
×	×	×	×	×	×	×	×
	×					★	
	×						★

43 マス使える

対角線の 8 マスだけで

(X, Y) の情報を読み取ることを考える

もしこれが可能であれば...

- $X = Y$ のとき 42 文字分記録できる
- $X \neq Y$ のとき 43 文字分記録できる

つまり、対角線だけで (X, Y) を読めれば $N = 42$ で解ける

でも、本当にできるのか？

×							
×	×	×	×	×	×	×	×
×		★					
×			★				
×				★			
×					★		
×						★	
×							★

$X \neq Y$ の場合、★の部分は決められるが

対角線の 2 マスはビ太郎が塗る

→対角線の色の並びとしては $2^2=4$ 通りがあり得る

×							
×	×	×	×	×	×	×	×
×		赤					
×			赤				
×				青			
×					青		
×						赤	
×							青

$X \neq Y$ の場合、★の部分は決められるが
対角線のうち 2 マスはビ太郎が塗る

→対角線の色の並びとしては $2^2=4$ 通りがあり得る

例：★印を「赤赤青青赤青」にする場合…

赤赤赤赤青青赤青、赤青赤赤青青赤青、

青赤赤赤青青赤青、青青赤赤青青赤青の 4 つ

★			×				
	★		×				
		★	×				
×	×	×	×	×	×	×	×
			×	★			
			×		★		
			×			★	
			×				★

$X = Y$ の場合、★の部分は決められるが

対角線のうち 1 マスはピ太郎が塗る

→対角線の色の並びとしては $2^1=2$ 通りがあり得る

つまり、対角線の色の並びとしては（重複含め）

- $X \neq Y$ の場合 : $56 \times 4 = 224$ 通り
- $X = Y$ の場合 : $8 \times 2 = 16$ 通り

240 通りが存在する！

そして、この並びは全部異ならなければならない※
つまり $2^8=256$ 通りの中に 240 通りを埋め込む必要がある

こんなこと出来るのか？

※もし対角線の色の並びに同じものがあった場合、 (X, Y) を正しく読み取ることができない

そして、この並びは全部異ならなければならない※

つまり $2^8=256$ 通りの中に 240 通りを埋め込む必要がある

焼きなまし法を使えば、構成できる

こんなこと出来るのか？

※もし対角線の色の並びに同じものがあった場合、 (X, Y) を正しく読み取ることができない

×	×	×	×	×	×	×	×
×	★						
×		★					
×			★				
×				★			
×					★		
×						★	
×							★

42 マス使える

$X = Y$ の場合、対角線以外で
Anna が書けるマスが 42 個しかない

このままでは 42 文字しか記録できないので
対角線に X, Y だけでなく「最後の 1 文字」も
記録することを考える

すると、対角線の色の並びの数は…

- $X \neq Y$ の場合 : $56 \times 4 = 224$ 通り
- $X = Y$ の場合 : $8 \times 2 \times 2 = 32$ 通り

の合計 256 通りになる !

そこで、 $2^8=256$ 通りの中に 256 通りを埋め込むことは可能か？

一見厳しそうに思えるが...

焼きなまし法を高速化すれば
構成できる！

※解説担当者の方針では、1億～10億回程度のループが必要になるので
数分以内に実行を終わらせるには、スコア計算を $O(1)$ 時間で終わらせる必要がある

4

$N = 43$ の解法

102 / 105

100点獲得！

解法のまとめ

Stage 1

赤の個数で文字列を識別

Stage 2

区画ごとに分けて多数決をとる

Stage 3

Bruno が x, y の値を読み取れるようにする

Stage 4

対角線に x, y を記録 + 焼きなまし法で埋め込み法を構成

